Cloud Computing

From Basic Concepts to Fog and Edge Computing

Modules 1–4

October 14, 2026

Contents

1 Cloud Computing **1**

1.1 Let's Start With Two Simple Questions . . . 1
1.2 What Is Cloud Computing? . . . 1
1.3 First, Understand the Word "Cloud" . . . 2
1.4 Real-Life Example 1 — Google Drive . . . 2
1.5 Real-Life Example 2 — Gmail . . . 2
1.6 Why Do We Need Cloud Computing? . . . 3
1.7 Advantages of Cloud Computing . . . 3
1.8 Disadvantages of Cloud Computing . . . 4
1.9 Popular Cloud Services We Use Every Day . . . 4
1.10 Recent Trends in Cloud Computing . . . 4
1.10.1 Trends 1–5 . . . 4
1.10.2 Trends 6–10 . . . 5
1.11 The Evolution of Cloud Computing . . . 5
1.11.1 A 70-Year Journey . . . 5
1.11.2 Mainframe Computing (1950 – 1970) . . . 6
1.11.3 Distributed Systems (1970 – 1980) . . . 6
1.11.4 Cluster Computing (1980 – 1990) . . . 6
1.11.5 Grid Computing (1990 – 2000) . . . 7
1.11.6 Utility Computing (Late 1990s – 2000s) . . . 7
1.11.7 Virtualization (1980 – Present) . . . 8
1.11.8 Web 2.0 (Gained Popularity ∼2004) . . . 8
1.11.9 Service Orientation (2000s) . . . 8
1.11.10 All Roads Lead to the Cloud . . . 9
1.12 Public vs Private vs Hybrid Cloud . . . 9
1.12.1 Three Deployment Models, One Decision . . . 9
1.12.2 Public Cloud — Owned & Shared by a Provider . . . 10
1.12.3 Private Cloud — Dedicated to One Organization . . . 10
1.12.4 Hybrid Cloud — The Best of Both Worlds . . . 11
1.12.5 Head-to-Head Comparison . . . 11
1.12.6 Which One Should You Choose? . . . 12
1.12.7 Quick Revision Summary . . . 12
1.13 Categories of Cloud Computing: Everything as a Service . . . 13
1.13.1 Infrastructure as a Service (IaaS) . . . 13
1.13.1.1 What Does IaaS Provide? . . . 13
1.13.1.2 Examples of IaaS . . . 14
1.13.1.3 Advantages of IaaS . . . 14
1.13.1.4 Simple Example . . . 14
1.13.2 Platform as a Service (PaaS) . . . 14
1.13.2.1 What Does PaaS Provide? . . . 14

1.13.2.2 Examples of PaaS . . . 15
1.13.2.3 Advantages of PaaS . . . 15
1.13.2.4 Simple Example . . . 15
1.13.3 Software as a Service (SaaS) . . . 15
1.13.3.1 Examples of SaaS . . . 16
1.13.3.2 Advantages of SaaS . . . 16
1.13.3.3 Simple Example . . . 16
1.13.4 Security as a Service (SECaaS) . . . 16
1.13.4.1 What Can SECaaS Provide? . . . 17
1.13.4.2 Advantages of SECaaS . . . 17
1.13.4.3 Simple Example . . . 17
1.14 Understanding the Difference Between IaaS, PaaS and SaaS . . . 18
1.15 An Easy Way to Remember IaaS, PaaS and SaaS . . . 19
1.16 Quick Revision: Cloud Service Models . . . 19
1.17 Pros and Cons of Cloud Computing . . . 20
1.17.1 Advantages of Cloud Computing . . . 20
1.17.1.1 Cost Reduction . . . 20
1.17.1.2 Scalability . . . 20
1.17.1.3 Accessibility . . . 20
1.17.1.4 Flexibility . . . 21
1.17.1.5 High Availability . . . 21
1.17.1.6 Backup and Disaster Recovery . . . 21
1.17.1.7 Automatic Updates . . . 21
1.17.1.8 Faster Deployment . . . 21
1.17.2 Disadvantages of Cloud Computing . . . 21
1.17.2.1 Internet Dependency . . . 21
1.17.2.2 Security and Privacy Risks . . . 22
1.17.2.3 Vendor Lock-In . . . 22
1.17.2.4 Limited Control . . . 22
1.17.2.5 Downtime . . . 22
1.17.2.6 Unpredictable Costs . . . 23
1.17.2.7 Performance Issues . . . 23
1.17.2.8 Compliance and Legal Issues . . . 23
1.17.3 Advantages and Disadvantages at a Glance . . . 23
1.17.4 Simple Example . . . 23
1.17.4.1 Traditional Approach . . . 24
1.17.4.2 Cloud Approach . . . 24
1.18 What Is Virtualization? . . . 24
1.19 Levels of Virtualization Implementation . . . 25
1.19.1 Instruction Set Architecture (ISA) Level . . . 25
1.19.2 Hardware Abstraction Level (HAL) . . . 26
1.19.3 Operating System Level . . . 26
1.19.4 Library Support Level . . . 26
1.19.5 User-Application Level . . . 27
1.19.6 Relative Merits of Different Virtualization Levels . . . 27
1.20 VMM Design Requirements and Providers . . . 27
1.21 Virtualization Support at the OS Level . . . 29
1.21.1 Why OS-Level Virtualization? . . . 29
1.21.2 Advantages of OS Extensions . . . 29
1.21.3 Disadvantages of OS Extensions . . . 30
1.21.4 Virtualization on Linux and Windows Platforms . . . 31

1.22 Middleware Support for Virtualization 32
1.23 Virtualization Structures, Tools, and Mechanisms 33
1.23.1 Hypervisor and Xen Architecture 33
1.23.2 Para-Virtualization with Compiler Support 34
1.24 Virtualization of CPU, Memory, and I/O Devices 34
1.24.1 Hardware Support for Virtualization 34
1.24.2 CPU Virtualization 35
1.24.3 Memory Virtualization 36
1.24.4 I/O Virtualization 37
1.24.5 Virtualization in Multi-Core Processors 41
1.25 Desktop Virtualization 41
1.25.1 What Is Desktop Virtualization? 41
1.25.2 Virtual Desktop Infrastructure (VDI) 42
1.25.3 VDI Access Flow: Local vs. Remote Users 42
1.25.4 VDI in the Real World 42
1.25.5 Popular Virtualization / VDI Software 44
1.26 Virtualization for Data Center Automation: The Big Picture 45
1.26.1 Start With a Simple Problem 45
1.26.2 What Does “Automation” Mean Here? 45
1.26.3 Our Roadmap 46
1.27 Virtual Clusters 46
1.27.1 First, What Is a Cluster? 46
1.27.1.1 Why Do We Use Clusters? 47
1.27.2 What Is a Physical Cluster? 48
1.27.3 Why Do We Need Virtual Clusters? 48
1.27.4 Definition of a Virtual Cluster 49
1.27.5 Physical Cluster vs. Virtual Cluster 50
1.27.6 Virtual Cluster Architecture 50
1.27.7 How a Virtual Cluster Is Created (Dynamic Provisioning) 51
1.27.8 Real-Life Example: An Online Shopping Website 52
1.27.9 Template VMs: Make One, Copy Many 52
1.27.10 VM Image Management 53
1.28 Resource Management in the Cloud 53
1.28.1 What Is Resource Management? 53
1.28.2 Why Is It Needed? 54
1.28.3 Resource Management Architecture 55
1.28.4 Main Functions of Resource Management 55
1.28.5 Worked Example 1: Resource Allocation 56
1.28.6 Worked Example 2: Resource Scheduling 56
1.28.7 Load Balancing 57
1.28.8 Server Consolidation 58
1.29 VM Migration 58
1.29.1 What Is VM Migration? 58
1.29.2 Why Do We Migrate VMs? 58
1.29.3 Live VM Migration 59
1.29.4 Live Migration Steps 59
1.29.5 Worked Example of Live Migration 59
1.29.6 Migration Performance Factors 60
1.29.7 Memory Migration 60
1.29.7.1 Iterative Memory Transfer (Pre-Copy) 61
1.29.8 File System Migration 61

1.29.9 Network Migration . . . 62
1.30 Dynamic Resource Allocation and Elasticity . . . 62
1.30.1 Dynamic Resource Allocation . . . 62
1.30.2 The Resource Management Decision Cycle . . . 62
1.30.3 Example: A University Cloud . . . 63
1.30.4 Static vs. Dynamic Allocation . . . 64
1.30.5 Summary of Resource Management Techniques . . . 64
1.30.6 Fault Tolerance . . . 64
1.31 Virtual Cluster Management Models . . . 65
1.31.1 Guest-Based vs. Host-Based Management . . . 65
1.32 Advantages and Challenges of Virtual Clusters . . . 66
1.33 Putting It All Together . . . 66
1.33.1 Complete Resource Management Flow . . . 66
1.33.2 End-to-End Example: A Machine-Learning Experiment . . . 67
1.33.3 Another Example: An E-Commerce Company . . . 67
1.34 Virtualization for Data-Center Automation . . . 67
1.34.1 What Is Data-Center Automation? . . . 67
1.34.1.1 Market Trend (A Little History) . . . 68
1.34.1.2 The Four Topics We Will Study . . . 68
1.34.2 Server Consolidation in Data Centers . . . 68
1.34.2.1 The Problem: Two Kinds of Workloads . . . 69
1.34.2.2 The Solution: Server Consolidation . . . 69
1.34.2.3 Benefits of Virtualization-Based Consolidation . . . 70
1.34.2.4 What Must Be Considered to Automate a Data Center? . . . 70
1.34.2.5 Scheduling at Three Levels . . . 70
1.34.2.6 Dynamic CPU Allocation and the Two-Level Controller . . . 71
1.34.2.7 Multicore Processors and Power Budgeting . . . 71
1.34.3 Virtual Storage Management . . . 71
1.34.3.1 What Does “Storage Virtualization” Mean Here? . . . 71
1.34.3.2 Encapsulation and Isolation . . . 72
1.34.3.3 Why Is Storage So Difficult to Manage? . . . 72
1.34.3.4 Two Ideas: CAS and Parallax . . . 72
1.34.3.5 Example: How Parallax Works . . . 73
1.34.4 Cloud OS for Virtualized Data Centers . . . 73
1.34.4.1 What Is a Cloud OS? . . . 73
1.34.4.2 Four Well-Known VI Managers . . . 74
1.34.4.3 Example 1: Eucalyptus . . . 74
1.34.4.4 Example 2: VMware vSphere 4 (a Commercial Cloud OS) . . . 75
1.34.5 Trust Management in Virtualized Data Centers . . . 76
1.34.5.1 Why Does Virtualization Bring New Security Questions? . . . 76
1.34.5.2 A Subtle Problem: Rolling Back a VM . . . 77
1.34.5.3 VM-Based Intrusion Detection . . . 77
1.34.5.4 Honeypots and Honeynets . . . 78
1.34.5.5 Example: Trusted Zones for Virtual Clusters (EMC and VMware) 78
1.34.6 Putting the Four Topics Together . . . 79
1.34.7 Likely Exam Questions and Short Answers . . . 79
1.34.8 Quick Revision Summary . . . 80
1.34.9 The Big Question: Where Does Your Request Run? . . . 80
1.35 Practice Section . . . 81
1.35.1 Group Problem: Where Should the VM Go? . . . 81
1.35.2 Check Your Understanding . . . 81

1.35.3 Exam Preparation 82

2 Cloud Computing Architecture and Management 83
2.1 Let's Start With a Simple Question 83
2.2 What Is Cloud Computing Architecture? 84
2.2.1 The Front End 84
2.2.2 The Back End 85
2.2.3 The Network (Internet) 85
2.3 How a Request Travels: Opening an Email 85
2.4 The Layered View of Cloud Architecture 86
2.5 Middleware and APIs — The "Glue" of the Cloud 87
2.5.1 Middleware 87
2.5.2 APIs (Application Programming Interfaces) 87
2.6 Who Is Who in the Cloud? — Cloud Roles 88
2.6.1 What Does a Cloud Broker Do? 88
2.6.2 Responsibilities of Providers and Users 89
2.7 Cloud Service Models: IaaS, PaaS and SaaS 89
2.7.1 What Is a Cloud Service Model? 89
2.7.2 Infrastructure as a Service (IaaS) 90
2.7.3 Platform as a Service (PaaS) 90
2.7.4 Software as a Service (SaaS) 91
2.7.5 Who Manages What? 92
2.7.6 An Easy Way to Remember: Getting Around Town 93
2.7.7 Head-to-Head Comparison 93
2.7.8 More "as a Service" Offerings 94
2.7.9 Choosing the Right Service Model 94
2.7.9.1 Checklist for Choosing a Model 94
2.7.9.2 Which Model Fits? — Common Situations 95
2.7.10 Using All Three Together: An Online Business 95
2.7.11 Traditional Software vs SaaS 96
2.8 Service Management in Cloud Computing 96
2.8.1 What Is Service Management? 96
2.8.2 The Service Management Cycle 97
2.8.2.1 Service Catalog 97
2.8.2.2 Service Provisioning 97
2.8.2.3 Service Monitoring 97
2.8.2.4 Service Metering and Billing 97
2.8.2.5 Support, Review and Improvement 98
2.8.3 Service Level Agreement (SLA) 98
2.8.3.1 What Does "99.9% Uptime" Really Mean? 98
2.8.3.2 Features of a Good SLA 98
2.8.3.3 Checklist Before Signing an SLA 99
2.8.4 Quality of Service (QoS) 99
2.8.5 Scalability and Elasticity 100
2.8.6 Why Is Service Management So Important? 100
2.9 Data Management in Cloud Computing 100
2.9.1 What Is Data Management? 100
2.9.2 Kinds of Data 101
2.9.3 The Data Lifecycle 101
2.9.4 Data Replication, Redundancy and Consistency 102
2.9.5 Backup, Recovery and Archiving 102
2.9.6 High Availability and Disaster Recovery 103

2.9.7 Data Security and Privacy . . . 104
2.9.7.1 Data Encryption . . . 104
2.9.7.2 Access Control . . . 104
2.9.7.3 Data Integrity . . . 104
2.9.7.4 Data Breach . . . 105
2.9.8 Data Governance . . . 105
2.9.9 Who Is Responsible for the Data? . . . 106
2.10 Resource Management in Cloud Computing . . . 106
2.10.1 What Is Resource Management? . . . 106
2.10.2 Physical and Virtual Resources . . . 107
2.10.3 Resource Pooling . . . 107
2.10.4 The Main Functions of Resource Management . . . 108
2.10.4.1 Resource Allocation . . . 108
2.10.4.2 Resource Scheduling . . . 108
2.10.4.3 Load Balancing . . . 109
2.10.4.4 Auto-Scaling . . . 109
2.10.4.5 Resource and Performance Monitoring . . . 110
2.10.4.6 Cost Management . . . 110
2.10.5 Risks of Poor Resource Management . . . 110
2.11 Cloud Implementation . . . 111
2.11.1 What Is Cloud Implementation? . . . 111
2.11.2 Steps of Cloud Implementation . . . 111
2.11.3 Capacity Planning . . . 111
2.11.4 Cloud Migration . . . 112
2.11.5 Testing Before Going Live . . . 112
2.11.6 Deployment and Rollback . . . 113
2.11.7 Example Implementation Plans . . . 113
2.11.7.1 A School Moving Its Records From Paper to the Cloud . . . 113
2.11.7.2 A Retail Company Moving Its Inventory System Online . . . 113
2.12 Putting It All Together . . . 114
2.13 Quick Revision Summary . . . 114
2.14 Likely Exam Questions and Short Answers . . . 115
2.15 Practice Section . . . 116
2.15.1 Group Problem: Choose, Size and Check . . . 116
2.15.2 Check Your Understanding . . . 117
2.15.3 Exam Preparation . . . 117

3 Fog and Edge Computing 118
3.1 Let's Start With a Simple Question . . . 118
3.2 What Is Edge Computing? . . . 119
3.3 Real-Life Example 1 — A Smart Doorbell Camera . . . 120
3.4 Real-Life Example 2 — A Self-Driving Car . . . 120
3.5 Why Do We Need Edge Computing? . . . 121
3.6 Important Words You Must Know . . . 121
3.6.1 Worked Example: How Much Data Can the Edge Save? . . . 122
3.7 What Is Fog Computing? . . . 122
3.7.1 Edge vs Fog — What Is the Difference? . . . 123
3.8 Cloud vs Fog vs Edge — Head-to-Head Comparison . . . 123
3.9 The Three-Tier Architecture: Cloud, Fog and Edge . . . 124
3.9.1 Following the Data: A Smartwatch Example . . . 124
3.9.2 Many Edge Nodes, One Fog Node . . . 125
3.10 Relevant Technologies . . . 125

3.10.1 The Big Picture . . . 125
3.10.2 Internet of Things (IoT) . . . 126
3.10.3 Wireless Sensor Networks (WSN) . . . 126
3.10.4 Machine-to-Machine Communication (M2M) . . . 127
3.10.5 Cyber-Physical Systems (CPS) . . . 127
3.10.6 Embedded Systems . . . 127
3.10.7 From Mobile Computing to Mobile Edge Computing . . . 127
3.10.8 5G Networks . . . 128
3.10.9 SDN and NFV . . . 128
3.10.10 Content Delivery Networks (CDN) . . . 128
3.10.11 Quick Summary of Relevant Technologies . . . 129
3.11 Fog and Edge Computing Completing the Cloud . . . 129
3.11.1 What Does "Completing the Cloud" Mean? . . . 129
3.11.2 An Easy Analogy: Touching a Hot Stove . . . 130
3.11.3 Two Big Questions: SCALE and SCANC . . . 130
3.12 SCALE — The Five Advantages of FEC . . . 131
3.12.1 S — Security . . . 131
3.12.2 C — Cognition . . . 132
3.12.3 A — Agility . . . 132
3.12.4 L — Latency . . . 133
3.12.5 E — Efficiency . . . 133
3.12.6 Compare All Five SCALE Advantages . . . 134
3.13 SCANC — How Fog and Edge Computing Works . . . 134
3.13.1 The FEC Node . . . 134
3.13.2 S — Storage . . . 135
3.13.3 C — Compute . . . 136
3.13.3.1 I/PaaS — Bring Your Own Application . . . 136
3.13.3.2 SaaS — Use a Ready-Made Service . . . 136
3.13.4 A — Acceleration . . . 137
3.13.4.1 Networking Acceleration . . . 137
3.13.4.2 Computing Acceleration . . . 137
3.13.5 N — Networking . . . 138
3.13.5.1 Vertical Networking . . . 138
3.13.5.2 Horizontal Networking . . . 138
3.13.6 C — Control . . . 139
3.13.7 Quick Revision: SCALE and SCANC . . . 139
3.14 Putting It All Together . . . 140
3.14.1 Example 1: A Smart Traffic Junction . . . 140
3.14.2 Example 2: A Smart Hospital . . . 140
3.15 Advantages and Disadvantages of Edge Computing . . . 140
3.16 Where Is Edge Computing Used? . . . 141
3.16.1 When Should You Choose the Edge, and When the Cloud? . . . 141
3.17 Hierarchy of Fog and Edge Computing . . . 141
3.17.1 What Does "Hierarchy" Mean? . . . 141
3.17.2 The Simple Three-Layer View . . . 142
3.17.3 The Book's View: Inner, Middle and Outer Edge . . . 142
3.17.3.1 Inner Edge . . . 142
3.17.3.2 Middle Edge . . . 143
3.17.3.3 Outer Edge . . . 143
3.18 Business Models . . . 143
3.18.1 First, Some Basic Words . . . 143

3.18.2 The Three Business Models of FEC . . . 144
3.18.2.1 X as a Service (XaaS) . . . 144
3.18.2.2 Support Services . . . 144
3.18.2.3 Application Services . . . 144
3.18.3 Edge as a Service . . . 144
3.18.4 Pay-Per-Use vs Subscription . . . 145
3.18.5 The Edge Computing Ecosystem . . . 145
3.18.6 Monetizing Edge Data . . . 145
3.19 Edge Computing Platforms . . . 146
3.19.1 What Is an Edge Computing Platform? . . . 146
3.19.2 What Does a Platform Do? . . . 146
3.19.3 Orchestration: One Controller, Many Devices . . . 147
3.19.4 Platforms Named in the Book . . . 147
3.19.5 Major Platforms Used Today . . . 148
3.19.6 Checklist: Choosing a Platform . . . 148
3.20 Opportunities and Challenges . . . 148
3.20.1 Opportunities . . . 148
3.20.2 Challenges . . . 149
3.20.3 How Can We Handle These Challenges? . . . 150
3.20.4 The Book's Three Big Challenges . . . 150
3.20.5 Opportunities and Challenges at a Glance . . . 151
3.21 Quick Revision Summary . . . 151
3.21.1 The Big Question: Where Should This Task Run? . . . 152
3.22 Likely Exam Questions and Short Answers . . . 152
3.23 Practice Section . . . 153
3.23.1 Group Problem: Where Should Each Task Run? . . . 153
3.23.2 Check Your Understanding . . . 154
3.23.3 Exam Preparation . . . 154

4 Federating Edge Resources and Optimization in Fog Computing 155
4.1 Let's Start With Two Simple Questions . . . 155
4.2 What Does "Federating Edge Resources" Mean? . . . 156
4.3 Challenges in Federating Edge Resources . . . 157
4.3.1 The Networking Challenge . . . 157
4.3.1.1 From "Where" to "What": The Service-Centric Model . . . 157
4.3.1.2 Multiple Administrative Domains . . . 158
4.3.2 Addressing the Networking Challenge: Software-Defined Networking (SDN) 158
4.3.2.1 How SDN Helps a Federated Edge . . . 158
4.3.2.2 Many Controllers for Many Domains . . . 159
4.3.2.3 Future Research Directions (Networking) . . . 159
4.3.3 The Management Challenge . . . 159
4.3.3.1 What Current Research Offers . . . 160
4.3.3.2 Example 1: The Edge-as-a-Service (EaaS) Platform . . . 160
4.3.3.3 Example 2: The ENORM Framework . . . 161
4.3.3.4 Future Research Directions (Management) . . . 161
4.3.4 Miscellaneous Challenges: Resources and Modeling . . . 161
4.3.4.1 Resource Challenges . . . 161
4.3.4.2 Modeling Challenges . . . 162
4.4 The Case for Optimization in Fog Computing . . . 162
4.4.1 First, What Is Optimization? . . . 162
4.4.2 Why Does Fog Computing Need Optimization? . . . 163
4.4.3 A Worked Example: To Offload or Not? . . . 163

4.5 Formal Modeling Framework for Fog Computing . . . 164
4.5.1 Why Do We Need a Formal Model? . . . 164
4.5.2 The Fog Infrastructure as Layers . . . 164
4.5.3 The Building Blocks of the Model . . . 165
4.5.4 The Optimization Problem in Words and Symbols . . . 165
4.5.5 Worked Example: Placing Three Components . . . 166
4.6 Metrics . . . 166
4.6.1 Performance . . . 166
4.6.2 Resource Usage . . . 167
4.6.3 Energy Consumption . . . 167
4.6.4 Financial Cost . . . 167
4.7 Further Quality Attributes . . . 168
4.8 Optimization Opportunities along the Fog Architecture . . . 169
4.8.1 Inside Each Layer . . . 169
4.8.2 Between Layers . . . 170
4.9 Optimization Opportunities along the Service Life Cycle . . . 170
4.9.1 Design Time . . . 170
4.9.2 Deployment Time . . . 171
4.9.3 Run Time . . . 171
4.10 How Are Fog Optimization Problems Solved? . . . 172
4.11 Putting It All Together . . . 172
4.12 Quick Revision Summary . . . 173
4.13 Likely Exam Questions and Short Answers . . . 174
4.14 Practice Section . . . 175
4.14.1 Group Problem: Where Should the Video Analysis Run? . . . 175
4.14.2 Check Your Understanding . . . 176
4.14.3 Exam Preparation . . . 176

References and Further Reading **177**

Chapter 1

Cloud Computing

1.1 Let's Start With Two Simple Questions

Think about this for a second:

1. How many of us use Google Drive, Gmail, YouTube, or WhatsApp every day?
2. Where do you think your photos, emails, and videos are actually stored? Inside your phone?

The surprising answer: No, most of that data is *not* stored inside your phone. It is stored in huge, powerful computers called **servers**, which sit somewhere else in the world — maybe thousands of kilometers away. You simply access that data through the **Internet**.

This whole idea — storing and using data or software that lives on someone else's computer, accessed over the internet — is called **Cloud Computing**.

1.2 What Is Cloud Computing?

Core Concept

Cloud Computing means using computing services — like **storage, software, databases, and servers** — through the Internet, instead of relying on your own personal computer.

Even simpler way to think about it: Instead of buying and maintaining an expensive, powerful computer yourself, you **rent** computing resources whenever you need them, and you access them through the internet — just like renting a car instead of buying one.

The four main things you can "rent" from the cloud are:

- **Storage** – space to keep your files
- **Software** – applications you can use online
- **Servers** – powerful machines that do the processing

- **Databases** – organized places to store and manage data

1.3 First, Understand the Word "Cloud"

The word "cloud" is not magic — it simply means:

> A collection of powerful computers (servers) connected together via the Internet.

These servers don't sit in a random room — they live inside massive buildings called **Data Centers**. Key facts about data centers:

- They contain **thousands of servers**
- They are **powered 24 hours a day, 7 days a week** (they never really switch off)
- They are **heavily guarded and secured** (physically and digitally)
- They are **connected to the world via the Internet**

So basically, when people say "it's in the cloud," they really mean "it's saved on someone else's server, in some data center, and you can reach it over the internet."

1.4 Real-Life Example 1 — Google Drive

The scenario: Imagine your phone has:

- 5 GB of photos
- 10 GB of videos
- But only 2 GB of free storage space left

What do you do? Instead of deleting your precious files, you upload them to **Google Drive**.

What actually happens behind the scenes:

1. Your phone sends the files over the **Internet**
2. The files travel to a **Google Data Center**
3. Your files are now stored there — NOT inside your phone anymore

The best part? You can download or view these files anytime, from any device, as long as you have internet access.

1.5 Real-Life Example 2 — Gmail

Question: When you send an email to a friend, where does it actually get stored?

Answer: Not on your laptop. It gets stored on **Google's cloud servers**.

Why this matters: Even if your laptop breaks or gets stolen tomorrow, you can simply log into Gmail from any other computer or phone — and your email will still be there,

safe and sound. That's because your email was never really "on" your laptop; it always lived in the cloud.

Core Concept

Your data outlives your device — because your data was never stored on the device in the first place.

1.6 Why Do We Need Cloud Computing?

Imagine you're starting a brand-new company. To run your business digitally, you would need: Servers, Storage, a Database, and Software.

Now you have **two choices**:

Option	What it means
Buy everything yourself	You need to spend 20 Lakh or more upfront just to buy hardware — before you've even made a single sale
Rent from the cloud	You pay only for what you actually use, and you can scale up or down anytime

This is exactly why both small startups and huge enterprises prefer cloud computing — it removes the need for a massive upfront investment.

1.7 Advantages of Cloud Computing

Advantages

1. **Cost Saving** — You don't need to buy expensive hardware. A startup can simply rent servers instead of purchasing them, which saves a huge amount of money upfront.
2. **Access Anywhere** — You can access your files from home, college, office, or your mobile phone — the only requirement is an internet connection.
3. **Large Storage** — Need more space? Just upgrade your cloud storage plan. No need to physically buy and install a new hard disk.
4. **Automatic Backup** — Cloud providers automatically back up your data in the background, which greatly reduces the risk of losing it.
5. **High Security** — Big cloud companies (like Google, Amazon, Microsoft) invest heavily in protecting your data with strong safeguards — often much stronger than what an individual or small company could set up on their own.
6. **Easy Collaboration** — Multiple people can work on the same document at the same time — think of how Google Docs lets a whole team edit one file together, live.

1.8 Disadvantages of Cloud Computing

Disadvantages

1. **Internet dependency** — You need an internet connection to access your data. No internet means no access.
2. **Data privacy concerns** — You are trusting a third-party company with your personal or business data.
3. **Ongoing costs** — Unlike a one-time hardware purchase, cloud services usually involve continuous monthly subscription fees.
4. **Service outages** — If the cloud provider's servers go down, you may temporarily lose access to your own data.

1.9 Popular Cloud Services We Use Every Day

Service	Category
Gmail	Email
Google Drive	Storage
Dropbox / OneDrive	Storage
YouTube / Netflix	Video Streaming
Spotify	Music Streaming
Google Photos	Photo Backup

Notice something? We already use cloud computing constantly in daily life — we just don't always realize it.

1.10 Recent Trends in Cloud Computing

As more and more businesses and individuals adopt cloud computing, big providers like **Google, Microsoft, IBM, and Amazon** keep innovating to make their services faster, safer, and smarter. Here are the top 10 current trends.

1.10.1 Trends 1–5

1. **AI & Machine Learning** — Cloud platforms are becoming self-learning and personalized, with AI/ML also helping strengthen data privacy.
2. **Data Security** — Stronger encryption, authentication, and recovery systems to minimize data breaches and data loss.
3. **Multi & Hybrid Cloud** — Businesses spread their workloads across multiple cloud providers to reduce risk and cut costs.
4. **Low-Code / No-Code** — Tools like Figma and Zoho let people build apps and websites quickly, without deep coding knowledge.

5. **Edge Computing** — Data is processed closer to where it is generated (instead of a faraway server) to reduce delay — this is helped by the rise of 5G networks.

1.10.2 Trends 6–10

6. **IoT (Internet of Things)** — Sensors and smart devices generate massive amounts of data, which is stored and analyzed using cloud servers.
7. **Kubernetes & Docker** — Tools that automate the scaling, deployment, and management of software containers for large applications.
8. **Serverless Computing** — A pay-as-you-go model where you don't have to manage servers yourself; the system scales automatically based on demand.
9. **DevSecOps** — Security is built directly into the software development process, to defend against cloud-based cyberattacks.
10. **Disaster Recovery** — Cloud backups help protect important data from outages, cyberattacks, and hardware failures.

Other emerging trends to keep an eye on: Blockchain, Cloud Gaming, and Quantum Computing.

1.11 The Evolution of Cloud Computing

From Mainframes in the 1950s to the Modern Cloud — 8 Generations of Innovation

1.11.1 A 70-Year Journey

Cloud computing did not appear overnight — it evolved slowly over roughly 70 years. Each new generation of technology fixed the biggest weakness of the previous one, and eventually, all these ideas combined into what we now call "the cloud."

The 8 generations, in order, are shown below.

#	Generation	Time Period
1	Mainframe Computing	1950 – 1970
2	Distributed Computing	1970 – 1980
3	Cluster Computing	1980 – 1990
4	Grid Computing	1990 – 2000
5	Utility Computing	Late 1990s – 2000s
6	Virtualization	1980 – Present
7	Web 2.0	∼2004
8	Service Orientation	2000s

Let's go through each one, one at a time.

1.11.2 Mainframe Computing (1950 – 1970)

Mainframes were the very first "big iron" computers — extremely powerful and reliable machines, built to handle huge amounts of input and output data with almost zero downtime.

The problem: They pushed computing power far beyond what was previously possible, but they were **so expensive** that only large institutions (banks, governments, airlines) could afford them.

Real-world examples:

- **Banking Systems** — used for overnight batch-processing of millions of transactions
- **SABRE Airline System** — one of the first mainframe-powered flight reservation systems
- **Government Census** — large-scale data processing for national records

What happened next: This cost problem is exactly what pushed the industry toward finding a cheaper alternative — cluster computing.

1.11.3 Distributed Systems (1970 – 1980)

Distributed systems combined multiple independent computers that worked together but appeared to the user as a single system.

What they introduced:

- Scalability (could grow bigger)
- Concurrency (multiple things happening at once)
- Continuous availability (did not fully stop working if one part failed)
- Independence from failures
- Machines did not need to sit in the same physical location — they simply needed to communicate over a network

Real-world examples:

- **ARPANET** — an early academic network linking computers that were geographically far apart (this eventually grew into the modern internet)
- **File-Sharing Networks** — multiple machines sharing resources as though they were one logical system
- **Multi-Site Research Labs** — universities pooling computing power across different campuses

What happened next: This model later evolved further into cluster and grid computing.

1.11.4 Cluster Computing (1980 – 1990)

Cluster computing appeared as a **cheaper alternative to mainframes**.

How it worked: Regular machines were linked together using high-bandwidth networks. Together, they delivered computing power comparable to a mainframe, and you could easily add more machines ("nodes") whenever you needed more power.

The problem it solved: Cost — clusters were far cheaper than mainframes.

The problem it still had: The machines still needed to be **geographically close** to each other (usually in the same room or building).

Real-world examples:

- **Beowulf Clusters** — ordinary, off-the-shelf PCs networked together for scientific research
- **University Research Labs** — used low-cost clusters instead of buying expensive mainframes
- **Early Web Server Farms** — clustered machines sharing the load of website traffic

1.11.5 Grid Computing (1990 – 2000)

Grid computing solved the "geography problem" of clusters by connecting systems in **completely different geographic locations** via the internet — forming a "grid" across multiple organizations.

The problem it solved: Machines no longer needed to be physically close together.

The new problem it created: Limited bandwidth (internet speed) over such long distances.

Core Concept

Grid computing is often called the **direct predecessor of cloud computing** — meaning it is the technology that led most directly into what we now call "the cloud."

Real-world examples:

- **SETI@home** — volunteers around the world donated their computer's idle processing power to help analyze radio signals from space
- **CERN LHC Computing Grid** — physics data from the Large Hadron Collider processed across dozens of countries
- **Folding@home** — distributed protein-folding research spread across the globe

1.11.6 Utility Computing (Late 1990s – 2000s)

Utility computing introduced a completely new way of *paying* for computing.

The core idea: Compute power, storage, and infrastructure are billed on a **pay-per-use basis** — exactly like how you pay for electricity or water at home. You only pay for what you actually use.

Why this matters: This "pay only for what you use" idea became one of the foundational principles behind modern cloud pricing (the same model AWS, Azure, and Google Cloud use today).

Real-world examples:

- **Sun Grid Utility** — an early pay-per-CPU-hour computing service
- **IBM On-Demand Computing** — metered enterprise computing resources
- **Precursor to AWS Pricing** — this model set the template for today's pay-as-you-go cloud billing

1.11.7 Virtualization (1980 – Present)

Virtualization creates a **virtual layer** over physical hardware, which allows multiple independent "virtual machines" to run at the same time on a single physical machine.

Why it's important: Even though this technology was introduced decades ago, it is still the key technology underlying modern cloud platforms today. In fact, hardware virtualization remains the most common form used right now.

Real-world examples:

- **Amazon EC2** — thousands of virtual machines run on shared physical servers
- **VMware vCloud** — an enterprise-grade hardware virtualization platform
- **VirtualBox / Hyper-V** — widely used desktop and enterprise virtualization tools

1.11.8 Web 2.0 (Gained Popularity ~2004)

Web 2.0 is best understood as the **interface layer** — it is how cloud services actually interact with everyday users.

The shift it represents: Moving from static, unchanging web pages to interactive, dynamic web experiences.

Why it matters: Web 2.0 is the technology that made modern social media possible, and it greatly increased flexibility across the entire web.

Real-world examples:

- **Google Maps** — interactive, dynamic mapping right inside your browser
- **Facebook** — social media built entirely on interactive web pages
- **Twitter** — real-time, dynamic content sharing at a global scale

1.11.9 Service Orientation (2000s)

Service orientation became the **reference model** for cloud computing — meaning it is the blueprint that modern cloud services are still built on.

What it introduced:

- **Quality of Service (QoS)** with **Service Level Agreements (SLAs)** — formal promises about uptime and performance
- **Software as a Service (SaaS)** — software delivered as an online service instead of something you install

Real-world examples:

- **Salesforce** — one of the first major SaaS companies, offering contractual service guarantees
- **SLA-Backed Hosting** — providers that formally guarantee uptime and performance
- **Early Web Services (SOAP/XML)** — standardized, service-oriented application architecture

1.11.10 All Roads Lead to the Cloud

Putting it all together:

Core Concept

Cloud Computing means storing and accessing data and programs on **remote servers hosted on the internet**, instead of on a computer's hard drive or a local server.

Each generation contributed one essential ingredient to modern cloud computing:

Generation	What it contributed
Mainframe	Raw computing power
Distributed	Remote access
Virtualization	Virtualized resources
Utility Computing	Pay-per-use billing
Service Orientation	SaaS & SLAs

Bottom line: Today's cloud platforms (AWS, Azure, Google Cloud) are essentially the **sum of every generation that came before them.**

1.12 Public vs Private vs Hybrid Cloud

Choosing the Right Cloud Deployment Model

1.12.1 Three Deployment Models, One Decision

The main question that separates these three models is:

> Who owns the infrastructure — and who else shares it with you?

Model	Meaning	Tenancy
Public Cloud	Shared infrastructure owned by a third-party provider, pay-as-you-go	Multi-Tenant

Model	Meaning	Tenancy
Private Cloud	Dedicated infrastructure for one single organization, maximum control	Single-Tenant
Hybrid Cloud	A strategic mix of both — sensitive workloads stay private, the rest goes public	Mixed

1.12.2 Public Cloud — Owned & Shared by a Provider

Definition: Infrastructure and services that are fully owned, operated, and maintained by a third-party provider, delivered over the public internet, and shared among multiple organizations at once. This sharing setup is called **multi-tenancy**.

Core Concept

You pay only for the exact resources you consume.

Well-known examples: Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP)

Advantages

- Cost-efficient — no hardware to buy, purely pay-as-you-go
- Automatic updates, patches & maintenance handled by the provider
- High accessibility — usable from anywhere with an internet connection

Disadvantages

- Security & privacy concerns since servers are shared with others
- Limited control over the underlying infrastructure
- Performance depends on reliable internet connectivity
- May not meet strict compliance requirements (for example, HIPAA in healthcare)
- Unmonitored usage can cause unexpected cost overruns

1.12.3 Private Cloud — Dedicated to One Organization

Definition: A computing environment dedicated entirely to a single organization. It can be hosted on-site (inside the company itself) or by a third party — but the key point is that resources are **never shared** with other tenants.

Core Concept

Maximum control and isolation for sensitive operations.

Well-known examples: VMware, HPE / Dell Infrastructure, Bespoke Internal Data Centers

Advantages

- Higher security — full control to customize protections
- Deep customization of infrastructure and services
- Greater privacy over exactly who accesses data and resources

Disadvantages

- Higher cost — dedicated hardware & networking is expensive
- Limited scalability — capped by the physical hardware you own
- Technical complexity — requires specialized IT expertise to manage
- Still vulnerable to breaches and cyberattacks (private does not automatically mean safe)
- Lack of standardization can complicate future migration
- Ongoing maintenance and upgrades require real, continuous resources

1.12.4 Hybrid Cloud — The Best of Both Worlds

Definition: Combines public and private cloud environments, allowing data and applications to move between them. Sensitive, mission-critical workloads run on the private cloud, while the public cloud absorbs burst traffic or less-sensitive workloads.

Core Concept

Optimizing for security, performance, and cost — all at the same time.

Well-known examples: AWS Outposts, Azure Stack, IBM Hybrid Cloud

Advantages

- Flexibility — sensitive data stays private, while the public cloud adds extra reach
- Scalability — workloads can shift between the two clouds on demand
- Balanced security combined with the public cloud's cost savings

Disadvantages

- Complexity of integrating two very different environments
- Often costlier overall than using just one model alone
- Security risks can appear at the "seams" between environments
- Data governance challenges across environments (e.g., GDPR, HIPAA compliance)
- Network latency (delay) between environments
- Integration & compatibility challenges
- Possible vendor lock-in when using multiple providers

1.12.5 Head-to-Head Comparison

Factor	Public Cloud	Private Cloud	Hybrid Cloud
Resource Allocation	Shared / multi-tenant	Dedicated / single-tenant	Mix of shared & dedicated
Data Storage	Provider's remote servers	Internal, isolated servers	Sensitive data private, rest public
Pricing Model	Pay-as-you-go, variable	High fixed upfront cost	Mixed: fixed + variable
Management	Fully by provider	Internal IT team	Co-managed
Scalability	Near-infinite, instant	Limited by hardware	Flexible — bursts to public
Overall Cost	Cost-effective at scale	Expensive to build/maintain	Optimized by workload routing

1.12.6 Which One Should You Choose?

There is **no single "best" model** — the right choice always depends on your specific workload sensitivity, budget, and need for scale.

- **Choose Public Cloud if:** You're a startup, have variable workloads, and want zero hardware overhead.
- **Choose Private Cloud if:** You're in a regulated industry, handle sensitive data, or need full control over your infrastructure.
- **Choose Hybrid Cloud if:** Your business needs to balance sensitive workloads with the ability to scale on demand.

1.12.7 Quick Revision Summary

- **Cloud Computing** = using storage/software/servers/databases over the internet instead of your own computer.
- Your data (photos, emails) is stored on remote **servers** in **data centers**, not on your device.
- **Advantages:** cost saving, access anywhere, large storage, auto backup, high security, easy collaboration.
- **Disadvantages:** needs internet, privacy concerns, ongoing costs, possible outages.
- **8 generations that led to cloud computing:** Mainframe → Distributed → Cluster → Grid → Utility → Virtualization → Web 2.0 → Service Orientation.
- **Grid computing** = direct predecessor of cloud computing.
- **Three deployment models:**
  - **Public** = shared, cheap, less control (AWS, Azure, GCP)
  - **Private** = dedicated, expensive, full control (VMware, internal data centers)
  - **Hybrid** = mix of both (AWS Outposts, Azure Stack, IBM Hybrid Cloud)

1.13 Categories of Cloud Computing: Everything as a Service

We have already seen that cloud computing allows us to use computing resources through the Internet. But what exactly can we get from the cloud?

Sometimes we need only a server or storage. Sometimes we need a complete platform for developing an application. In other cases, we simply want to use ready-made software.

This leads to the idea of **"Everything as a Service"** or **XaaS (Anything as a Service)**.

The main service models discussed here are:

1. **Infrastructure as a Service (IaaS)**
2. **Platform as a Service (PaaS)**
3. **Software as a Service (SaaS)**
4. **Security as a Service (SECaaS)**

The easiest way to understand these models is to think about **how much work is done by the user and how much is managed by the cloud provider**.

1.13.1 Infrastructure as a Service (IaaS)

What is IaaS?

Imagine that a company wants to start a website. It needs a server, storage, networking and other computing resources. Buying all this physical hardware can be expensive.

Instead, the company can rent these resources from a cloud provider. This is called **Infrastructure as a Service (IaaS)**.

Core Concept

IaaS provides basic computing infrastructure such as servers, storage and networking through the Internet.

The cloud provider takes care of the physical hardware. The user can manage things such as the operating system, applications and data.

1.13.1.1 What Does IaaS Provide?

- Virtual machines
- Servers
- Storage
- Networking
- Virtual networks
- Firewalls

- Load balancers

1.13.1.2 Examples of IaaS

- Amazon EC2
- Microsoft Azure Virtual Machines
- Google Compute Engine

1.13.1.3 Advantages of IaaS

Advantages

- No need to buy expensive physical servers.
- Resources can be increased or decreased easily.
- Users generally pay only for the resources they use.
- It is suitable for large and changing workloads.
- It gives users considerable control over their computing environment.

1.13.1.4 Simple Example

Suppose a company needs a server for its website.

Instead of buying a physical server, the company can **rent a virtual server from a cloud provider**. If the website becomes popular and needs more resources, the company can increase them.

Remember: IaaS = Rent the infrastructure.

1.13.2 Platform as a Service (PaaS)

What is PaaS?

Now imagine that you are a programmer. You want to develop an application, but you do not want to spend time installing servers, operating systems, databases and development environments.

A cloud provider can give you a **ready-made development environment**. This is called **Platform as a Service (PaaS)**.

Core Concept

PaaS provides a ready-to-use platform for developing, testing and deploying applications.

The cloud provider manages most of the underlying infrastructure. Developers can mainly concentrate on writing and managing their application code.

1.13.2.1 What Does PaaS Provide?

- Development tools

- Runtime environments
- Databases
- Middleware
- Operating systems
- Development frameworks
- Application hosting environments

1.13.2.2 Examples of PaaS

- Google App Engine
- AWS Elastic Beanstalk
- Microsoft Azure App Service
- Heroku

1.13.2.3 Advantages of PaaS

Advantages

- Makes application development easier.
- Saves development time.
- Developers do not need to manage physical servers.
- Application deployment becomes easier.
- Supports development teams.
- Resources can be scaled when required.

1.13.2.4 Simple Example

Suppose a group of students wants to develop an **online shopping application**.

They do not want to spend time setting up servers and operating systems. They can use a PaaS platform and concentrate mainly on **developing the application**.

Remember: PaaS = Rent the development platform.

1.13.3 Software as a Service (SaaS)

What is SaaS?

Now think about applications such as **Gmail or Google Docs**.

Do we need to buy a server?

Do we need to install the operating system?

Do we need to maintain the application ourselves?

No.

We simply open the application and use it through the Internet.

This is called **Software as a Service (SaaS)**.

Core Concept

SaaS provides complete software applications to users through the Internet.

The cloud provider manages the application, servers, storage, updates and maintenance.

1.13.3.1 Examples of SaaS

- Gmail
- Google Docs
- Microsoft 365
- Salesforce
- Zoom
- Slack

1.13.3.2 Advantages of SaaS

Advantages

- No need to install the software locally.
- Can be accessed through the Internet.
- Software updates are handled by the provider.
- Very little maintenance is required from the user.
- Can be accessed from different devices.
- Usually available through subscription or usage-based plans.

1.13.3.3 Simple Example

Suppose you want to prepare a document.

You open **Google Docs** in your browser and start typing. You do not have to worry about where the servers are located or how the software is updated.

The cloud provider takes care of these things.

Remember: SaaS = Use ready-made software.

1.13.4 Security as a Service (SECaaS)

Why do we need Security as a Service?

Cloud computing is useful, but **security is very important**. Organizations need to protect their data, applications, networks, users and devices.

Setting up and maintaining all security systems by themselves can be expensive and difficult. Instead, organizations can obtain security services from cloud providers.

This is called **Security as a Service (SECaaS)**.

Core Concept

Security as a Service provides security functions through the cloud to protect systems, applications, networks and data.

1.13.4.1 What Can SECaaS Provide?

- Antivirus and malware protection
- Firewall services
- User authentication
- Identity and access management
- Intrusion detection
- Intrusion prevention
- Security monitoring
- Data encryption
- Vulnerability scanning
- Security event monitoring

1.13.4.2 Advantages of SECaaS

Advantages

- Reduces the cost of maintaining security systems.
- Provides access to specialized security tools.
- Security services can be scaled when required.
- Provides centralized security monitoring.
- Security updates can be managed by the provider.
- Useful for organizations that do not have a large security team.

1.13.4.3 Simple Example

Imagine a college has hundreds of computers connected to its network.

Instead of installing and maintaining separate security systems for everything, the college can use a **cloud-based security service** to monitor the network and identify suspicious activities.

Remember: SECaaS = Get security through the cloud.

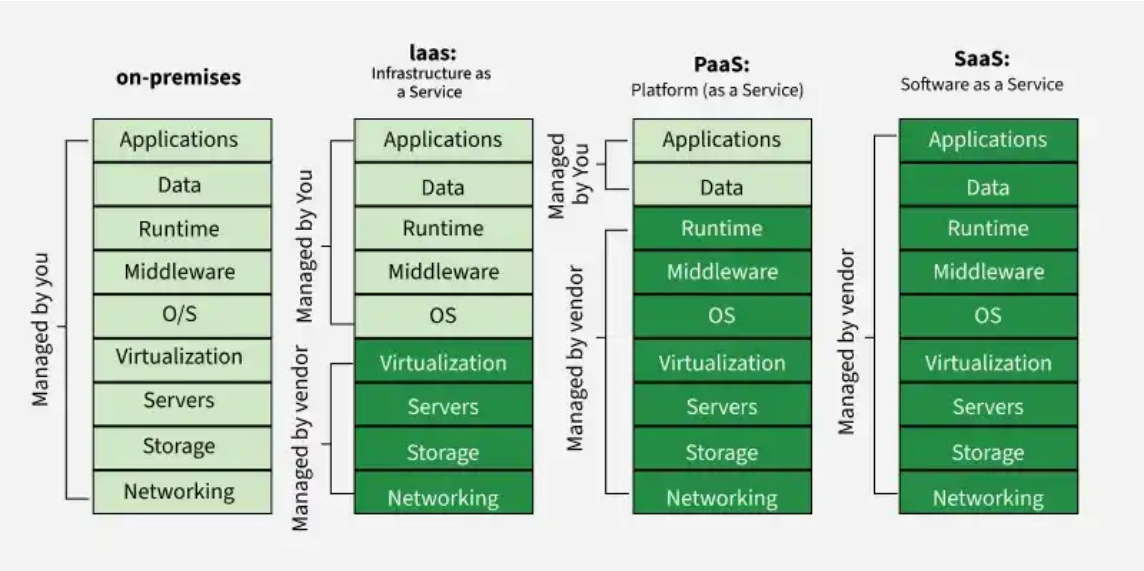


Figure 1.1: Management responsibilities in On-Premises, IaaS, PaaS and SaaS

Understanding the Figure:
The figure shows how the responsibility is divided between the user and the cloud service provider.
In an **on-premises** system, the organization manages almost everything, including networking, storage, servers, virtualization, operating systems and applications.
In **IaaS**, the cloud provider manages the basic infrastructure, while the user manages the operating system, middleware, runtime, data and applications.
In **PaaS**, the cloud provider manages more of the system, including the operating system, middleware and runtime. The user mainly manages the application and data.
In **SaaS**, the cloud provider manages almost the entire stack. The user mainly uses the application.

1.14 Understanding the Difference Between IaaS, PaaS and SaaS

The easiest way to understand the difference is to ask:

How much do I manage, and how much does the cloud provider manage?

Feature	IaaS	PaaS	SaaS
Main purpose	Provides infrastructure	Provides a development platform	Provides complete software
User control	More control	Moderate control	Very little technical control

Feature	IaaS	PaaS	SaaS
User mainly manages	Operating system, applications and data	Application code and data	Mainly the use of the application
Main users	IT administrators and developers	Software developers	End users
Example	Amazon EC2	Google App Engine	Google Docs

1.15 An Easy Way to Remember IaaS, PaaS and SaaS

Think about preparing food.

- **IaaS:** You get the kitchen and equipment. You prepare most things yourself.
- **PaaS:** You get a ready kitchen with many things already arranged. You mainly concentrate on preparing the food.
- **SaaS:** You simply order the finished food and eat it.

The same idea can be applied to cloud computing:

Core Concept

- **IaaS** → Infrastructure
- **PaaS** → Platform
- **SaaS** → Software
- **SECaaS** → Security

1.16 Quick Revision: Cloud Service Models

Service	What does it provide?	Easy way to remember
IaaS	Servers, storage and networking	**Rent infrastructure**
PaaS	Development platform and tools	**Develop applications**
SaaS	Complete software applications	**Use the software**
SECaaS	Security services and protection	**Protect systems and data**

Core Concept

One-line summary:
IaaS provides the **infrastructure**, PaaS provides the **platform**, SaaS provides the **software**, and SECaaS provides **security** through the cloud.

1.17 Pros and Cons of Cloud Computing

Cloud computing provides computing resources such as servers, storage, databases, networking, and software through the Internet instead of requiring users to maintain all computing resources locally. It provides flexibility and scalability, but it also introduces challenges related to security, cost, connectivity, and dependence on service providers.

1.17.1 Advantages of Cloud Computing

1.17.1.1 Cost Reduction

Cloud computing reduces the need to purchase and maintain expensive hardware and software.

- No need for large initial investment in servers.
- Maintenance costs are reduced.
- Users generally pay only for the resources they use.
- Suitable for startups, educational institutions, and small businesses.

Example: A company can rent cloud storage instead of purchasing and maintaining its own storage servers.

1.17.1.2 Scalability

Cloud resources can be increased or decreased according to workload requirements.

- Resources can be added during periods of high demand.
- Resources can be reduced when demand decreases.
- Helps organizations handle changing workloads efficiently.

Example: An online shopping website can increase computing resources during a festival sale.

1.17.1.3 Accessibility

Cloud services can be accessed from different locations using an Internet connection.

- Supports remote work.
- Users can access files and applications from multiple devices.
- Facilitates collaboration among geographically distributed teams.

Example: Students and teachers can access educational materials from home or college.

1.17.1.4 Flexibility

Cloud computing allows organizations to quickly select and configure the resources they need.

- Computing power can be changed easily.
- Storage capacity can be increased when required.
- Different services can be selected according to business needs.

1.17.1.5 High Availability

Many cloud providers design their infrastructure with redundant servers and systems.

- Services can continue even when some hardware fails.
- Data may be replicated across multiple systems or locations.
- Improves service availability.

1.17.1.6 Backup and Disaster Recovery

Cloud platforms can be used to store backup copies of data.

- Protects data against hardware failures.
- Helps recover data after accidental deletion or disasters.
- Reduces the need for maintaining separate physical backup infrastructure.

1.17.1.7 Automatic Updates

Cloud service providers generally manage many software and infrastructure updates.

- Reduces administrative workload.
- Users can receive updated versions of services.
- Security patches can be deployed by the provider.

1.17.1.8 Faster Deployment

Cloud resources can usually be provisioned much faster than traditional physical infrastructure.

Example: A developer can create a virtual server in minutes instead of purchasing and configuring a physical server.

1.17.2 Disadvantages of Cloud Computing

1.17.2.1 Internet Dependency

Cloud services normally require a reliable Internet connection.

- Poor connectivity can affect performance.
- Internet failure may prevent access to cloud resources.

- Network latency can affect applications requiring real-time responses.

1.17.2.2 Security and Privacy Risks

Data stored in the cloud is managed using infrastructure controlled by a cloud provider. Potential security and privacy concerns include:

- Unauthorized access
- Data breaches
- Account compromise
- Incorrect access permissions
- Data privacy issues

Therefore, proper authentication, encryption, and access control are important.

1.17.2.3 Vendor Lock-In

Organizations may become dependent on a particular cloud provider.

- Moving applications to another provider may be difficult.
- Data migration can be expensive or time-consuming.
- Different providers may use different technologies and interfaces.

Example: An application designed heavily around one provider's services may require significant modification before it can run elsewhere.

1.17.2.4 Limited Control

In cloud computing, the provider manages much of the underlying infrastructure. Users may have limited control over:

- Physical servers
- Data-center infrastructure
- Hardware configuration
- Certain security mechanisms

1.17.2.5 Downtime

Cloud providers can experience service interruptions.

Even though cloud platforms generally provide high availability, failures can occur because of:

- Hardware problems
- Software failures
- Network problems
- Configuration errors

- Cyberattacks

1.17.2.6 Unpredictable Costs

Although cloud computing can reduce initial investment, poorly managed usage can result in unexpectedly high bills.

For example:

- Running unnecessary virtual machines
- Excessive storage usage
- High data-transfer usage
- Unused cloud resources

Therefore, cloud cost monitoring is important.

1.17.2.7 Performance Issues

Cloud performance may vary depending on:

- Network bandwidth
- Server load
- Distance from the data center
- Type of cloud service
- Resource allocation

Applications requiring very low latency may not always perform optimally in a public cloud.

1.17.2.8 Compliance and Legal Issues

Organizations handling sensitive or regulated data must comply with applicable laws and regulations.

Issues may include:

- Where data is physically stored
- Who can access the data
- Data retention requirements
- Industry-specific regulations

1.17.3 Advantages and Disadvantages at a Glance

1.17.4 Simple Example

Consider a college that needs an online learning system.

Table 1.9: Advantages and Disadvantages of Cloud Computing

Advantages	**Disadvantages**
Lower initial cost	Internet dependency
Easy scalability	Security and privacy concerns
Accessible from anywhere	Vendor lock-in
Flexible resource management	Limited infrastructure control
High availability	Possibility of downtime
Backup and disaster recovery	Unpredictable costs
Automatic updates	Possible performance issues
Faster deployment	Compliance challenges

1.17.4.1 Traditional Approach

The college must purchase and maintain:

- Servers
- Storage devices
- Networking equipment
- Backup systems

It must also manage hardware maintenance and upgrades.

1.17.4.2 Cloud Approach

The college can use cloud-based servers and storage.

Benefits:

- Lower initial investment
- Easy expansion when the number of students increases
- Remote access
- Backup facilities

Challenges:

- Requires Internet connectivity
- Student data must be properly protected
- Monthly usage costs must be monitored
- The college may become dependent on the selected provider

1.18 What Is Virtualization?

Virtualization is a computer architecture technology that allows multiple virtual machines (VMs) to be multiplexed on the same physical hardware. The concept dates back to the 1960s. Its purpose is to enhance resource sharing among many users and to improve computer performance in terms of resource utilization and application flexibility.

Hardware resources (CPU, memory, I/O devices, etc.) or software resources (operating system and software libraries) can be virtualized at various functional layers. The core idea is to separate the hardware from the software to yield better system efficiency — the same principle behind virtual memory, which gave users access to a much larger memory space than physically existed.

Before virtualization: a traditional computer runs a single host operating system tailored to its hardware.
After virtualization: a software layer called the hypervisor / Virtual Machine Monitor (VMM) is inserted between hardware and the OS, allowing multiple guest operating systems (and their applications) to run independently on the same physical machine.

Figure 3.1 — Architecture Before and After Virtualization

Figure 3.1Architecture of a computer system before and after virtualization (VMM = Virtual Machine Monitor).

According to a 2009 Gartner Report, virtualization was rated the top strategic technology poised to change the computer industry. Given sufficient storage, any computer platform can, in principle, be installed on another host computer — even one using a different instruction set and running a different operating system.

1.19 Levels of Virtualization Implementation

The virtualization layer creates the abstraction of VMs by being interposed at different levels of a computer system. There are five common levels at which virtualization can be implemented:

- Instruction Set Architecture (ISA) level
- Hardware Abstraction Level (HAL)
- Operating System level
- Library Support level
- User-Application level

Figure 3.2 — The Five Levels of Virtualization

Figure 3.2Virtualization ranging from the hardware to the application level (the diagram you already have — Application / Library / OS / HAL / ISA levels with examples such as JVM, WINE, Jail, VMware, Bochs).

1.19.1 Instruction Set Architecture (ISA) Level

At the ISA level, virtualization works by emulating a given instruction set using the ISA of the host machine. For example, MIPS binary code can be run on an x86-based host with the help of ISA emulation. This makes it possible to run large amounts of legacy binary code — written for other processors — on any new hardware host.

- Basic emulation is done through code interpretation: an interpreter converts source instructions to target instructions one at a time (one source instruction may need tens or hundreds of native target instructions), which is relatively slow.
- For better performance, **dynamic binary translation** is used — this translates whole basic blocks (or larger program traces/super blocks) of source instructions to target instructions at once, improving efficiency.
- This approach requires adding a processor-specific software translation layer to the compiler, producing a Virtual Instruction Set Architecture (V-ISA).

1.19.2 Hardware Abstraction Level (HAL)

Hardware-level virtualization is performed directly on top of the bare hardware. It generates a virtual hardware environment for a VM and simultaneously manages the underlying real hardware. The goal is to virtualize resources — processors, memory, and I/O devices — so that hardware utilization is improved when many users work concurrently.

- First implemented in the IBM VM/370 in the 1960s.
- More recently, the Xen hypervisor has been used to virtualize x86-based machines to run Linux or other guest operating systems.

1.19.3 Operating System Level

This is an abstraction layer inserted between the traditional OS and user applications. OS-level virtualization creates isolated containers on a single physical server, along with OS instances, to make full use of the hardware and software in data centers.

- The containers behave like real, independent servers.
- Commonly used to create virtual hosting environments that allocate hardware resources among many mutually distrusting users (e.g., web hosting).
- Also used, to a lesser extent, for server consolidation — moving services from separate physical hosts into containers or VMs on one server.

1.19.4 Library Support Level

Most applications use APIs exported by user-level libraries rather than making lengthy system calls directly to the OS. Since these APIs are well documented, they become a natural candidate for virtualization.

- Virtualization is achieved by controlling the communication link between applications and the rest of the system through API hooks.
- **Example — WINE**: implements this approach to let Windows applications run on top of UNIX hosts.
- **Example — vCUDA**: allows applications running inside VMs to leverage GPU hardware acceleration.

1.19.5 User-Application Level

Application-level virtualization virtualizes an application as a VM. Since an application normally runs as a process on a traditional OS, this is also known as process-level virtualization.

- The most popular approach is deploying High-Level Language (HLL) VMs, where the virtualization layer sits as a program on top of the OS and exports an abstract machine that runs programs compiled for it.
- Examples: Microsoft .NET CLR and the Java Virtual Machine (JVM).
- Other forms are called application isolation, application sandboxing, or application streaming — the application is wrapped in a layer isolated from the host OS and other applications, making it easy to distribute and remove.
- Example: the LANDesk application virtualization platform, which deploys software as self-contained executables that need no installation, system modification, or elevated privileges.

1.19.6 Relative Merits of Different Virtualization Levels

The table below compares the levels on four technical merits: Higher Performance, Application Flexibility, Implementation Complexity, and Application Isolation. More X marks (up to five) means a stronger advantage on that merit.

tableheadfill

ISA Level	X	XXXXX	XXXXX	XX
Hardware Abstraction Level	XXXXX	XX	XXXXX	XXX
Operating System Level	XXXXX	XX	XX	XX
Library (User-Level API) Level	XXX	XX	XX	XX
User Application Level	XX	XXX	XXX	XXXXX

Overall, hardware- and OS-level support tend to yield the highest performance, but they are also the most expensive to implement. User isolation is hardest to achieve at the OS level. ISA-level implementation offers the best application flexibility.

1.20 VMM Design Requirements and Providers

Hardware-level virtualization inserts a layer — the Virtual Machine Monitor (VMM), also called a hypervisor — between the real hardware and traditional operating systems.

The VMM manages the hardware resources of the computing system and captures every attempt by a program to access the hardware, effectively acting like a traditional OS. A single hardware component (e.g., the CPU) can be virtualized into several virtual copies, allowing several traditional operating systems — the same or different — to run simultaneously on the same hardware.

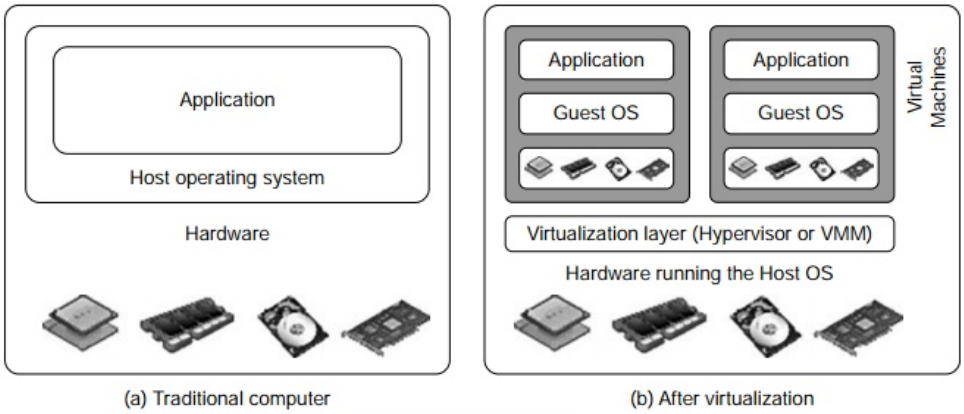


FIGURE 3.1

The architecture of a computer system before and after virtualization, where VMM stands for virtual machine monitor.

Figure 1.2: VMM/hypervisor layered between physical hardware and multiple guest OS + application stacks.

Three Requirements of a VMM

1. Equivalence / Fidelity — A VMM should provide an environment for programs that is essentially identical to the original physical machine.

2. Performance — Programs running in this environment should show, at worst, only minor decreases in speed.

3. Resource Control — A VMM must be in complete control of the system resources.

Two exceptions to the "identical environment" rule are permitted: (a) differences caused by the availability of system resources when more than one VM runs on the same machine (the sum of resources given to all VMs is greater than the real machine's total), and (b) differences caused by timing dependencies, due to the extra software layer and other VMs running concurrently. These exceptions affect only performance, not function — which is why an ordinary time-sharing OS does not qualify as a VMM.

A VMM must also be efficient. Traditional emulators and complete software interpreters (simulators) emulate every instruction through functions or macros — the most flexible approach, but too slow for practical use as a real machine. To guarantee efficiency, a statistically dominant subset of the virtual processor's instructions must be executed directly by the real processor, without software intervention by the VMM.

Complete Control of Resources

- The VMM is responsible for allocating hardware resources to programs.

- A program cannot access any resource that has not been explicitly allocated to it.
- Under certain circumstances, the VMM can regain control of resources already allocated.

Not every processor architecture satisfies these requirements naturally. For example, the x86 architecture has historically had difficulty trapping on some privileged instructions. When a processor is not designed to support virtualization directly, the hardware itself must be modified — this is known as **hardware-assisted virtualization**.

1.21 Virtualization Support at the OS Level

VM technology underpins cloud computing, which shifts the hardware and staffing costs of running a computing center to a third party — much like a bank. Cloud computing faces two key challenges that OS-level virtualization helps address:

- The need to use a variable number of physical machines and VM instances depending on workload — a task may need only one CPU at times and hundreds at others.
- The slow operation of instantiating new VMs — new VMs typically start either as fresh boots or as replicas of a template VM, unaware of the current application state.

1.21.1 Why OS-Level Virtualization?

Hardware-level VMs are slow to initialize because each VM builds its own image from scratch — a serious problem when a cloud environment may need to initialize thousands of VMs at once. Storing many VM images is also costly, since there is considerable repeated content across images. Full hardware-level virtualization additionally suffers from slower performance, lower density, and (for para-virtualization) the need to modify the guest OS. OS-level virtualization offers a lighter-weight alternative to these problems.

Operating system virtualization inserts a virtualization layer inside an OS to partition a machine's physical resources, enabling multiple isolated VMs within a single OS kernel. Such a VM is often called a virtual execution environment (VE), Virtual Private System (VPS), or simply a container.

- From the user's point of view, each VE looks like a real server, with its own processes, file system, user accounts, network interfaces/IP addresses, routing tables, and firewall rules.
- All VEs share the same OS kernel, so OS-level virtualization is also called single-OS-image virtualization.

1.21.2 Advantages of OS Extensions

- Minimal startup/shutdown cost, low resource requirements, and high scalability, because all OS-level VMs on a machine share a single OS kernel.

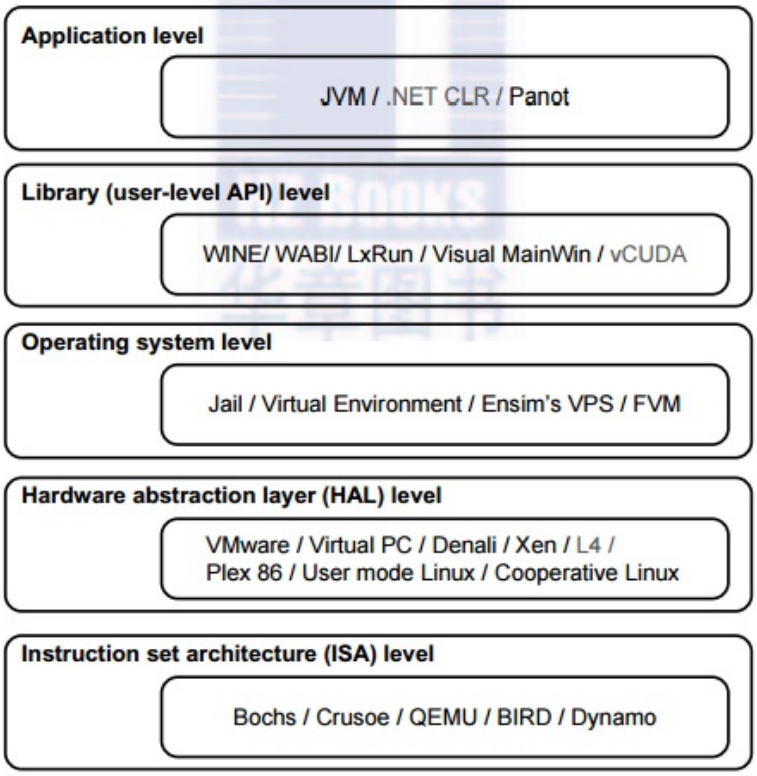


FIGURE 3.2

Virtualization ranging from hardware to applications in five abstraction levels.

Figure 1.3: OS-level virtualization viewed from the machine stack — a virtualization layer inside the OS partitions hardware resources among several containers/VPS.

- A VM and its host environment can synchronize state changes when necessary, since the virtualization layer lets VM processes access host resources (without modifying them).

These two benefits directly overcome the hardware-level VM problems of slow initialization and being unaware of the current application state.

1.21.3 Disadvantages of OS Extensions

All OS-level VMs on one physical machine must run the same operating system family as the host — for example, a Windows XP guest cannot run inside a Linux-based container. This is limiting in cloud environments where users want a free choice between Windows, Linux, and other operating systems.

To implement OS-level virtualization, isolated execution environments are created from a single OS kernel, and access requests from a VM are redirected to that VM's own resource partition. For example, the `chroot` command in UNIX creates several virtual root directories, one per VM.

- **Approach 1** — duplicate common resources into every VM partition: simple but incurs significant resource cost and overhead.

- **Approach 2** — share most resources with the host and create private copies only on demand: more efficient, and generally preferred.

1.21.4 Virtualization on Linux and Windows Platforms

Most reported OS-level virtualization systems are Linux-based (support on Windows remains largely at the research stage). Because new hardware may require a new/patched kernel, different Linux distributions ship patched kernels for extended functionality — though most Linux platforms are not tied to any single special kernel, so a host can run several VMs simultaneously on the same hardware.

tableheadfill

Linux VServer	Linux	OS-level virtualization for running multiple Linux-based guest environments
OpenVZ	Linux	Open-source, container-based virtualization; modifies the Linux kernel for isolation, resource management, and checkpointing
FVM (Feather-weight Virtual Machine)	Windows NT	OS-level virtualization built specifically for the Windows NT platform

Example — OpenVZ

OpenVZ is an open-source, Linux-based, container-based virtualization tool. It modifies the Linux kernel to support virtualization, subsystem isolation, limited resource management, and checkpointing. Several Virtual Private Servers (VPSes) can run simultaneously on one physical machine, each looking like a normal Linux server with its own files, users/groups, process tree, virtual network, virtual devices, and IPC (semaphores and messages).

OpenVZ's resource-management subsystem has three components:

- **Two-level disk allocation** — the OpenVZ administrator sets total disk space per VM (level 1); the VM's own administrator then allocates that space among its users/groups (level 2).
- **Two-level CPU scheduler** — a first-level scheduler decides which VM receives the next time slice (based on virtual CPU priority/limits); a second-level scheduler within each VM behaves like a normal Linux scheduler.
- **Resource controller** — about 20 tunable parameters covering all aspects of VM operation, giving fine control over the resources a VM can use.

OpenVZ also supports checkpointing and live migration: a VM's complete state can be saved to a disk file in seconds, transferred to another physical machine, and restored there — although established network connections still introduce a brief delay during migration.

1.22 Middleware Support for Virtualization

Library-level virtualization — also called user-level Application Binary Interface (ABI) or API emulation — creates an execution environment for running an "alien" program on a platform, without creating a full VM to run an entire guest operating system. The key mechanism is API call interception and remapping.

tableheadfill	
WABI (Windows ABI)	Middleware that converts Windows system calls into Solaris system calls
Lxrun	System-call emulator that lets Linux applications built for x86 run on UNIX systems
WINE	Provides library support to virtualize x86 processors, allowing Windows applications to run on UNIX hosts
Visual MainWin	Compiler support system for developing Windows applications (via Visual Studio) that run on certain UNIX hosts
vCUDA	Virtualizes the CUDA library so GPU-accelerated CUDA applications inside a guest OS can use the host's GPU

Example — vCUDA for GPU Virtualization

CUDA is a programming model and library for general-purpose GPU (GPGPU) computing, letting compute-intensive applications exploit GPU performance on the host OS. Running CUDA applications directly inside hardware-level VMs is difficult, so vCUDA virtualizes the CUDA library and installs it inside the guest OS. When a guest application calls the CUDA API, vCUDA intercepts the call and redirects it to the real CUDA API on the host OS.

vCUDA uses a client–server model with three user-space components:

- **vCUDA library** — resides in the guest OS as a drop-in replacement for the standard CUDA library; intercepts and redirects API calls to the stub.
- **Virtual GPU (vGPU) in the guest OS** — acts as the client; gives applications a uniform view of the underlying GPU hardware, returns a local virtual address when memory is allocated (while notifying the stub to allocate the real device memory), and records the CUDA API call flow.
- **vCUDA stub in the host OS** — acts as the server; receives and interprets remote requests, creates the execution context for each API call, returns results to the guest OS, and manages the actual physical GPU resource allocation.

1.23 Virtualization Structures, Tools, and Mechanisms

Before virtualization, the operating system manages the hardware directly. After virtualization, a virtualization layer is inserted between the hardware and the operating system. This layer is responsible for converting portions of the real hardware into virtual hardware, so that different operating systems — such as Linux and Windows — can run on the same physical machine simultaneously.

Depending on where the virtualization layer sits, there are three typical classes of VM architecture:

- **Hypervisor architecture** — the virtualization layer (hypervisor / VMM) sits directly on the bare hardware.
- **Para-virtualization** — the guest OS is modified to cooperate with the virtualization layer.
- **Host-based virtualization** — the virtualization layer sits on top of an existing host operating system.

The hypervisor is also known as the VMM (Virtual Machine Monitor); both terms describe the same virtualization function.

1.23.1 Hypervisor and Xen Architecture

The hypervisor supports hardware-level virtualization on bare-metal devices like the CPU, memory, disk, and network interfaces. The hypervisor software sits directly between the physical hardware and the guest OS. This virtualization layer is referred to as either the VMM or the hypervisor, and it provides *hypercalls* for the guest OSes and applications.

Depending on functionality, a hypervisor can assume one of two architectures:

- **Micro-kernel hypervisor** (e.g., Microsoft Hyper-V) — includes only the basic and unchanging functions, such as physical memory management and processor scheduling. Device drivers and other changeable components sit outside the hypervisor, so its code size is smaller.
- **Monolithic hypervisor** (e.g., VMware ESX for server virtualization) — implements all the aforementioned functions, *including* the device drivers, making its code base larger.

A hypervisor must be able to convert physical devices into virtual resources dedicated to the VM that is using them.

Architecture — Hypervisor Sitting Between Hardware and Guest OSes

The Xen Architecture

The core components of a Xen system are the hypervisor, kernel, and applications. Many guest OSes can run on top of the Xen hypervisor, but not all guest OSes are created equal — one in particular controls the others.

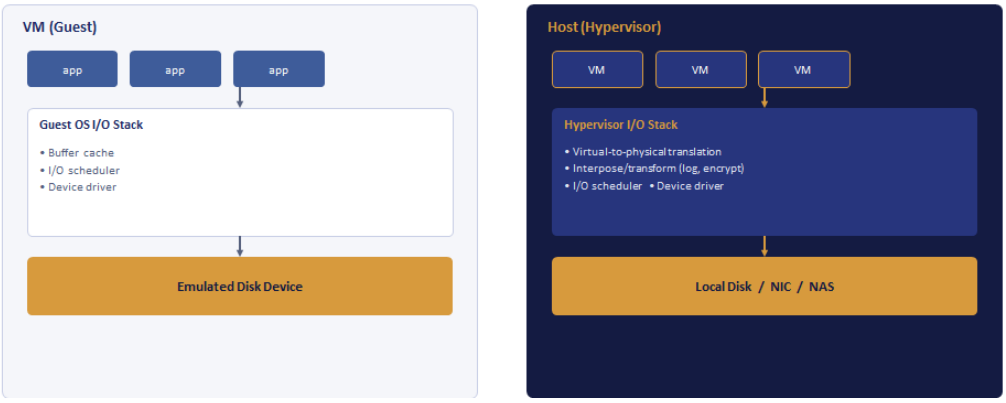


Figure 1.4: Generic hypervisor/VMM architecture — several guest OS + application stacks running on top of the hypervisor, which sits directly on the physical hardware.

- **Domain 0** — the privileged guest OS of Xen. It is the first domain loaded when Xen boots, before any file-system drivers are available. Domain 0 is designed to access hardware directly and manage devices.
- **Domain U** — the unprivileged guest domains. Domain 0 is responsible for allocating and mapping hardware resources for the guest domains (Domain U).

1.23.2 Para-Virtualization with Compiler Support

Para-virtualization requires modifying the guest operating system. A para-virtualized VM provides special APIs that require substantial OS modifications in user applications. Performance degradation is a critical issue of a virtualized system — no one wants to use a VM if it is much slower than using a physical machine.

The traditional x86 processor offers four instruction execution rings: Rings 0, 1, 2, and 3. The lower the ring number, the higher the privilege of the instruction being executed. The OS is responsible for managing the hardware and runs privileged instructions at Ring 0, while user-level applications run at Ring 3.

1.24 Virtualization of CPU, Memory, and I/O Devices

To support virtualization, processors such as the x86 employ a special running mode and instructions, known as **hardware-assisted virtualization**. In this way, the VMM and guest OS run in different modes, and all sensitive instructions of the guest OS and its applications are trapped in the VMM. Mode switching is completed by hardware, to save processor states efficiently. Intel and AMD each have proprietary technologies for hardware-assisted virtualization on the x86 architecture.

1.24.1 Hardware Support for Virtualization

Modern operating systems and processors permit multiple processes to run simultaneously. If there were no protection mechanism in a processor, all instructions from different

processes would access the hardware directly and could cause a system crash. Therefore, all processors have at least two modes:

- **User mode** — runs unprivileged instructions.
- **Supervisor mode** — runs privileged instructions, ensuring controlled access to critical hardware.

In a virtualized environment, correctness is harder to guarantee because there are more layers in the machine stack.

1.24.2 CPU Virtualization

A VM is a duplicate of an existing computer system in which the majority of VM instructions execute on the host processor in native mode. Thus, unprivileged instructions of VMs run directly on the host machine for higher efficiency. Other critical instructions must be handled carefully for correctness and stability. These critical instructions fall into three categories:

- **Privileged instructions** — execute in a privileged mode and are trapped if executed outside that mode.
- **Control-sensitive instructions** — attempt to change the configuration of resources used.
- **Behavior-sensitive instructions** — have different behaviors depending on the configuration of resources, including load and store operations over virtual memory.

A CPU architecture is **virtualizable** if it supports running the VM's privileged and unprivileged instructions in the CPU's user mode, while the VMM runs in supervisor mode. When privileged, control-sensitive, or behavior-sensitive instructions of a VM execute, they are trapped in the VMM, which acts as a unified mediator for hardware access across all VMs — ensuring correctness and stability for the whole system. RISC CPU architectures can be naturally virtualized because all control- and behavior-sensitive instructions are privileged instructions there.

Hardware-Assisted CPU Virtualization

This technique simplifies virtualization, since full or para-virtualization is otherwise complicated. Intel and AMD add an additional privilege level, called Ring -1, to x86 processors. As a result:

- The operating system can still run at Ring 0.
- The hypervisor runs at Ring -1.
- All privileged and sensitive instructions are trapped in the hypervisor *automatically*.

This removes the difficulty of implementing binary translation for full virtualization, and it also lets the operating system run inside VMs *without modification*.

1.24.3 Memory Virtualization

Virtual memory virtualization is similar to the virtual memory support provided by modern operating systems. In a traditional (non-virtualized) execution environment, the OS maintains mappings of virtual memory to machine memory using page tables — a **one-stage mapping** from virtual memory to machine memory. All modern x86 CPUs include a Memory Management Unit (MMU) and a Translation Lookaside Buffer (TLB) to optimize virtual memory performance.

In a virtual execution environment, memory virtualization involves sharing the physical system RAM and dynamically allocating it to the physical memory of the VMs. This requires a **two-stage mapping process**, maintained by the guest OS and the VMM respectively:

- Virtual memory → Physical memory (maintained by the guest OS)
- Physical memory → Machine memory (maintained by the VMM)

The guest OS continues to control the mapping of virtual addresses to the physical memory addresses of the VM, but it **cannot** directly access the actual machine memory. The VMM is responsible for mapping guest physical memory to the actual machine memory. MMU virtualization is transparent to the guest OS.

Architecture — Two-Level Memory Mapping Procedure

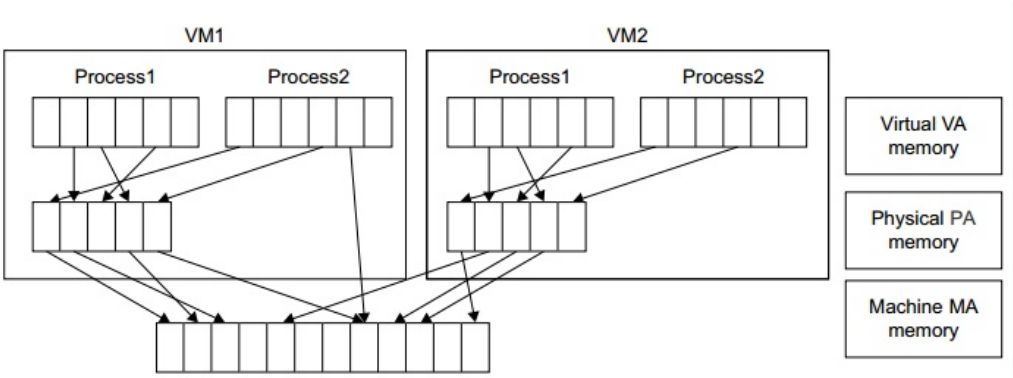


Figure 1.5: Two-level memory mapping procedure — per-process virtual (VA) memory in VM1 and VM2 is mapped to each VM's physical (PA) memory, which is in turn mapped to the host's real machine (MA) memory.

Practical Memory Management in a Hypervisor

Modern hypervisors add several techniques on top of the two-stage mapping to use physical RAM efficiently across many VMs:

- **Paging** — when memory blocks need to be freed for newer information, older, less-recently-used blocks (*pages*, typically 4 KB) are written to disk. The disk acts as an extension of physical memory; the file the pages are copied to is called the *page file*. Paging is far slower than working directly with memory.

- **Memory overcommitment** — allocating more virtual memory across VMs than the host physically has, on the assumption that VMs rarely use 100% of their allocated memory simultaneously.
- **Ballooning** — to reclaim physical memory from a VM under contention, a *balloon driver* inside the guest OS inflates, forcing the guest OS to flush its own least-recently-used pages to disk. Once the pages are flushed, the balloon deflates and the hypervisor reclaims that physical memory. This usually happens only when there is contention for memory.
- **Page sharing / copy-on-write** — memory pages with identical content (e.g., across VMs running the same OS) can be shared among multiple VMs on the same physical host, which is highly useful for read-only pages. If a VM needs to write to a page that has been marked as shared, the hypervisor creates a private copy for that VM — this is called **copy-on-write**. In practice, page sharing can save an additional 10% to over 40% of available physical memory, and it is especially valuable for Virtual Desktop Infrastructure (VDI), where many VMs run near-identical desktop images.
- **Swap space** — when a VM is first booted, the hypervisor reserves a disk-based swap space for storing memory pages if paging is required. If ballooning does not free enough memory, the hypervisor can swap the VM's memory pages out of physical memory entirely.

1.24.4 I/O Virtualization

I/O virtualization involves managing the routing of I/O requests between virtual devices and the shared physical hardware. There are three ways to implement I/O virtualization:

- Full device emulation
- Para-virtualization (split driver model)
- Direct I/O

Full Device Emulation

Full device emulation is the first approach to I/O virtualization. Generally, this approach emulates well-known, real-world devices. All the functions of a device or bus infrastructure — device enumeration, identification, interrupts, and DMA — are replicated in software. This software sits inside the VMM and acts as a virtual device. I/O access requests from the guest OS are trapped in the VMM, which interacts with the real I/O devices on the guest's behalf.

A single hardware device can be shared by multiple VMs running concurrently. However, software emulation runs much slower than the hardware it emulates.

Para-Virtualization I/O (Split Driver Model)

The para-virtualization method of I/O virtualization is typically used in Xen. It is also known as the **split driver model**, consisting of a *frontend driver* and a *backend driver*:

- The **frontend driver** runs in Domain U and manages the I/O requests of the guest OSes.
- The **backend driver** runs in Domain 0 and is responsible for managing the real I/O devices and multiplexing the I/O data of different VMs.

They interact with each other via a block of shared memory. Although para I/O virtualization achieves better device performance than full device emulation, it comes with a higher CPU overhead.

Direct I/O Virtualization

Direct I/O virtualization lets the VM access devices directly, achieving close-to-native performance without high CPU costs. However, current direct I/O virtualization implementations mostly focus on networking for mainframes, and there are many challenges for commodity hardware devices — for example, when a physical device is reclaimed (required by workload migration) for later reassignment, it may have been left in an arbitrary state (e.g., mid-DMA to some arbitrary memory location) that can function incorrectly or even crash the whole system. Since software-based I/O virtualization requires a very high overhead of device emulation, hardware-assisted I/O virtualization is critical.

The Guest OS I/O Stack (Inside the VM)

When an application inside a VM performs a disk or network operation, the request does **not** go straight to physical hardware. It first passes through the guest OS's own I/O stack:

- **Applications** — one or more apps issue reads/writes; the entry point of any I/O request.
- **Buffer cache** — recently used data is cached in memory to avoid slow disk access on repeat requests.
- **I/O scheduler** — orders and prioritizes queued requests to improve throughput and fairness.
- **Device driver** — the software layer the guest OS uses to talk to what it believes is real hardware.
- **Emulated disk device** — not a physical disk; the hypervisor presents a virtual disk that the guest OS treats as real.

After leaving the VM, the request passes through a second, independent I/O stack managed by the hypervisor on the host, before it ever reaches a real disk, NIC, or storage array:

- **Virtual-to-physical translation** — maps each VM's virtual disk blocks to real locations on physical storage.
- **Interpose / transform** — the hypervisor can intercept traffic to add logging, encryption, replication, or compression — transparently to the VM.
- **I/O scheduler** — fairly arbitrates I/O requests arriving from many competing VMs at once.

- **Device driver** — the real driver that communicates with actual physical hardware.
- **Local disk / NIC / NAS** — requests land on local physical disks, or travel over the network card to shared network-attached storage.

Architecture — Hypervisor I/O Stack

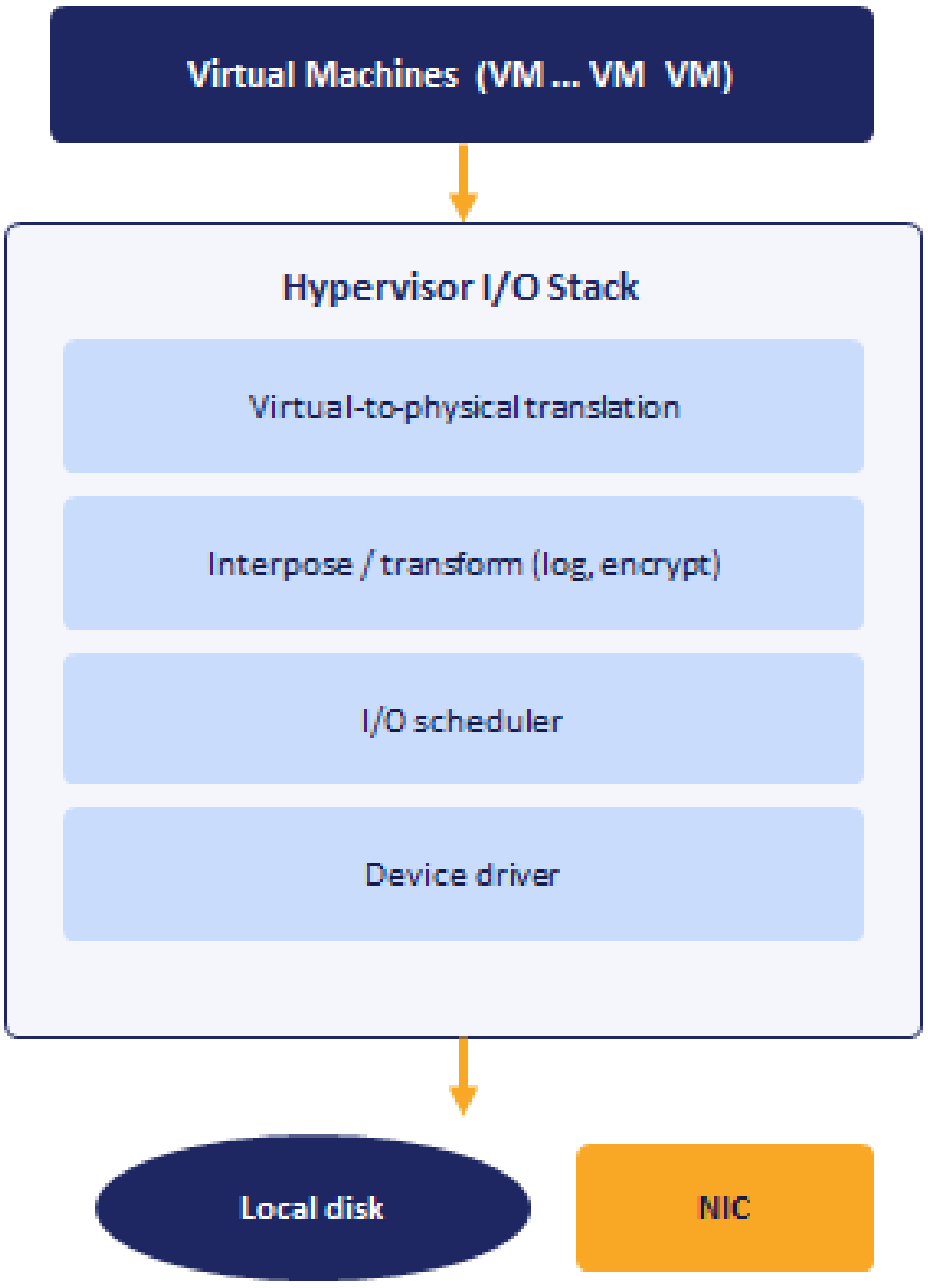


Figure 1.6: Hypervisor I/O stack at the host: Virtual Machines (VM1 ... VMn) → Virtual-to-physical translation → Interpose/transform (log, encrypt) → I/O scheduler → Device driver → Local disk / NIC / NAS.

Guest Stack vs. Hypervisor Stack — Side-by-Side

tableheadfill

Scope	Handles one VM's own I/O	Handles I/O from all VMs on the host
Awareness of hardware	Believes it uses real hardware (emulated)	Talks directly to real physical hardware
Key extra step	Buffer cache for speed	Virtual-to-physical address translation
Special capability	None — standard OS behavior	Interpose/transform: log, encrypt, replicate
Final destination	Emulated disk device	Local disk, NIC, or remote NAS

Why Two Stacks? The Trade-off

- **The cost — overhead:** every I/O request now travels through two full stacks instead of one, adding extra processing steps (translation, scheduling twice), which makes virtualized I/O slightly slower than I/O on a plain, non-virtualized computer. *Example:* a database server running inside a VM typically sees somewhat higher disk latency than the same database running directly on hardware.
- **The benefit — new powers:** because the hypervisor sits in the middle of every I/O request, it can add valuable features transparently — without the guest OS or application ever knowing or needing to change. *Example:* the hypervisor can automatically encrypt every VM's disk writes, log activity for auditing, or move a VM's storage to a NAS — all invisibly to the VM.

Follow One Request, Start to Finish

1. An application inside the VM issues an I/O request (e.g., saving a file).
2. The request enters the Guest OS I/O stack: the buffer cache is checked first, then the I/O scheduler queues it.
3. The guest's device driver sends the request to what it believes is a real disk — but it is actually emulated.
4. The hypervisor intercepts the request as it leaves the VM.
5. Virtual-to-physical translation converts the VM's virtual address into a real storage location.
6. The interpose/transform layer may encrypt, log, or otherwise modify the request — invisibly to the VM.
7. The hypervisor's I/O scheduler fits this request in fairly among all the other VMs requesting I/O at the same time.

8. The real device driver sends the request out to the local disk, or over the NIC to a NAS.

Steps 1–3 happen entirely inside the VM (the guest OS does not know it is virtualized); steps 4–8 happen at the host, invisibly to the guest.

1.24.5 Virtualization in Multi-Core Processors

Virtualizing a multi-core processor is relatively more complicated than virtualizing a unicore processor. Though multicore processors are claimed to have higher performance by integrating multiple processor cores in a single chip, multi-core virtualization raises new challenges for computer architects, compiler constructors, system designers, and application programmers. There are mainly two difficulties:

- Application programs must be parallelized to use all cores fully.
- Software must explicitly assign tasks to the cores — a very complex problem.

1.25 Desktop Virtualization

1.25.1 What Is Desktop Virtualization?

Desktop Virtualization runs a user's entire desktop environment — operating system, applications, and data — on a remote server instead of on the local device. The user's device only displays the desktop and sends back keyboard/mouse input.

Everyday analogy: it is like streaming a movie instead of downloading it. You do not need the movie file on your own device — you just see the picture. With Desktop Virtualization, you do not need the desktop's processing power on your device — you just see the screen.

Traditional Desktop vs. Virtualized Desktop

tableheadfill

Where it runs	Locally, on the user's own PC	On a server in the data center
Client device	Full PC required	A **thin client** — a lightweight, low-cost terminal with minimal local processing — is enough, since it only renders the remote screen
Software examples	—	VMware Horizon, Citrix Virtual Apps & Desktops, Microsoft Windows 365

A **thin client** is a lightweight terminal that does little local processing — it exists mainly to render the screen it receives from the data center and to send back keyboard/mouse

input.

1.25.2 Virtual Desktop Infrastructure (VDI)

VDI moves the desktop computing environment off the end-user's device and into centrally managed servers. Users see and interact with a full desktop, but all processing, storage, and applications actually run in the data center — the local device is essentially just a display and input terminal.

- Centralized management, security, and patching.
- Consistent desktop experience across devices.
- Data stays in the data center, not on endpoints.
- Access is controlled differently for local vs. remote users.
- A pool of servers hosts the virtual desktop sessions.

Architecture — Virtual Desktop Infrastructure (VDI)

1.25.3 VDI Access Flow: Local vs. Remote Users

1. The user's device requests a desktop session.
2. The data center's server pool delivers (streams back) the desktop session.
   - **Local users** connect directly to the data center over the trusted internal network.
   - **Remote users** must pass through a **firewall**, which authenticates and filters traffic before granting data-center access.

Architecture — VDI Access Flow

1.25.4 VDI in the Real World

tableheadfill	
Call centers	Hundreds of agents need the same simple desktop; IT sets it up once in the data center instead of configuring hundreds of individual PCs.
Remote / hybrid work	Employees working from home connect securely to their office desktop without company data ever leaving the data center.
Schools & labs	Students get a fresh, identical desktop every session on shared, low-cost devices — no need for powerful individual computers.
Regulated industries	Banks and healthcare providers keep sensitive data centralized and secure, instead of scattered across employee laptops.



Figure 1.7: VDI architecture — a pool of servers in the data center hosts virtual desktop sessions; end-user devices (thin clients / PCs / laptops) connect over the network and receive only the rendered desktop screen, sending back keyboard and mouse input.

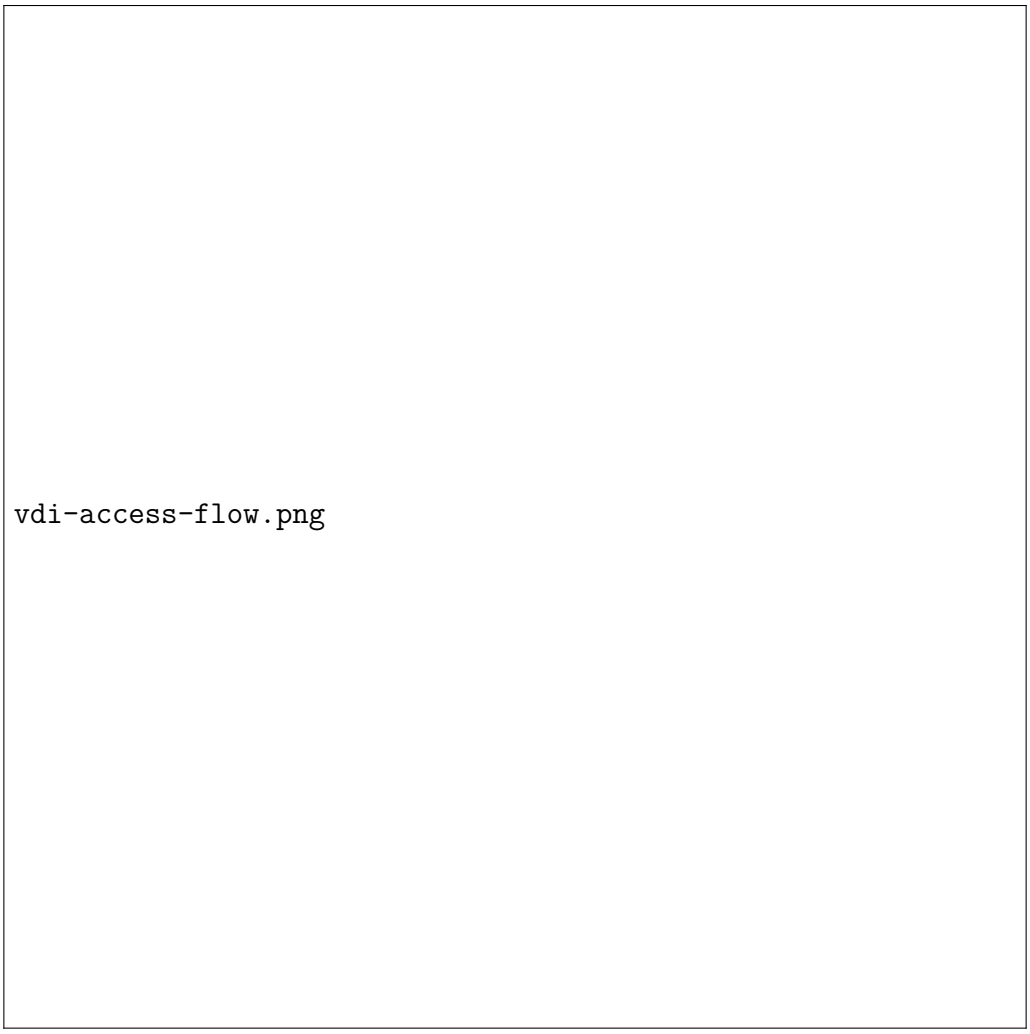


Figure 1.8: VDI access flow — a local user connects directly to the data center's server pool (SERVER, SERVER, ...); a remote user's request is routed through a firewall, which authenticates and filters traffic, before reaching the same server pool. Both flows: (1) request desktop session, (2) desktop session delivered/streamed back.

1.25.5 Popular Virtualization / VDI Software

tableheadfill

VMware ESXi / vSphere	Type 1 hypervisor	Industry-standard bare-metal hypervisor for enterprise data centers
Microsoft Hyper-V	Type 1 hypervisor	Built into Windows Server; widely used for server virtualization
Oracle VirtualBox	Type 2 hypervisor	Free, popular for running VMs on a personal desktop or laptop
VMware Workstation	Type 2 hypervisor	Commercial desktop virtualization for development and testing
KVM (Kernel-based VM)	Type 1 (Linux-integrated)	Open-source hypervisor built into the Linux kernel
Citrix / VMware Horizon	Desktop virtualization (VDI)	Deliver full virtual desktops to thin clients over the network

Quick Revision Summary

- **Virtualization structures:** hypervisor architecture (bare metal), para-virtualization (modified guest OS with hypercalls), and host-based virtualization (VMM on top of a host OS) are the three typical VM architectures. Xen splits guest domains into privileged **Domain 0** and unprivileged **Domain U**.
- **Full virtualization** = binary translation + direct execution; the guest OS is unmodified and unaware it is virtualized.
- **CPU virtualization** traps privileged, control-sensitive, and behavior-sensitive instructions into the VMM; hardware-assisted virtualization adds a Ring -1 for the hypervisor.
- **Memory virtualization** uses a two-stage mapping (virtual → physical → machine memory), plus techniques like ballooning, page sharing/copy-on-write, and memory overcommitment to use RAM efficiently across VMs.
- **I/O virtualization** has three approaches — full device emulation, para-virtualization (split frontend/backend driver model), and direct I/O — and every I/O request passes through two stacks: the guest OS I/O stack, then the hypervisor I/O stack.
- **Desktop Virtualization (VDI)** runs the entire desktop in the data center; thin clients just render the screen. Local users connect directly; remote users pass through a firewall.

1.26 Virtualization for Data Center Automation: The Big Picture

1.26.1 Start With a Simple Problem

Think about a huge data center run by a company like Amazon, Google or Microsoft. It holds **tens of thousands of servers**. Now imagine that every time a customer says, "I need a new server," a person has to walk into the room, pick a machine, install an operating system, connect cables, and set up the network.

That would be slow, expensive, and full of human mistakes. It simply cannot work at cloud scale.

The real-life picture: An old-style telephone exchange needed a human operator to plug a wire in for every single call. Today, calls connect automatically in a fraction of a second. Virtualization does the same thing for servers — it lets *software*, instead of humans, create, move, and remove computers.

1.26.2 What Does "Automation" Mean Here?

Data center automation means that routine jobs happen by themselves, controlled by software:

- Creating a new server when a customer asks for one

- Deciding *which* physical machine should run it
- Adding more servers when many users arrive, and removing them when they leave
- Moving work away from a machine that is overloaded or about to fail
- Switching off machines that are not needed, to save electricity

Core Concept

Virtualization turns hardware into software-controlled building blocks. Once a server becomes a "virtual machine" (a file plus some settings), a program can create it, copy it, move it, or delete it — with no human touching the hardware.

1.26.3 Our Roadmap

This topic has two big parts. Each one answers a different question.

Part	Question it answers	Main ideas
Virtual Clusters	How do we group virtual machines so they work together like one big system?	Physical vs. virtual cluster, architecture, dynamic provisioning, templates, images
Resource Management	How do we share CPU, memory, storage and network fairly and efficiently?	Scheduling, load balancing, consolidation, VM migration, fault tolerance

1.27 Virtual Clusters

1.27.1 First, What Is a Cluster?

A **cluster** is a group of connected computers that work together to do a job, instead of depending on one very powerful computer.

Everyday example — a restaurant kitchen. One cook making 200 meals alone would be slow, and if that cook fell sick the restaurant would close. A team of cooks shares the work, and if one is absent the others carry on. A cluster is that team of cooks, but made of computers.

University example. A university may use three servers:

- Server 1 stores the **student database**
- Server 2 runs the **Learning Management System (LMS)**
- Server 3 runs the **examination system**

Together, these three servers form one cluster.

cluster-university.png

Figure 1.9: A cluster — a university uses three servers (student database, LMS, examination system) that work together.

1.27.1.1 Why Do We Use Clusters?

Benefit	Simple meaning
Higher performance	Many servers share the work, so jobs finish faster than on one machine.
Better availability	If one server is under maintenance, the others keep the service running.
Load sharing	Requests are spread across servers instead of overloading a single one.
Fault tolerance	If one server (node) fails, the whole system does not go down.

Benefit	Simple meaning
Scalability	When demand grows, you can simply add more servers.

1.27.2 What Is a Physical Cluster?

A **physical cluster** is made of real, physical servers connected by a real network (cables and switches). Each server is a separate machine with its own dedicated CPU and RAM.

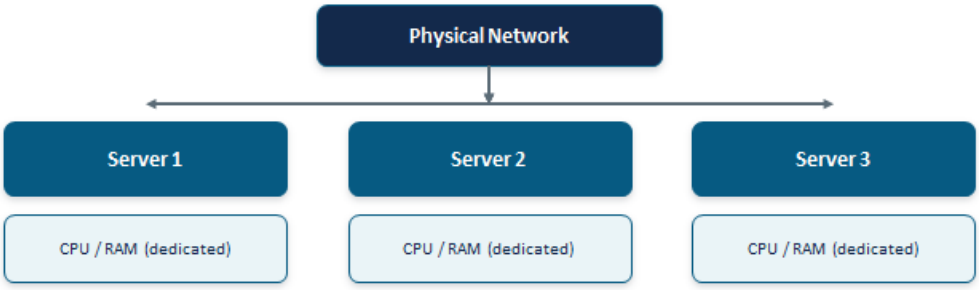


Figure 1.10: A physical cluster — three standalone servers, each with dedicated CPU and RAM, connected through a physical network.

The problem: What if you need one more server? You must buy it, deliver it, install it, and connect it. This is:

- **Expensive** — new hardware costs money
- **Time-consuming** — ordering and installing can take days or weeks
- **Less flexible** — you cannot "shrink" it easily when demand drops

Real-life feeling: It is like buying a new house every time a guest comes to stay, and then being stuck with an empty house after they leave.

1.27.3 Why Do We Need Virtual Clusters?

Now imagine that *one* physical server can run *several* virtual machines (VMs) at the same time. Each VM behaves like an independent computer, with its own operating system and applications.

Stage	What happens
Before	4 applications → 4 physical servers
After	4 applications → 4 VMs → just 1 (or a few) physical servers

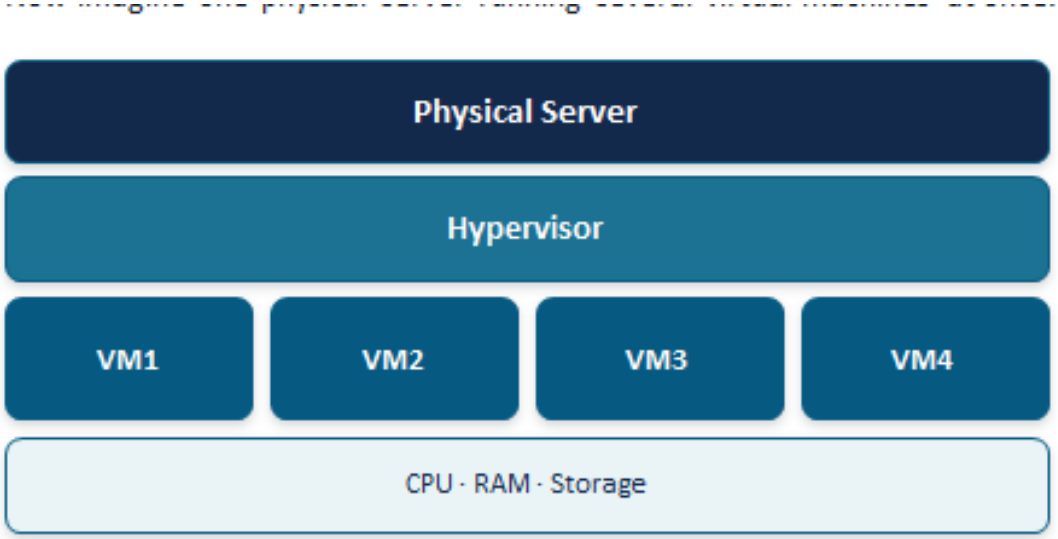


Figure 1.11: One physical server, with a hypervisor on top, running several independent VMs (before vs. after virtualization).

Real-life example — a big flat divided into rooms. Instead of building four small houses for four people, you build one large house and give each person a private room with their own lock and key. The house is the physical server; the rooms are the VMs. Everyone has privacy, but the building is shared and fully used.

So physical capacity is no longer tied one-to-one with applications. This gives better **resource utilization** and much more **flexibility**.

1.27.4 Definition of a Virtual Cluster

Core Concept

A **virtual cluster** is a collection of virtual machines, spread across one or more physical servers, and connected together through a virtual network.

Five features make a virtual cluster special:

1. **VMs may span hosts.** The member VMs can run on different physical machines, not only one.
2. **Virtual networking.** VMs talk to each other over a software-defined virtual network.
3. **Dynamic membership.** VMs can be added to or removed from the cluster whenever needed.
4. **OS heterogeneity.** Different operating systems (for example Linux and Windows) can run side by side on the same physical machine.
5. **Mobility.** A VM can be moved from one physical host to another while it keeps running.

1.27.5 Physical Cluster vs. Virtual Cluster

Physical Cluster	Virtual Cluster
Uses physical servers	Uses virtual machines
Scaling needs new hardware	Scaling can be done by software
Less flexible	Highly flexible
Higher hardware cost	Better use of existing hardware
Migration is difficult	VM migration is possible
Provisioning takes a long time	Provisioning is dynamic and almost instant
Tied to the hardware	More hardware-independent

Small example:

- *Physical cluster:* Server 1 → Application A, Server 2 → Application B, Server 3 → Application C
- *Virtual cluster:* Server 1 → VM1 + VM2, Server 2 → VM3 + VM4, Server 3 → VM5

In the virtual version, the same three servers now host five VMs, so nothing sits idle.

1.27.6 Virtual Cluster Architecture

Now let us see how all the parts fit together. A virtual cluster is built in layers, from the user at the top to the physical hardware at the bottom.

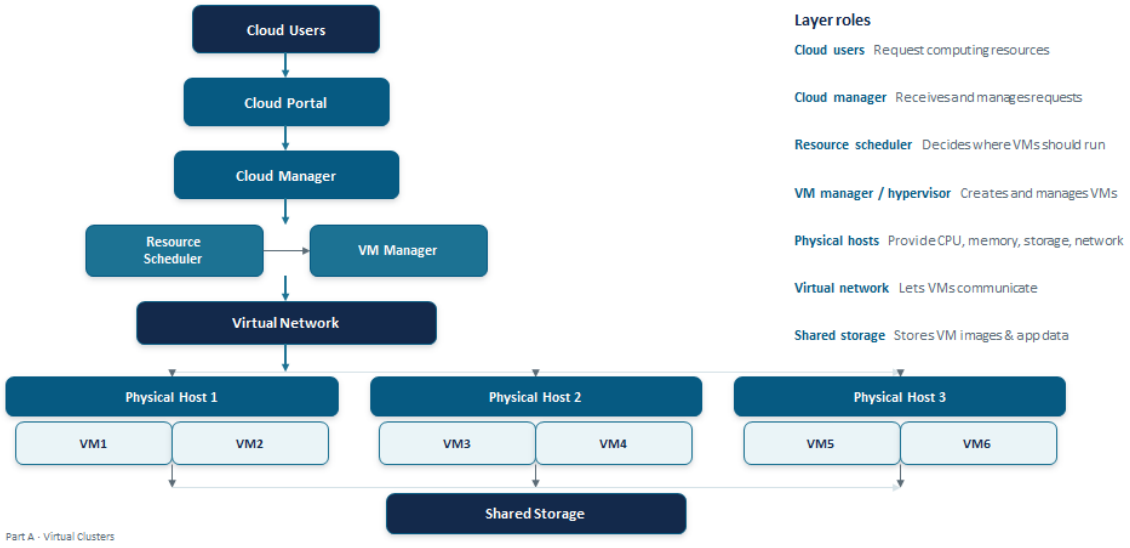


Figure 1.12: Virtual cluster architecture — from cloud users at the top down to physical hosts, VMs and shared storage.

Layer	Role	Hotel comparison
Cloud users	Ask for computing resources	The guest asking for a room
Cloud portal	The website / interface where requests are made	The reception desk
Cloud manager	Receives and manages requests	The hotel manager
Resource scheduler	Decides where each VM should run	The person who assigns room numbers
VM manager / hypervisor	Creates and manages the VMs	Housekeeping preparing the room
Physical hosts	Provide CPU, memory, storage and network	The floors and rooms of the building
Virtual network	Lets VMs communicate	Corridors and lifts
Shared storage	Stores VM images and application data	The central storeroom

1.27.7 How a Virtual Cluster Is Created (Dynamic Provisioning)

Suppose a researcher says:

> "I need a cluster of 5 Linux machines for a Big Data experiment."

The cloud handles this automatically, in these steps:

1. **User request** — the request arrives.
2. **Resource check** — the system checks how much capacity is free.
3. **Select physical hosts** — suitable machines are chosen.
4. **Create VMs** — the 5 virtual machines are made.
5. **Clone VM images** — the operating system and software are copied in.
6. **Configure the virtual network** — addresses and links are set up.
7. **Connect the VMs** — they are joined into one cluster.
8. **Cluster ready** — the user can start working.

Core Concept

This process is called **dynamic provisioning**. The number of nodes in the cluster can automatically increase or decrease depending on demand.

1.27.8 Real-Life Example: An Online Shopping Website

An online store normally needs only a few VMs. But demand is never constant.

Situation	VMs running	What happens
Normal traffic	3 VMs	Everyday shopping
Festival sale	20 VMs	Thousands of shoppers arrive together
After the sale	3 VMs	Extra VMs are removed



Figure 1.13: Elasticity in action — an online shop grows from 3 to 20 VMs during a sale and shrinks back afterwards.

Core Concept

Elasticity means resources increase automatically when demand goes up, and decrease again when demand falls. A fixed physical cluster cannot do this without buying new hardware — a virtual cluster can grow and shrink freely.

Real-life feeling: A restaurant sets out extra tables on a festival day and folds them away the next morning. It does not build a bigger restaurant.

1.27.9 Template VMs: Make One, Copy Many

Imagine a cloud that must create thousands of VMs. Building each one from scratch would take forever. So the cloud first prepares one **template VM**.

- The template holds the common software: operating system, libraries, runtime environment, and common applications.
- Every new VM is a **clone** (copy) of that template.

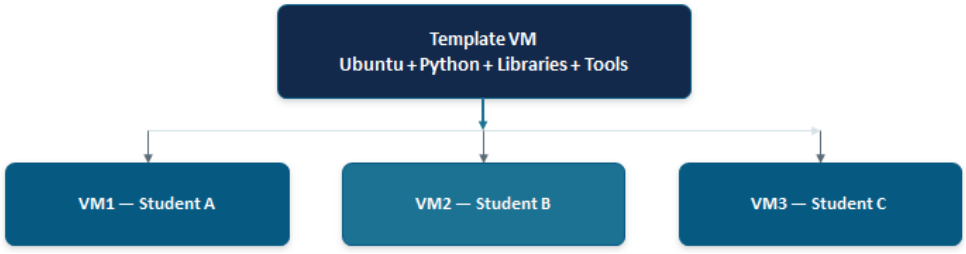


Figure 1.14: A template VM (Ubuntu + Python + libraries + tools) cloned into separate VMs for Student A, B and C.

Real-life examples:

- A teacher prepares one *master* question paper and photocopies it for every student.
- A school computer lab installs the software on one computer and then copies that setup to every other computer.

This reduces deployment time and makes image management much simpler.

1.27.10 VM Image Management

A **VM image** is the saved "package" of a virtual machine. It may contain the operating system, applications, libraries, settings, and user files.

Now think about the size of the job. If a cloud runs **10,000 VMs** and each image is **50 GB**, then:

$$10{,}000 \times 50 \text{ GB} = 500{,}000 \text{ GB} \approx 500 \text{ TB}$$

That is an enormous amount of storage. So clouds need efficient:

Image task	Simple meaning
Storage	Keeping the images safely and compactly
Sharing	Letting many hosts use the same image
Replication	Keeping extra copies in case one is lost
Caching	Keeping popular images close by so they load fast
Distribution	Delivering images quickly to the hosts that need them

Real-life example: Video apps like YouTube keep popular videos on servers near you so they start instantly. Clouds do the same with popular VM images.

Without good image management, dynamic provisioning and templates would be too slow and too costly at large scale.

1.28 Resource Management in the Cloud

1.28.1 What Is Resource Management?

Core Concept

Resource management is the process of allocating, monitoring, and optimizing cloud resources according to the needs of applications.

The main resources are **CPU** (thinking power), **RAM** (short-term memory), **storage** (disk space), and **network** (data connection).

The goal is to give every application what it needs, while avoiding:

- **Overloading** some servers

- **Underutilization** of other servers (sitting idle)
- **Performance degradation** (things becoming slow)
- **Unnecessary cost**

Real-life example: A traffic police officer does not build new roads; the officer directs vehicles so that no single road jams while others stay empty. Resource management does the same with computing work.

1.28.2 Why Is It Needed?

Suppose we have three servers.

Server	CPU usage	Problem
Server A	95%	Almost full — users feel slowness
Server B	20%	Mostly idle — wasted capacity
Server C	15%	Mostly idle — wasted capacity

One server is overloaded while two sit nearly idle. This is **poor resource utilization**. Good resource management shifts some work from A to B and C, so that all three run at a comfortable level (for example, around 40–50% each) and performance stays smooth.

Goal: balance the workload + improve utilization + maintain performance.

1.28.3 Resource Management Architecture

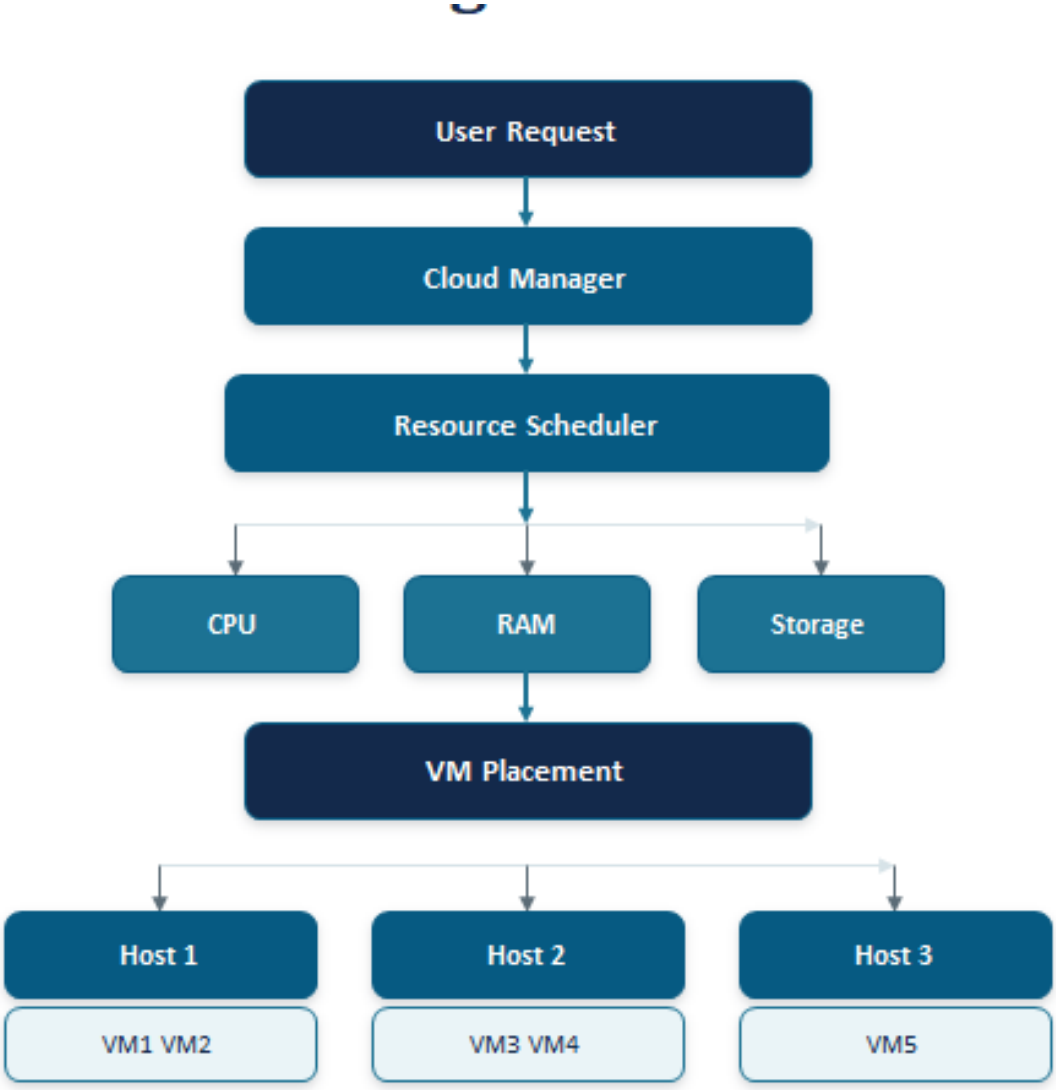


Figure 1.15: Resource management architecture — the Cloud Manager passes each request to the Resource Scheduler, which decides where VMs are placed.

Reading the flow from top to bottom:

1. A user request enters through the **Cloud Manager**.
2. The **Resource Scheduler** checks how much CPU, RAM and storage is available.
3. It decides **VM placement** — which physical host should run which VM.
4. The VMs are placed on hosts. The hosts are continuously monitored, so the cycle can repeat.

1.28.4 Main Functions of Resource Management

Function	What it does	Everyday comparison
Resource allocation	Assigns CPU, memory, storage and network to workloads	Handing out desks in an office
Resource scheduling	Decides which VM gets resources, and where	Assigning rooms in a hotel
Monitoring	Tracks CPU, memory, network and storage usage	A fuel gauge and speedometer
Load balancing	Spreads work evenly across servers	Opening more billing counters in a supermarket
VM migration	Moves VMs between physical hosts	Shifting a shop to a new building
Server consolidation	Packs workloads onto fewer servers	Using fewer buses when few people travel
Fault management	Detects and recovers from failures	A backup generator during a power cut

1.28.5 Worked Example 1: Resource Allocation

A physical host has **16 CPU cores** and **64 GB RAM**. Three VMs are placed on it:

VM	CPU	RAM
VM1	4 cores	16 GB
VM2	6 cores	24 GB
VM3	2 cores	8 GB
Total used	**12 cores**	**48 GB**

$$\text{Remaining capacity} = (16 - 12) = 4 \text{ cores} \quad \text{and} \quad (64 - 48) = 16 \text{ GB}$$

The scheduler keeps a record of how much capacity is already committed. With 4 cores and 16 GB still free, it can accept one more suitably sized VM. If a request needs more than what remains, it must be **rejected or redirected** to another host, so the host does not get overcommitted.

1.28.6 Worked Example 2: Resource Scheduling

Scheduling answers one question: **Which physical machine should run this VM?**

A new VM needs **at least 4 CPU cores and at least 8 GB RAM**. The hosts look like this:

Host	Free CPU	Free RAM	Fits?
H1	2 cores	16 GB	No (CPU too low)
H2	8 cores	4 GB	No (RAM too low)
H3	6 cores	16 GB	**Yes**

Answer: Host 3. It is the only host that satisfies both conditions at once.

Lesson: A scheduler must check *every* resource, not just one. It is like booking a hotel room that needs both a sea view *and* a double bed — a room with only one of the two is not good enough.

1.28.7 Load Balancing

Problem: Host 1 is at 95% CPU, while Host 2 is at 20% and Host 3 is at 25%. Host 1 is overloaded and slow, and the others have plenty of spare capacity.

Solution: Move some of Host 1's workload to Host 2.

Host	Before	After	Change
Host 1	95%	65%	−30%
Host 2	20%	50%	+30%
Host 3	25%	25%	no change

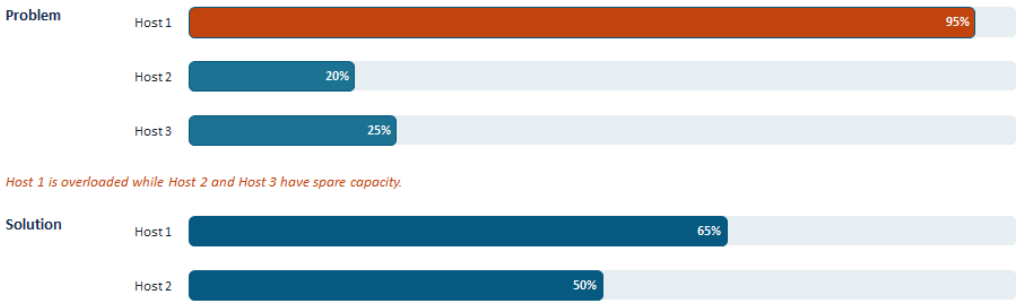


Figure 1.16: Load balancing — moving part of Host 1's work to Host 2 so that no host becomes a bottleneck.

Core Concept

Load balancing continuously redistributes work so that no single host becomes a bottleneck.

Real-life example: In a supermarket, if one billing counter has a long queue and the others are empty, the manager asks people to move to the free counters.

1.28.8 Server Consolidation

Server consolidation means running many workloads on *fewer* physical servers.

Before: four servers, each using only 20–30% of its CPU.

After: the four workloads are packed onto two servers, and the two empty servers are **switched off**.

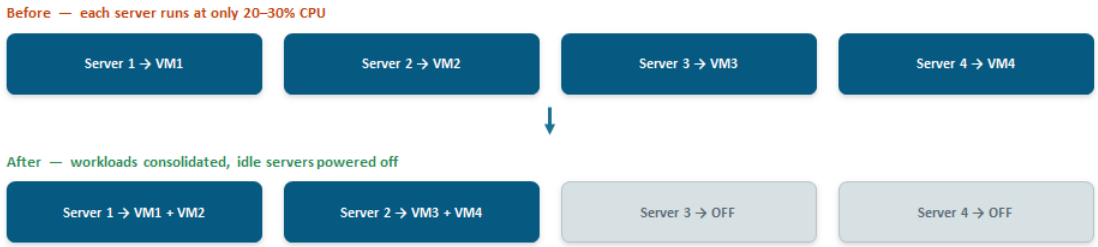


Figure 1.17: Server consolidation — four lightly used servers become two busy servers, and two are powered off.

Advantages

- Lower power consumption
- Lower cooling cost
- Better resource utilization
- Lower infrastructure cost

Real-life example: At night, a hostel keeps the lights on only in the rooms where students actually sleep and switches off the rest. (One caution: do not pack servers so tightly that a sudden rise in demand causes overload — keep some spare capacity.)

Load balancing vs. consolidation: They look like opposites, but they serve different goals. *Load balancing* spreads work out to protect **performance**. *Consolidation* packs work in to save **energy and cost**. A good cloud uses both, depending on the situation.

1.29 VM Migration

1.29.1 What Is VM Migration?

Core Concept

VM migration means moving a virtual machine from one physical host to another.

Example: Host A is running VM1 and VM2. Host B is empty. The cloud moves VM2 from Host A to Host B. Afterwards, VM1 stays on Host A and VM2 works on Host B.

1.29.2 Why Do We Migrate VMs?

Reason	Explanation
Load balancing	Move a VM away from an overloaded server.
Server maintenance	Move VMs off a server before shutting it down for repairs or upgrades.
Fault avoidance	Move workloads away from a host that shows signs of trouble.
Energy efficiency	Consolidate workloads so that unused servers can be switched off.
Performance improvement	Move a VM to a host with better available resources.

1.29.3 Live VM Migration

Core Concept

Live migration moves a *running* VM from one physical host to another with very little interruption to the service.

For example, a web server VM is busy serving users. The cloud moves it from Host A to Host B. The users keep using the website and usually do not even notice.

Real-life example: When you make a phone call while travelling in a train, your call passes from one mobile tower to the next. The call does not drop — you keep talking. Live migration works with the same spirit: the service continues while it moves.

Live migration is considered one of the major design issues in virtual clusters.

The VM's **execution state** — CPU registers, memory contents and device state — is sent across the network. The destination host then takes over and keeps the VM running.

1.29.4 Live Migration Steps

1. **Select the VM** that must be moved.
2. **Select the destination host.** It is chosen and reserved (preparation).
3. **Transfer memory.** Most memory pages are copied first, while the VM keeps running.
4. **Transfer changed memory.** Pages that changed during copying are sent again, in further rounds.
5. **Briefly suspend the VM.** It is paused for a very short time to copy the last small bit of changed state.
6. **Activate on the destination.** The VM resumes on the new host.

1.29.5 Worked Example of Live Migration

Host A is at **95% CPU** and runs three VMs:

VM on Host A	CPU share
VM1	10%
VM2	30%
VM3	40%

The three VMs add up to 80%. The remaining 15% is normally used by the hypervisor and other background work. The cloud manager decides to migrate **VM3** to Host B, which had spare capacity (assume it was at 25%).

Host	Before	After
Host A	95%	about 55%
Host B	25%	about 65%

Small note: If you count only the VMs that stay on Host A (VM1 + VM2 = 10% + 30% = 40%), you get 40%. Adding the 15% background usage gives about 55%. Either way, the two hosts are now much more balanced than 95% against 25%.

1.29.6 Migration Performance Factors

While a VM is being moved, three things have to be balanced:

Factor	Question it asks
Downtime	How long is the service unavailable during the final "suspend and switch" step?
Network bandwidth	How much network capacity is used to send the memory and state?
Migration time	How long does the whole migration take, from start to finish?

Objective: low downtime + low network usage + a reasonable migration time. These three pull against each other. For example, finishing very quickly uses lots of bandwidth; using little bandwidth makes the move slower. Real systems tune the memory transfer to find a good balance.

1.29.7 Memory Migration

A VM's running state lives mostly in **memory** (operating system, applications, running processes, cache and data). So moving a VM mainly means moving its memory.

The challenge: If a VM has 16 GB of memory, sending all of it needs a lot of network bandwidth — and the VM is *still running and changing its memory* while we copy.

Real-life example: Imagine photocopying a 500-page notebook while your friend is still writing in it. By the time you finish, some pages have changed.

1.29.7.1 Iterative Memory Transfer (Pre-Copy)

The solution is to copy memory in **rounds**, without stopping the VM:

1. **Round 1:** copy most of the memory.
2. **Round 2:** copy only the memory that changed during round 1.
3. **Round 3:** copy only the memory that changed during round 2.
4. **Pause:** stop the VM for a very short time and copy the last tiny changes.
5. **Activate:** start the VM on the destination host.

Illustration (numbers are only for understanding):

Round	Data copied	Changed meanwhile
Round 1	16 GB (almost everything)	about 2 GB
Round 2	2 GB	about 200 MB
Round 3	200 MB	about 20 MB
Final pause	20 MB	—

Why does downtime become tiny? Each round only copies what changed since the last round. Because each round is shorter, the VM has less time to change things, so the amount shrinks every time. In the end, so little is left that the VM needs to pause only for a moment.

1.29.8 File System Migration

The destination host must also be able to reach the VM's files. There are two common approaches.

Approach	How it works	Trade-off
1. Move the virtual disk	The whole virtual disk is copied to the destination before (or while) the VM moves.	Simple, but may move a lot of data.
2. Shared / distributed file system	All hosts can reach the VM's files directly, so nothing needs to be copied.	Needs shared storage, but migration is much faster.

Real-life example: Approach 1 is like carrying your files from one computer to another in a pen drive. Approach 2 is like keeping your files in Google Drive — every device can open them, so there is nothing to carry.

1.29.9 Network Migration

After moving, the VM must keep talking to other systems. Each VM carries a **virtual IP address** (its network address) and a **virtual MAC address** (its hardware-style ID). These move *with* the VM.

Because the VM keeps its network identity, existing connections can continue with very little disruption. The client still talks to "VM1" and does not care that VM1 now sits on Host B.

Real-life example: Mobile number portability. You can change your phone company or city but keep the same number, so everyone can still reach you.

The three parts of a migration together: *memory* (what the VM is thinking), *files* (what the VM has stored), and *network identity* (how others find the VM). All three must move correctly for the VM to feel "unmoved."

1.30 Dynamic Resource Allocation and Elasticity

1.30.1 Dynamic Resource Allocation

Cloud demand changes all the time, so the resources given to an application must change with it.

- Low demand → 2 VMs
- High demand → 10 VMs
- Very high demand → 20 VMs
- When demand decreases, the number scales back down: 20 → 10 → 2 VMs

Two useful words:

- **Scale out:** add more VMs when demand rises.
- **Scale in:** remove VMs when demand falls.

This is the direct mechanism behind the elasticity in the online-shopping example.

1.30.2 The Resource Management Decision Cycle

Resource management is not a one-time action. It is a **continuous feedback loop**:

1. **Monitor** — collect live CPU, memory and network data from every host.
2. **Analyze** — study the data to spot overload or underuse.
3. **Decide** — make a scheduling decision (allocate more? migrate a VM?).
4. **Act** — allocate or migrate resources. Then the cycle repeats.

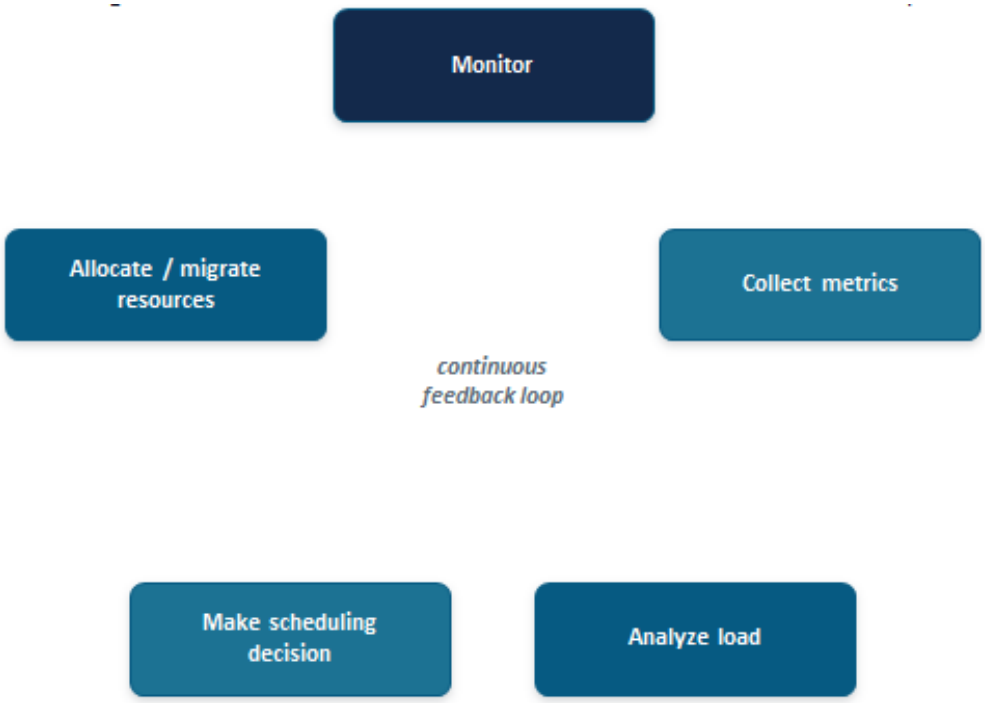


Figure 1.18: The resource management decision cycle — monitor, analyze, decide, act, and repeat.

Real-life example: A driver keeps checking the road, decides whether to speed up or slow down, presses the pedal, and then checks again — all the time.

1.30.3 Example: A University Cloud

A university cloud automatically changes the number of VMs to match the daily rhythm of college life:

Time	Situation	VMs	Level
Morning	Normal workload	10 VMs	Normal
10 AM	Online classes	20 VMs	Higher
Exam time	Peak demand	50 VMs	Peak
Night	Minimal usage	5 VMs	Low

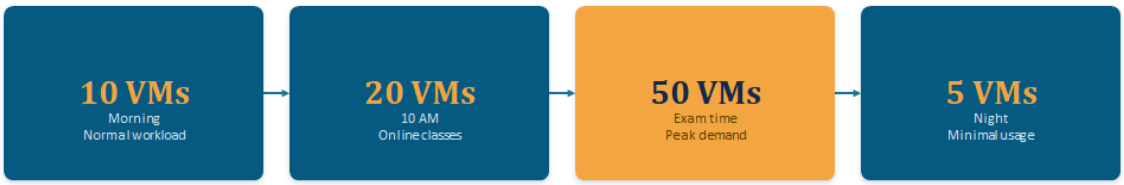


Figure 1.19: A university cloud scales up for online classes and exams, and scales down at night to save cost and energy.

1.30.4 Static vs. Dynamic Allocation

Aspect	Static allocation	Dynamic allocation
Meaning	Resources are fixed in advance and do not change.	Resources change according to the real-time workload.
Example	VM1, VM2 and VM3 always get 4 CPUs each.	Low load → fewer resources; high load → more resources.
Good point	Simple.	Flexible and efficient.
Weak point	Less flexible, so resources are wasted or short.	Needs monitoring and smarter software.
Suitability	Steady, predictable work.	**Cloud environments**, where workload is rarely constant.

Real-life example: Static allocation is like a fixed monthly electricity plan. Dynamic allocation is like a meter — you use, and pay for, what you need.

1.30.5 Summary of Resource Management Techniques

Technique	Purpose
Resource allocation	Assign resources to workloads
Scheduling	Decide where workloads run
Load balancing	Distribute workloads evenly
VM migration	Move running workloads between hosts
Server consolidation	Reduce the number of active servers
Autoscaling	Increase or decrease resources with demand
Monitoring	Observe resource usage continuously
Fault tolerance	Handle failures gracefully
Energy management	Reduce power consumption

1.30.6 Fault Tolerance

Virtualization makes recovery and relocation far more flexible than on plain physical hardware. Because a VM is just software (files and settings), it can be restarted on another machine.

Without fault tolerance: If Host A fails, VM1, VM2 and VM3 all fail with it.

With fault tolerance:

1. Host A fails.
2. The system **detects** the failure.
3. It **restarts or activates** the VMs elsewhere.
4. The VMs **resume on Host B**.

Real-life example: A hospital has a backup generator. When the main electricity fails, the generator starts, and the operation theatre keeps working.

1.31 Virtual Cluster Management Models

Who is in charge of managing the cluster? There are four common models:

Model	Meaning
Guest-based management	The cluster manager runs *inside* the guest VMs.
Host-based management	The cluster manager runs at the physical host / infrastructure level.
Independent host and guest management	Separate management systems work at both levels.
Integrated management	Host and guest resources are managed together, as one system.

1.31.1 Guest-Based vs. Host-Based Management

Aspect	Guest-based	Host-based
Manager location	Runs inside the VM	Runs on the host / infrastructure
Visibility	Sees virtual resources	Sees physical resources
Best suited for	Virtual-cluster-aware software	Better infrastructure control
Involves	The guest OS	The hypervisor / host

Real-life comparison: In a rented flat, a *guest-based* manager is like a tenant who organizes things only inside their own flat. A *host-based* manager is like the building manager who controls the lifts, water and electricity for the whole building.

The virtual cluster adds one extra layer (the VM and the hypervisor). This costs a little overhead, but in return you get much greater **flexibility, portability, and easier scaling**.

1.32 Advantages and Challenges of Virtual Clusters

Advantages

Technical advantages

- Better resource utilization
- Dynamic scaling
- VM isolation (one VM cannot easily disturb another)
- Easy provisioning
- VM migration
- Fault tolerance
- Load balancing
- Hardware independence

Economic advantages

- Lower hardware requirements
- Lower energy consumption
- Reduced maintenance
- Better use of existing infrastructure

Disadvantages

1. **VM overhead** — virtualization itself uses some CPU and memory.
2. **Migration overhead** — moving VMs uses network bandwidth.
3. **Resource contention** — many VMs compete for the same physical resources.
4. **VM image management** — huge numbers of images need efficient storage.
5. **Security** — a compromised VM may threaten the shared infrastructure.
6. **Scheduling complexity** — choosing the best host becomes harder at large scale.

Real-life feeling for contention: Flatmates sharing one kitchen and one bathroom. Usually it works well, but in the morning rush everyone waits.

1.33 Putting It All Together

1.33.1 Complete Resource Management Flow

Here is the whole story from the moment a user clicks a button:

1. **Users** submit requests through the **Cloud Portal**.
2. The **Cloud Manager** coordinates the request.
3. The **Resource Monitor** supplies live usage data, and the **Resource Scheduler** decides the placement.
4. **VM Provisioning** creates the VM through the **Hypervisor**.
5. The VM runs on a **physical host**, backed by **shared storage** and connected over the **virtual network**.

6. The Resource Monitor keeps watching, closing the loop (the decision cycle seen earlier).

1.33.2 End-to-End Example: A Machine-Learning Experiment

> "Create a virtual cluster with 4 VMs for a machine-learning experiment."

1. The cloud **receives** the request.
2. The scheduler **checks available resources**.
3. Four suitable VMs are **created**.
4. The VMs are **connected** through a virtual network.
5. VM images are **cloned from a template**.
6. The ML application is **deployed**.
7. The resource manager **monitors** CPU, RAM and network.
8. If Host 1 becomes overloaded, **VM2 live-migrates** to Host 3.
9. After completion, the 4 VMs are **terminated** and the resources are **released** for others.

Notice how this one example uses almost everything we studied: provisioning, templates, virtual networking, monitoring, live migration and release of resources.

1.33.3 Another Example: An E-Commerce Company

An e-commerce company runs **50 VMs across 5 physical servers**.

Moment	CPU	What the resource manager does
Normal operation	30%	Nothing special — just monitors
During a sale	90%	**Scale out:** create more VMs, distribute the workload, monitor performance
After the sale	falls back	**Scale in:** remove unneeded VMs, release resources

1.34 Virtualization for Data-Center Automation

1.34.1 What Is Data-Center Automation?

In recent years, data centers have grown very fast. Big technology companies such as Google, Amazon, Microsoft, IBM, HP, Apple and Yahoo! have spent billions of dollars building and automating them.

Core Concept

Data-center automation means that the huge pool of hardware, software and database resources inside a data center is handed out **dynamically** (by software, on demand) to millions of Internet users at the same time — with guaranteed **Quality of Service (QoS)** and at a reasonable cost.

Real-life example: Think of a huge railway reservation system. Millions of people book tickets at the same time, and nobody at the railway office manually assigns each seat. Software does it in seconds. Data-center automation is the same idea, but for servers, storage and software.

This automation is being pushed forward by two things: the growth of **virtualization products** and the spread of **cloud computing services**.

1.34.1.1 Market Trend (A Little History)

A market report from IDC (published in 2007) estimated that virtualization business was worth about $1,044 million in 2006 and could grow to about $3.2 billion by 2011. Early on, most of the money came from *production consolidation* and *software development*. Later, attention moved toward:

- **High availability (HA)** — keeping services running even when something fails
- **Backup services** and disaster recovery
- **Workload balancing**
- **Utility computing** (pay for what you use) and more virtual clients
- **Mobility** (moving VMs around) and less *planned downtime* for maintenance

(These figures are old, but they show the direction: automation, service orientation, policy-based control and variable, pay-per-use costs.)

1.34.1.2 The Four Topics We Will Study

#	Topic	The question it answers
1	Server consolidation	How do we stop wasting half-empty servers?
2	Virtual storage management	How do we store thousands of VM disks efficiently?
3	Cloud OS for virtualized data centers	Which software manages the whole virtualized data center?
4	Trust management	How do we keep a shared, virtualized data center secure?

1.34.2 Server Consolidation in Data Centers

1.34.2.1 The Problem: Two Kinds of Workloads

A data center runs many different kinds of work at different times. Roughly, they fall into two groups:

Workload type	Meaning	Example
Chatty workload	Interactive. It has busy bursts and quiet periods, depending on how many people are using it.	A web video service: crowded at night, almost empty in the daytime
Non-interactive workload	Once submitted, it keeps running on its own without a person guiding it.	A scientific simulation (high-performance computing), or a big report that runs overnight

These workloads need very different amounts of resources at different moments. But to make sure the service never fails at its busiest time, the usual habit is **static allocation**: reserve enough resources for the *peak* demand and keep them reserved all the time.

Result: Most servers in a data center are **underutilized**. Hardware, floor space, electricity and management effort are all wasted.

Real-life example: A hotel that keeps every room lit and air-conditioned all day, just because on a few festival nights every room may be occupied.

1.34.2.2 The Solution: Server Consolidation

Core Concept

Server consolidation improves the poor usage of hardware by **reducing the number of physical servers** that are needed.

There are several ways to consolidate (for example *centralized* and *physical* consolidation), but **virtualization-based consolidation is the most powerful**. Why?

- Older techniques work with a *whole machine* as the smallest unit. So resource management stays far from ideal.
- Server virtualization can hand out **smaller slices** than one whole physical machine (a few CPU cores, a few GB of memory).

A small calculation. Suppose a company has 10 servers, and each one is busy only 15% of the time.

$$\text{Total work} = 10 \times 15\% = 150\% \text{ of one server} = 1.5 \text{ servers' worth}$$

If we put this work into VMs on **3** servers, each server is about $1.5/3 = 50\%$ busy — a healthy level. So **7 servers** can be switched off, saving power, cooling and space. (In real life we also keep some spare room for sudden bursts.)

1.34.2.3 Benefits of Virtualization-Based Consolidation

Advantages

1. **Better hardware utilization.** Many underused servers become a few well-used servers. Backup and disaster-recovery services also become easier.
2. **Faster, more agile provisioning.** The images of guest operating systems and their applications can simply be *cloned and reused*.
3. **Lower total cost of ownership.** New server purchases can be postponed, the data center needs less space, and maintenance, power, cooling and cabling costs go down.
4. **Better availability and business continuity.** If one guest OS crashes, the host OS and the other guests are not affected. Moving a VM to another server is easy because virtual servers do not depend on the details of the underlying hardware.

Disadvantages

- Using many VMs makes resource management **more complex**. The big challenge is how to improve utilization *and* still guarantee QoS.

1.34.2.4 What Must Be Considered to Automate a Data Center?

- Resource **scheduling**
- **Architectural support** from the hardware and software
- **Power management**
- **Automatic (autonomic) resource management** — the system manages itself
- **Performance analytical models** — mathematical models that predict performance

In a virtualized data center, an **efficient, on-demand, fine-grained scheduler** is one of the most important tools for better utilization.

1.34.2.5 Scheduling at Three Levels

Scheduling and reallocation can happen at three levels:

Level	What is decided
VM level	How much CPU and memory each VM inside a server gets
Server level	How a physical server shares its resources among its VMs
Data-center level	Which servers are used, where new VMs are placed, and which servers can rest

Ideally, all three levels would work together. But that is very complicated, so present techniques usually handle **only one level, or at most two**.

1.34.2.6 Dynamic CPU Allocation and the Two-Level Controller

CPU is allocated dynamically using two kinds of information: how busy each VM is, and **application-level QoS metrics** (for example, "the web page must load within 2 seconds"). Some methods adjust both CPU and memory automatically as the workload of a hosted service changes.

To tame the complexity, one scheme uses a **two-level resource management system**:

- A **local controller** at the **VM level**
- A **global controller** at the **server level**

They talk to each other and together decide the resource allocation *autonomically* (without a human). **Real-life example:** In a college, each class teacher (local controller) knows what their class needs — projector, extra chairs, a bigger room. The principal's office (global controller) sees the whole college and decides who gets what. Neither can do the job well alone.

1.34.2.7 Multicore Processors and Power Budgeting

Multicore processors (chip multiprocessors, CMPs) and virtualization can help each other, but multicore chips are still not used in the best possible way. For example, the **memory system** of a multicore chip can be improved by designing a **virtual hierarchy** for it. The goals are:

- Shorter memory access time
- Less **interference between VMs** (one noisy VM slowing its neighbours)
- Easier reassignment of VMs
- Safe sharing of data between VMs

Another idea is a **VM-aware power budgeting** scheme, where several managers work together to control power. Such policies must handle **heterogeneous** (mixed) hardware, and they always involve a **trade-off**: the more power you save, the more you risk data-center performance.

1.34.3 Virtual Storage Management

1.34.3.1 What Does "Storage Virtualization" Mean Here?

The words "storage virtualization" existed long before system virtualization became popular, and the meaning has changed.

When	Meaning
Older meaning	Combining and re-cutting physical disks, at a very coarse level, for use by *physical* machines.

When	Meaning
In system virtualization	The storage managed by the **VMM (hypervisor)** and by the **guest operating systems**.

The data stored in a virtualized environment is of two kinds:

- **VM images** — special to virtual environments (the "packaged" VMs)
- **Application data** — everything else, the same as in a normal OS

1.34.3.2 Encapsulation and Isolation

The two most important ideas of system virtualization are:

- **Encapsulation:** an operating system and its applications are wrapped up inside a VM as one neat package.
- **Isolation:** many VMs can share one physical machine, yet each is completely separated from the others.

Real-life example: Think of shipping containers. Each container packs its goods (encapsulation), and many containers can sit on the same ship without their contents mixing (isolation).

To support these ideas, CPUs, chipsets and system software have improved quickly. But **storage is lagging behind**, and so **storage has become the main bottleneck of VM deployment**.

1.34.3.3 Why Is Storage So Difficult to Manage?

1. **The guest is being fooled.** A guest OS behaves as if it uses a real hard disk, but it is not allowed to touch the disk directly. The VMM sits in between.
2. **Many VMs fight for one disk.** When lots of VMs run on one machine, they compete for the same storage device.
3. **The VMM has a harder job** than a normal OS, because it must handle the requests of *all* guests.
4. **Storage tools are not nimble.** Actions like moving (remapping) a volume between hosts or taking a checkpoint of a disk are often clumsy, awkward, or sometimes not possible at all.
5. **Image flood.** Data centers may hold thousands of VMs, so the number of VM images becomes huge.

Researchers therefore try to make storage **easier to manage**, **faster**, and **smaller** (less space taken by VM images).

1.34.3.4 Two Ideas: CAS and Parallax

- **Content Addressable Storage (CAS):** a technique that reduces the total size of VM images. Data is stored and found by *what it contains*, so content that is the

same in many images is kept only once instead of being saved again and again.

- **Parallax:** a distributed storage system designed specially for virtual environments.

Real-life example for CAS: A library keeps one copy of a textbook, and every student's reading list simply *points* to it, instead of storing 500 identical copies.

1.34.3.5 Example: How Parallax Works

Ordinary storage management was designed without virtual environments in mind. Parallax takes a different path. Storage functions that used to sit inside expensive, high-end storage arrays and switches are **moved out into a group (federation) of storage VMs**. These storage VMs run on the *same physical hosts* as the VMs they serve.

Key points of the architecture:

- Each physical machine gets a special **storage appliance VM**. It works as a **block virtualization layer** between the client VMs and the physical storage device.
- The appliance VM gives **each VM its own virtual disk**, even though all of them share one large physical disk system.
- A **cluster-wide administrative domain** manages all the appliance VMs, so storage management is easy for the administrator.
- Features that once needed special hardware (like **snapshots**) can now be done in **software**, on top of ordinary, inexpensive network storage.
- Parallax itself runs as an ordinary **user-level application** inside the appliance VM. It works with both full virtualization and para-virtualization.
- The core object is the **Virtual Disk Image (VDI)**: a single-writer virtual disk that can be reached from *any* host of the Parallax cluster, no matter where it physically sits (location-transparent).
- It uses Xen's **block tap driver** (a *tapdisk* library) as one block-virtualization service for all client VMs on a host.
- The real hardware drivers live inside the appliance VM, so an administrator can **upgrade block device drivers while the cluster is running**.

Real-life picture: Imagine a big office building with a common warehouse. Instead of one central store-keeper far away, *every floor* has its own store-keeper (the storage appliance VM). Each employee (VM) is given a personal locker (virtual disk), while all lockers actually sit in the same warehouse. If the store-keeper's tools need an upgrade, it can be done on that floor without closing the whole office.

1.34.4 Cloud OS for Virtualized Data Centers

1.34.4.1 What Is a Cloud OS?

To act as a cloud provider, a data center must first be virtualized. The software that does this is called a **virtual infrastructure (VI) manager** or **cloud OS**. It creates

VMs and groups them into **virtual clusters** that act as elastic resources. It is like an operating system, but for a whole data center instead of a single computer.

1.34.4.2 Four Well-Known VI Managers

Name	Type	Hypervisors used	Notable features
Nimbus	Open source	Xen, KVM	Virtual networking
Eucalyptus	Open source	Xen, KVM	Virtual networking; IaaS private clouds
OpenNebula	Open source	Xen, KVM	Adds dynamic resource provisioning and advance reservation
vSphere 4	Proprietary (VMware)	ESX, ESXi	Virtual networking + **virtual storage** + data protection

Points to remember: Nimbus, Eucalyptus and OpenNebula are free to use (open source). Only vSphere 4 is a commercial (proprietary) product. Only vSphere 4 supports virtual storage as well as virtual networking and data protection.

1.34.4.3 Example 1: Eucalyptus

What is it? Eucalyptus is open-source software mainly meant for building **IaaS** clouds, especially **private clouds**. It mainly provides **virtual networking** and **VM management**; it does *not* provide virtual storage. A Eucalyptus cloud can talk to users over Ethernet or the Internet, and can also talk to other private or public clouds. It is short on security and some other features needed for general-purpose grid or cloud use.

How is it built? Each high-level part is a stand-alone **web service** with a clear, language-independent interface (described in a WSDL document). Secure communication between parts uses web-service security policies (WS-Security).

Manager	What it does	Company comparison
Instance Manager	Runs on a host; controls the execution, inspection and termination of VM instances there	Shop-floor supervisor
Group Manager	Collects information, schedules VM execution on specific instance managers, and manages the virtual instance network	Regional manager

Manager	What it does	Company comparison
Cloud Manager	The entry point for users and administrators; asks for resource information, makes scheduling decisions and passes them to group managers	Head office

Eucalyptus is designed to work like the **Amazon Web Services (AWS)** APIs, so it can interact with Amazon EC2. It also offers a storage API that imitates Amazon S3 for storing user data and VM images. It installs on Linux, is EC2-compatible (SOAP and Query) and S3-compatible (SOAP and REST), and can be used through a command-line interface or a web portal.

1.34.4.4 Example 2: VMware vSphere 4 (a Commercial Cloud OS)

What is it? vSphere 4 is a commercial cloud OS released by VMware in April 2009. It extends earlier VMware products (VMware Workstation, ESX for server virtualization, and Virtual Infrastructure for clusters). It is mainly used to virtualize and manage data-center resources for **private clouds**. VMware described it as the first cloud OS that supports **availability, security and scalability**. Users and applications reach it through an interface layer called **vCenter**.

vSphere 4 has two groups of software:

1. Infrastructure services (three packages).

Package	Built from	Simple meaning
vCompute	ESX, ESXi, DRS	Virtualizes the processors and memory
vStorage	VMFS, thin provisioning	Manages virtual disks. *Thin provisioning* gives a VM disk space only as it actually uses it.
vNetwork	Distributed switching	Virtual network switches that can span many hosts

2. Application services (three groups).

Group	Features	Simple meaning
Availability	VMotion, Storage VMotion, HA, Fault Tolerance, Data Recovery	*VMotion*: live migration of a VM. *Storage VMotion*: moving a VM's disk while it runs. *HA*: restart VMs on another host if a host fails. *Fault Tolerance*: a standby copy that takes over instantly. *Data Recovery*: backup.
Security	vShield Zones, VMsafe	*vShield Zones*: firewall-like protection between groups of VMs. *VMsafe*: lets security products inspect VMs safely.
Scalability	DRS, Hot Add	*DRS*: automatically balances load across hosts. *Hot Add*: add CPU or memory to a running VM without a restart.

These infrastructure functions talk to the servers, disks and networks of the data center, and also to other external clouds.

Note: vSphere 4 is a 2009-era product; newer versions exist. The examples are used in textbooks because they show the *ideas* clearly: manage compute, storage and network together, and add availability, security and scalability on top.

1.34.5 Trust Management in Virtualized Data Centers

1.34.5.1 Why Does Virtualization Bring New Security Questions?

A VMM adds a new layer of software between the operating systems and the hardware. A VM **completely encapsulates** the state of the guest OS running inside it. That state can be **copied, shared over a network, and deleted like an ordinary file**. This is very handy — and it is also a security challenge.

- The VMM gives **secure isolation**, and every VM reaches the hardware only under the VMM's control. So the **VMM is the foundation of security** for a virtual system.
- Usually one VM acts as a **management VM**, which has special powers: create, suspend, resume and delete VMs.
- If a hacker breaks into the VMM or the management VM, **the whole system is in danger**.

Real-life example: A building's security depends on the master key and the security-control room. If a thief gets those, every flat is at risk.

1.34.5.2 A Subtle Problem: Rolling Back a VM

Some security methods depend on a random number being **fresh** (never used before), for example when creating a session key.

Now suppose a VM is **rolled back** (restored to an earlier snapshot) at a moment *after* a random number was chosen but *before* it was used. The VM continues, and picks up the same "fresh" number **again**. Two different messages may then be encrypted with the same key stream (in a stream cipher), which could expose both messages. Also, reusing TCP initial sequence numbers can allow **TCP hijacking** attacks.

Real-life example: It is like a video game. You roll the dice, then reload an old save — and get exactly the same dice roll. In a game that is harmless; in security, a "one-time" secret that repeats is no longer secret.

1.34.5.3 VM-Based Intrusion Detection

An **intrusion** is unauthorized access to a computer by a local or network user. An **intrusion detection system (IDS)** recognizes such access.

Type	How it works	Weakness
HIDS (host-based)	Runs on the monitored system itself	If the system is attacked, the HIDS can be attacked too
NIDS (network-based)	Watches network traffic	Cannot see what really happens inside the host, so it can miss fake actions

Real-life comparison: A HIDS is a guard standing *inside* the room — he sees everything, but a burglar can overpower him. A NIDS is a camera at the gate — it is safe, but it cannot see inside the rooms.

The virtualization-based approach combines the good points of both.

- The VMM **isolates** guest VMs on the same hardware. Even if one VM is broken into, the others are not affected (like a NIDS).
- The VMM also **monitors and audits** every request for hardware and system software, so fake actions can be caught (like a HIDS).

There are two ways to build a VM-based IDS:

1. The IDS runs as an **independent process in each VM**, or as a **high-privileged VM** on top of the VMM.
2. The IDS is **built into the VMM** and has the same privilege as the VMM for accessing hardware.

Garfinkel and Rosenblum proposed running the IDS as a **high-privileged VM on the VMM**.

- The IDS has a **policy engine** and a **policy module**. The policy framework watches events in different guest VMs through an OS interface library, and PTrace helps to follow the monitored host's activity against the security policy.
- **Logs matter.** It is hard to predict and stop every intrusion immediately, so *analysis after the attack* is very important. Most systems use logs for this, but a log can lose its credibility if an attacker can change it. A good design keeps the log service safe even when the guest OS has been invaded — which the VM-based design helps to do, because the IDS sits outside the guest.

Real-life comparison: A VM-based IDS is a security supervisor sitting in a control room outside all the rooms. Even if a burglar captures one room, the supervisor can still watch, record and raise the alarm.

1.34.5.4 Honeypots and Honeynets

Core Concept

A **honeypot** is a deliberately weak, fake system that pretends to be a real one. Its purpose is to attract attackers, watch what they do, and learn from it. A group of honeypots is a **honeynet**.

- Attackers see a fake view of the system, so the **real system stays protected**.
- Their actions can be analyzed, which helps to build a **better, secure IDS**.
- A honeypot can be **physical** or **virtual**. In a virtual honeypot, a guest OS and its applications form a VM.
- The host OS and VMM must be protected so that an attack cannot escape from the honeypot VM.

Real-life example: Police sometimes leave a "bait" wallet or a decoy car to catch thieves and study how they work.

1.34.5.5 Example: Trusted Zones for Virtual Clusters (EMC and VMware)

EMC and VMware together built **security middleware** for trust management in distributed systems and private clouds. The main idea is the **trusted zone**. Each tenant's virtual cluster (many applications and operating systems) is placed in its own separate virtual environment, forming a zone.

- The physical infrastructure at the bottom belongs to the **cloud provider**.
- Above it are the **virtual clusters of different tenants**, and at the top is the **public cloud** with global users.
- Security actions are applied at each level, between users, tenants and the provider: **antivirus, worm containment, intrusion detection, encryption and decryption**, and others.
- The main innovation is to **build trust zones between virtual clusters**.

- The end result is an **end-to-end view of security events and compliance** across all the virtual clusters used by different tenants.

Real-life example: A large apartment building. Each family has its own locked flat (a zone), while the building also has guards, CCTV and a visitor register that watch the whole building. A neighbour cannot walk into your flat, yet the building manager still has a complete view of what is happening.

1.34.6 Putting the Four Topics Together

Topic	Problem	Main idea
Server consolidation	Servers are sized for peak, so most sit idle	Pack VMs on fewer servers; schedule at VM, server and data-center levels
Virtual storage management	Storage is the bottleneck of VM deployment; images flood the data center	CAS shrinks images; Parallax gives each VM a virtual disk through storage appliance VMs
Cloud OS	Someone must create and manage VMs and virtual clusters at scale	Nimbus, Eucalyptus, OpenNebula (open source) and vSphere 4 (commercial)
Trust management	Shared hardware and copyable VMs create new security risks	Protect the VMM; use VM-based IDS, honeypots and trusted zones

1.34.7 Likely Exam Questions and Short Answers

Marks	Question
2 marks	What is data-center automation? What is server consolidation? Differentiate chatty and non-interactive workloads. What is a VDI in Parallax? What is a honeypot?
5 marks	Explain the benefits of virtualization-based server consolidation. Explain virtual storage management and the Parallax architecture. Compare Nimbus, Eucalyptus, OpenNebula and vSphere 4. Explain VM-based intrusion detection.
10 marks	Explain virtualization for data-center automation: server consolidation, virtual storage management, cloud OS, and trust management, with suitable diagrams and examples.

Model short answers:

- **Data-center automation:** allocating hardware, software and data resources of a data center dynamically to millions of users at once, with guaranteed QoS and low cost.
- **Server consolidation:** reducing the number of physical servers by running their work as VMs on fewer machines, to improve utilization.
- **Chatty workload:** interactive work with bursts and quiet periods (e.g., a video service). **Non-interactive workload:** runs by itself after submission (e.g., high-performance computing).
- **VDI (Parallax):** a single-writer virtual disk that any host in the cluster can access, regardless of where it is physically stored.
- **Honeypot:** a deliberately weak fake system used to attract and study attackers.

1.34.8 Quick Revision Summary

- **Data-center automation** = dynamic, software-driven handing out of resources, with QoS and low cost; driven by virtualization and cloud services.
- **Two workload types:** chatty (bursty) and non-interactive (self-running). Static peak allocation wastes resources.
- **Consolidation benefits:** better utilization, faster provisioning by cloning, lower cost of ownership, better availability.
- **Scheduling levels:** VM, server, data center. Two-level control = local controller (VM) + global controller (server).
- **Storage:** data = VM images + application data. Storage is the bottleneck. **CAS** shrinks images; **Parallax** uses a storage appliance VM per host and single-writer VDIs.
- **Cloud OS:** Nimbus, Eucalyptus, OpenNebula (open source, Xen/KVM); vSphere 4 (proprietary, ESX/ESXi, adds virtual storage).
- **Eucalyptus:** Instance Manager → Group Manager → Cloud Manager; AWS-compatible; IaaS private clouds.
- **vSphere 4:** vCenter + infrastructure services (vCompute, vStorage, vNetwork) + application services (availability, security, scalability).
- **Trust:** VMM and management VM are critical; VM rollback can reuse "fresh" random numbers; VM-based IDS combines HIDS and NIDS advantages; honeypots lure attackers; trusted zones isolate tenants' virtual clusters.

1.34.9 The Big Question: Where Does Your Request Run?

"When you open an online shopping app, how does the cloud decide where your request should run?"

The answer in plain words: Your request reaches the cloud. The *resource manager* looks at how busy every machine is. The *scheduler* picks a physical host that has enough free CPU, memory and storage. A *virtual machine* on that host runs your request, and

the response comes back to your phone — all within a moment, with no human involved. If that host becomes too busy later, the cloud can quietly migrate the VM elsewhere.

1.35 Practice Section

1.35.1 Group Problem: Where Should the VM Go?

Three hosts, each with 16 cores. A new VM needs **4 CPU cores**.

Host	CPU capacity	Current usage
H1	16 cores	90%
H2	16 cores	30%
H3	16 cores	40%

Step 1 — Calculate the free cores.

Host	Used	Free	Can take 4 cores?
H1	$0.90 \times 16 = 14.4$	1.6	No
H2	$0.30 \times 16 = 4.8$	11.2	Yes
H3	$0.40 \times 16 = 6.4$	9.6	Yes

Answers to the classroom questions:

1. **Which host is overloaded?** H1 (90% used, only 1.6 cores free).
2. **Where should the VM be placed?** On H2, which has the most free capacity (H3 would also work, but H2 leaves the system better balanced).
3. **Should a VM be migrated? Which one?** Yes. Live-migrate one VM from H1 to H3 (or H2). A VM using about 4–5 cores is a good choice, since it relieves H1 without overloading the destination.
4. **What is the objective of the migration?** Load balancing — to remove the overload on H1 and keep performance stable.
5. **What should be monitored?** CPU, RAM, network usage and storage on every host, continuously.

Check: After placing the new VM on H2 ($4.8 + 4 = 8.8$ cores, 55%) and moving a 4-core VM from H1 to H3 (H1 = 10.4 cores, 65%; H3 = 10.4 cores, 65%), all three hosts are at a healthy level. Remember: check *capacity and current usage* together before deciding.

1.35.2 Check Your Understanding

Q1. Why don't we simply add a new physical server every time demand increases?
It is expensive, slow, and leaves servers underused when demand later falls.

Q2. What is the major advantage of VM migration?
A workload can move between physical hosts without being manually reinstalled.

Q3. Why is load balancing required?
To stop some hosts from being overloaded while others sit underutilized.

1.35.3 Exam Preparation

Marks	Likely questions
2 marks	Define a virtual cluster. What is dynamic provisioning? What is resource management? What is server consolidation? What is a template VM? What is load balancing? Define live VM migration.
5 marks	Explain physical and virtual clusters. Explain virtual cluster architecture. Explain live VM migration. Explain memory and file migration. Explain resource management techniques.
10 marks	Explain virtual clusters and resource management in cloud computing with a suitable architecture. Discuss dynamic provisioning, VM migration, load balancing and server consolidation.

Model short answers (2 marks):

- **Virtual cluster:** a collection of VMs spread over one or more physical servers and connected through a virtual network.
- **Dynamic provisioning:** automatically creating, configuring and connecting VMs on demand, so the cluster can grow or shrink.
- **Resource management:** allocating, monitoring and optimizing CPU, memory, storage and network according to application needs.
- **Server consolidation:** running many workloads on fewer physical servers so unused servers can be switched off.
- **Template VM:** one prepared VM with common software, from which new VMs are cloned.
- **Load balancing:** spreading workload evenly so no host becomes a bottleneck.
- **Live VM migration:** moving a running VM from one host to another with very little service interruption.

Tip for the 10-mark answer: Follow this order — definition → architecture diagram → dynamic provisioning → resource management functions → load balancing → server consolidation → live migration (memory, file, network) → advantages and challenges → a short real-life exampl

Chapter 2

Cloud Computing Architecture and Management

2.1 Let's Start With a Simple Question

Think about this for a second:

1. You open Gmail on your phone and tap on a new email. It opens in less than a second.
2. But the email is not stored in your phone. So what exactly happened in that one second?

> **The short answer:** Your phone (the **front end**) sent a request over the **Internet** to Google's huge computers (the **back end**). The back end found your email, prepared it, and sent it back. The way all these parts are arranged and connected is called **Cloud Computing Architecture**.

In Chapter 1 we learnt *what* cloud computing is. In this chapter we open the box and see *how* it is built and *how* it is managed:

- **Architecture** — the parts of a cloud system and how they connect.
- **Service models** — IaaS, PaaS and SaaS, and who manages what.
- **Service management** — how providers deliver, measure and promise good service.
- **Data management** — how data is stored, copied, protected and recovered.
- **Resource management** — how CPU, memory and storage are shared fairly.
- **Cloud implementation** — how an organisation actually moves to the cloud.

2.2 What Is Cloud Computing Architecture?

Core Concept

Cloud Computing Architecture is the overall structure of a cloud system. It is made of two main parts — a **front end** (the user's side) and a **back end** (the provider's side) — connected through the **Internet**.

Even simpler way to think about it: Think of a restaurant. The dining area, the menu card and the waiter are the *front end* — what the customer sees. The kitchen, the store room and the chefs are the *back end* — hidden from the customer, but doing all the real work. The waiter carrying orders between them is like the *Internet*.

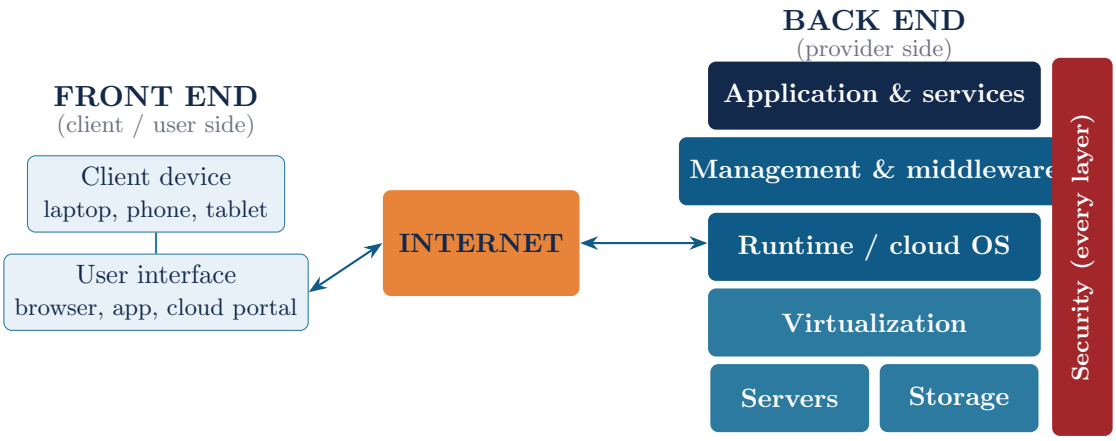


Figure 2.1: Cloud computing architecture: the front end (user side) and the back end (provider side) connected through the Internet.

2.2.1 The Front End

The **front end** is the part the user sees and touches. It has two parts:

- **Client device** – any device used to reach the cloud: a laptop, phone, tablet or office computer.
- **User interface (UI)** – the screens, menus, buttons and dashboards the user works with. This is usually a **web browser** (Chrome, Edge), a **mobile app**, or a **cloud portal** — a web dashboard where users manage their cloud services (for example, the AWS Management Console).

2.2.2 The Back End

The **back end** belongs to the cloud provider. It lives inside data centers and does all the heavy work.

Back-end part	What it does
Application	The actual software or service the user asked for (Gmail, Google Docs)
Service	The type of service offered — IaaS, PaaS or SaaS
Runtime / cloud OS	The environment that runs the applications
Management & middleware	Allocates resources, connects services, monitors everything
Virtualization	Splits one physical machine into many virtual machines
Servers (cloud servers)	Physical or virtual machines that do the computing
Storage	Keeps files and databases safely
Security	Protects every layer: login checks, encryption, firewalls

2.2.3 The Network (Internet)

The **Internet** is the bridge between the front end and the back end. Without it, the user cannot reach the cloud. That is why a good, stable internet connection is so important for cloud users.

2.3 How a Request Travels: Opening an Email

Let us follow one request, step by step, when you open an email in Gmail.

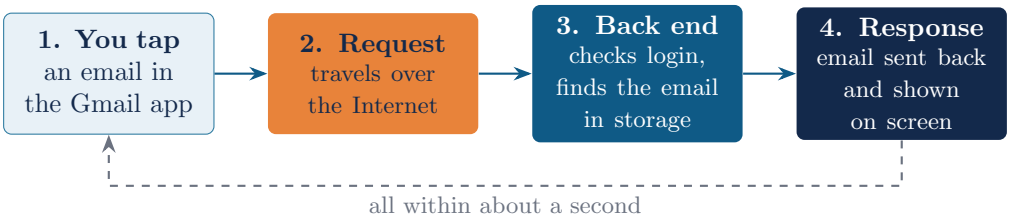


Figure 2.2: How a user's request travels from the front end to the back end and back again.

1. **Front end:** you tap the email. The app creates a request.
2. **Internet:** the request travels to Google's data center.
3. **Back end:** the servers check who you are (security), find your email in storage, and prepare it.
4. **Response:** the email travels back and the app displays it.

Core Concept

The user only ever sees the front end. The provider manages the whole back end — servers, storage, software and security — behind the scenes.

2.4 The Layered View of Cloud Architecture

Another useful way to look at cloud architecture is as a stack of **layers**, one on top of another. Each layer uses the layer below it.

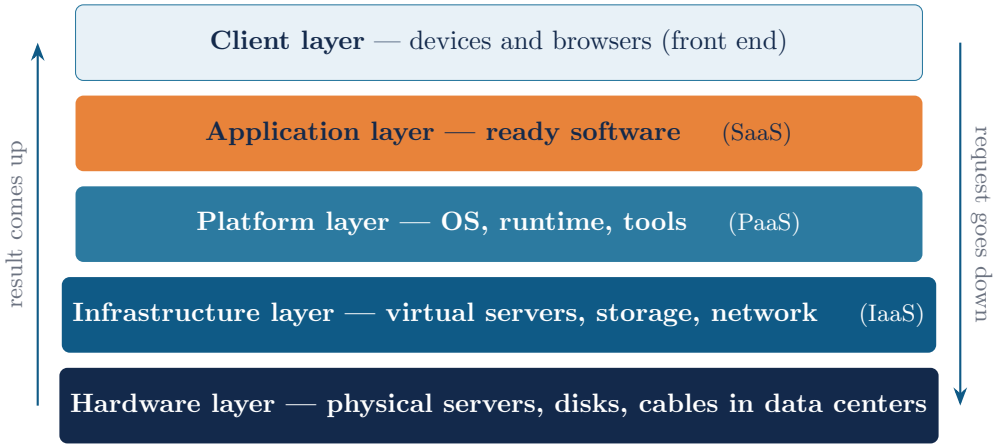


Figure 2.3: The layers of cloud architecture: client, application, platform, infrastructure and hardware.

Layer	What it contains	Gmail example
Client	User's device and browser or app	Your phone and the Gmail app
Application	User-facing software	The Gmail inbox and screens
Platform	Operating system, runtime, libraries	The mail-processing platform
Infrastructure	Virtual servers, virtual storage, networks	The virtual machines running Gmail
Hardware	Physical machines in data centers	Google's real servers and disks

Storage layer: Data storage runs through the lower layers. It is the part of the architecture that keeps and manages data — files, emails, photos and databases — usually spread over many disks in the data center.

2.5 Middleware and APIs — The “Glue” of the Cloud

2.5.1 Middleware

Core Concept

Middleware is software that sits **between** applications and the infrastructure. It helps different programs and services **talk to each other**, share data and work as one system.

What middleware does:

- Passes messages between different applications.
- Converts data from one format to another.
- Handles logins, security checks and transactions.
- Connects applications to databases.

Real-life comparison: Middleware is like a translator at an international meeting. Two people who speak different languages can still work together because the translator sits in the middle.

2.5.2 APIs (Application Programming Interfaces)

Core Concept

An **API** is a set of rules that lets one piece of software ask another piece of software for data or a service — like a menu that lists what you can order and how to order it.



Figure 2.4: An API lets one application use another application’s service.

How APIs help cloud architecture:

- They connect the front end to the back end (your app talks to the server through an API).
- They connect cloud services to each other (a shopping app uses a payment API and a map API).
- In PaaS, developers use APIs to reach databases, storage and messaging without building them.
- They let cloud services be automated — for example, creating a new server with a single API call.

2.6 Who Is Who in the Cloud? — Cloud Roles

The US National Institute of Standards and Technology (NIST) describes five main roles in cloud computing.

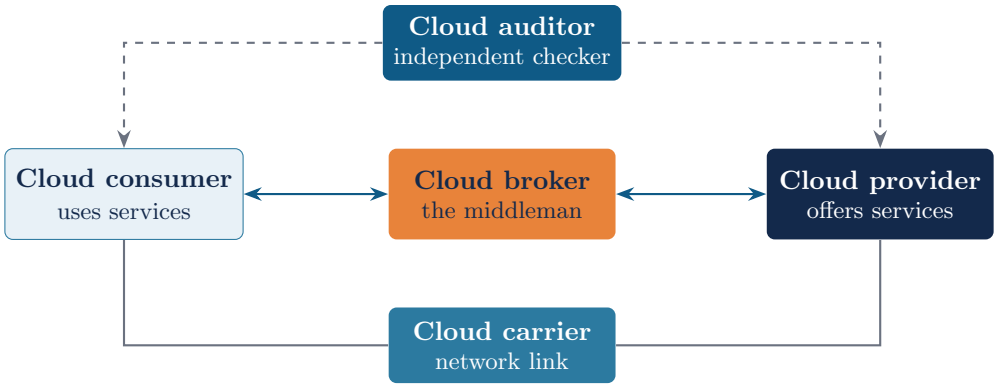


Figure 2.5: The five cloud roles: consumer, provider, broker, carrier and auditor.

Role	Meaning	Example
Cloud consumer	A person or organisation that uses cloud services	A college using Google Workspace
Cloud provider	A company that offers cloud services	AWS, Microsoft Azure, Google Cloud
Cloud broker	A middleman that helps consumers choose, connect to and manage services from one or more providers	A consulting firm that compares providers
Cloud carrier	The network company that connects consumers to providers	Jio, Airtel, BSNL
Cloud auditor	An independent party that checks security, performance and rules	A certified security auditing firm

2.6.1 What Does a Cloud Broker Do?

A cloud broker is like a **travel agent**. You could book flights and hotels yourself, but the agent compares many options, finds the best deal, and handles the paperwork. Similarly, a cloud broker:

1. Understands the customer's needs (cost, speed, security, location).
2. Compares several providers on price, features and reliability.
3. Negotiates contracts and SLAs.
4. Helps connect and combine services from more than one provider.
5. Manages the services and billing on the customer's behalf.

2.6.2 Responsibilities of Providers and Users

Cloud provider is responsible for	Cloud user is responsible for
Maintaining hardware and data centers	Managing their own data and applications
Keeping the service running (uptime)	Using resources within the agreed terms
Physical and infrastructure security	Protecting passwords and user accounts
Meeting the promises in the SLA	Choosing the right service and size
Applying updates and security patches (to the parts it manages)	Paying bills and controlling their own spending

The exact split changes with the service model — as we will see next.

2.7 Cloud Service Models: IaaS, PaaS and SaaS

What exactly are you renting from the cloud?

2.7.1 What Is a Cloud Service Model?

Core Concept

A **cloud service model** is a way of grouping cloud services by **what the provider delivers** and **how much the user has to manage**. The three basic models are **IaaS**, **PaaS** and **SaaS**.

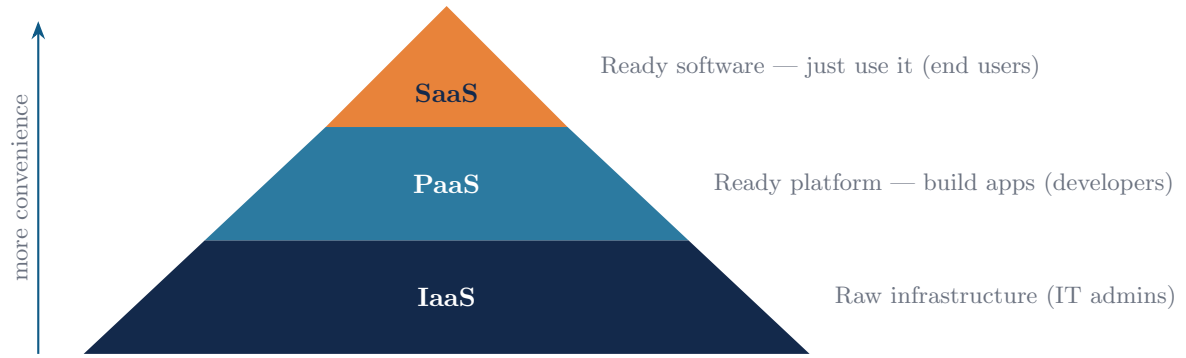


Figure 2.6: The cloud service pyramid: IaaS at the bottom, PaaS in the middle, SaaS at the top.

Each model is built on top of the one below it. As we move **up** the pyramid, the provider does more work and the user does less.

2.7.2 Infrastructure as a Service (IaaS)

Core Concept

Infrastructure as a Service (IaaS) provides **virtual computing resources** — servers, storage and networks — over the Internet. The provider manages the hardware and virtualization; the **user manages the operating system and everything above it**.

What you get:

- **Virtual servers** – software-based machines rented from the provider instead of buying physical computers. You choose the CPU, memory and operating system (for example, **Linux** such as Ubuntu, or **Windows Server**).
- **Storage** – virtual disks and storage space.
- **Networking** – virtual networks, IP addresses, firewalls.

What the user manages: the operating system, applications, runtime and data.

Examples: Amazon EC2, Microsoft Azure Virtual Machines, Google Compute Engine.

Advantages

- Full control over the operating system and software.
- No need to buy physical hardware.
- Scale up or down quickly; pay only for what you use.
- Good for custom software, research and testing.

Disadvantages

- User must install, update and secure the operating system.
- Needs skilled IT staff.
- Costs can rise if servers are left running unused.

Simple example: A university research team needs powerful machines to run its own custom simulation software for one month. Instead of buying servers, it rents 10 virtual servers on IaaS, installs its own Linux setup and software, and switches them off when the project ends.

2.7.3 Platform as a Service (PaaS)

Core Concept

Platform as a Service (PaaS) provides a **ready platform** — operating system, runtime environment, databases and development tools — so developers can **build, test and deploy** applications without managing servers.

What you get:

- **Runtime environment** – the operating system, language support (Python, Java, Node.js) and libraries needed to run your code.
- **Development tools** – editors, SDKs (software development kits), frameworks and testing tools.

- **Databases and APIs** – ready services the app can call.
- **Application deployment** – one command or click publishes your finished app so users can reach it.

What the user manages: only the application code and its data.

Examples: Google App Engine, Heroku, Microsoft Azure App Service, AWS Elastic Beanstalk.

Advantages

- Developers focus only on writing code.
- Faster development and launch (ready tools and frameworks).
- The whole team shares the same tools and libraries.
- Automatic scaling, patching and backups by the provider.

Disadvantages

- Less control over the operating system and settings.
- Limited to the languages and tools the platform supports.
- Moving to another provider can be hard (vendor lock-in).

Simple example: A start-up wants to launch a food-ordering app in two months. On PaaS it simply writes the code, connects a managed database, and deploys — no servers to set up.

2.7.4 Software as a Service (SaaS)

Core Concept

Software as a Service (SaaS) delivers **ready-to-use software** over the Internet. It is usually opened in a **web browser**, with **no installation**. The provider manages **everything**; the user simply uses the software.

Key points:

- **Web browser:** the tool used to open SaaS apps — nothing needs to be installed on the computer.
- **Software licensing:** instead of buying the software once, users usually pay a **subscription** (per user, per month or per year).
- **Automatic updates:** everyone always uses the latest version.

Examples: Gmail, Google Docs, Google Classroom, Microsoft 365, Zoom, Dropbox, Salesforce (CRM), Netflix.

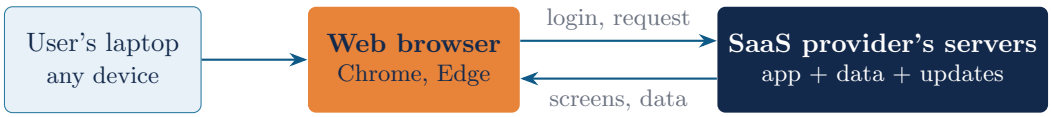


Figure 2.7: A user opening a SaaS application through a web browser.

Advantages

- Nothing to install or maintain; works from any device with internet.
- Very easy to start — just sign up.
- Low upfront cost; no IT staff needed.
- Easy collaboration and automatic updates.

Disadvantages

- Least control — you cannot change how the software works.
- Data is stored with the provider (privacy concerns).
- Needs internet; subscription costs continue every month.

Simple example: A college wants a learning platform. Instead of building one, it uses **Google Classroom** — teachers and students simply log in from a browser.

2.7.5 Who Manages What?

The biggest difference between the models is **how much you manage** and **how much the provider manages**.

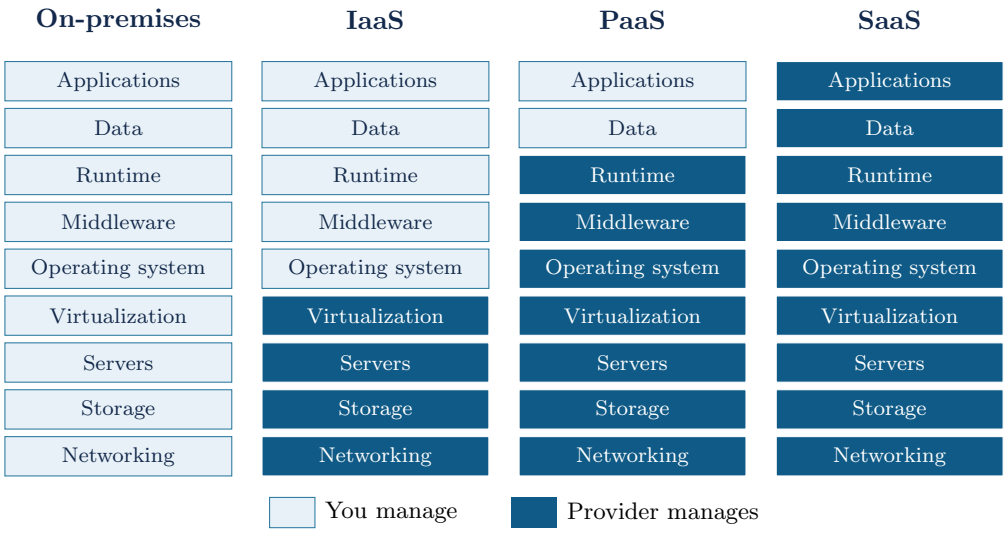


Figure 2.8: Management responsibilities in on-premises, IaaS, PaaS and SaaS.

Understanding the figure: In **on-premises** (your own computers), you manage everything. In **IaaS**, the provider manages networking, storage, servers and virtualization; you manage the operating system and above. In **PaaS**, the provider also manages the operating system, middleware and runtime; you manage only your application and data. In **SaaS**, the provider manages almost everything; you simply use the application.

Item	IaaS	PaaS	SaaS
Hardware	Provider	Provider	Provider
Virtualization	Provider	Provider	Provider

Operating system	**User**	Provider	Provider
Runtime & tools	**User**	Provider	Provider
Application	**User**	**User**	Provider
Data	**User**	**User**	**User** (settings and own data)

2.7.6 An Easy Way to Remember: Getting Around Town



Figure 2.9: The transport analogy: the more you move to the right, the less you manage.

Core Concept

- **IaaS** → rent **infrastructure** (most control)
- **PaaS** → rent a **platform** to build apps (moderate control)
- **SaaS** → use ready **software** (least control)

Control decreases from IaaS to SaaS, while **convenience increases**.

2.7.7 Head-to-Head Comparison

Feature	IaaS	PaaS	SaaS
Main purpose	Provides infrastructure	Provides a development platform	Provides complete software
User control	Most	Moderate	Least
User manages	OS, apps, data	Code and data	Only use and settings
Main users	IT administrators	Developers	End users
Flexibility	Highest	Medium	Lowest
Maintenance effort	High	Medium	Very low
Setup speed	Slowest	Faster	Instant
How you pay	For raw compute and storage used	For platform resources used	Usually per user per month
Examples	Amazon EC2, Azure VMs, Google Compute Engine	Google App Engine, Heroku, Azure App Service	Gmail, Microsoft 365, Salesforce, Zoom

One company, all three models: Amazon Web Services offers IaaS (Amazon EC2), PaaS (AWS Elastic Beanstalk) and SaaS (for example, Amazon WorkMail). Microsoft and Google also offer all three.

2.7.8 More "as a Service" Offerings

The idea of renting *anything* as a service has grown. These are often grouped under the name **XaaS** (anything as a service).

Service	Meaning	Example
Storage as a Service	Storage space rented and managed by the provider, billed by usage	Amazon S3, Google Drive
Database as a Service (DBaaS)	A managed database; the provider handles setup, backups, patching and scaling	Amazon RDS, Azure SQL Database
Network as a Service (NaaS)	Network connectivity and bandwidth delivered as a managed service	Connecting branch offices over a provider's network
Security as a Service (SECaaS)	Antivirus, firewall and identity tools delivered from the cloud	Cloud firewalls, identity services



Figure 2.10: Storage as a service: many devices store and read files through one managed storage service.

2.7.9 Choosing the Right Service Model

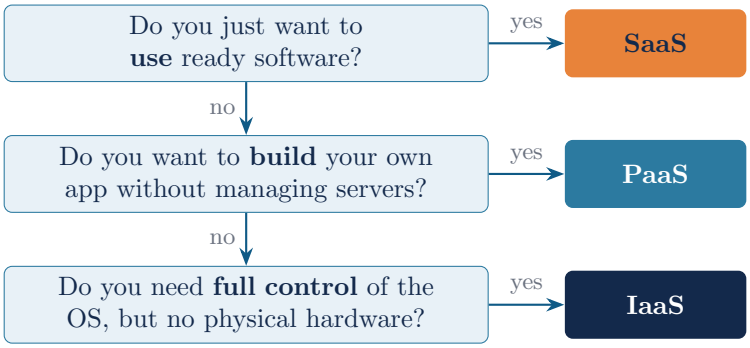


Figure 2.11: A simple way to choose between SaaS, PaaS and IaaS.

2.7.9.1 Checklist for Choosing a Model

1. Do we have skilled IT staff to manage operating systems? (Yes → IaaS is possible.)
2. Do we need to write our own application? (Yes → PaaS or IaaS.)
3. Is there already ready-made software that does the job? (Yes → SaaS.)

4. How much control and customisation do we need?
5. How quickly must we start?
6. What is our budget, and how will we pay (per use or per user)?
7. Are there security or legal rules about where data is kept?

2.7.9.2 Which Model Fits? — Common Situations

Situation	Model	Why
Company wants to rent servers but install its own OS and software	IaaS	Full control above the hardware
Start-up wants full OS control but no hardware purchase	IaaS	Control without buying machines
Bank keeps control of its database software but rents the VMs	IaaS	Bank manages software; provider manages hardware
Retailer needs extra computing only during festival sales	IaaS	Rent more servers for a few days, then release them
Developer wants to focus only on writing code	PaaS	Runtime and tools are ready
School wants to host its own custom student portal	PaaS	Build a custom app without managing servers
Start-up building a mobile-app backend with a database	PaaS	Managed database and hosting
Team needs a shared development environment	PaaS	Same tools and libraries for everyone
Small business wants email and office tools online	SaaS	Google Workspace or Microsoft 365, nothing to install
College wants a ready learning-management system	SaaS	Use existing software instead of building
Hospital wants ready patient-management software	SaaS	No development or server maintenance
Sales team needs CRM from anywhere	SaaS	Browser-based CRM such as Salesforce
Restaurant chain needs one order system for all branches	SaaS	One online app for every branch

2.7.10 Using All Three Together: An Online Business

A real online business often uses all three models at once.

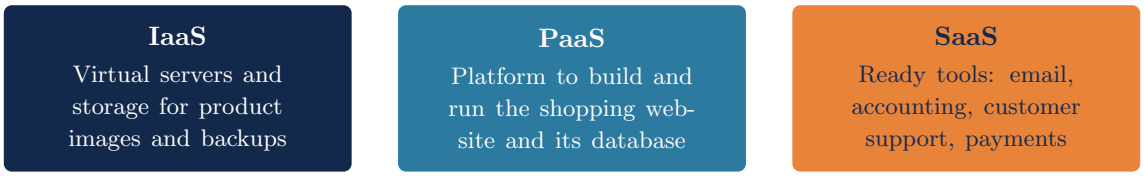


Figure 2.12: An online shop combining IaaS, PaaS and SaaS.

2.7.11 Traditional Software vs SaaS

Feature	Traditional (installed) software	SaaS
Installation	On every computer	None — open in a browser
Updates	Done on each computer	Automatic, by the provider
Payment	One-time licence, often costly	Monthly or yearly subscription
Access	Only on the computer where installed	Any device with internet
IT effort	High	Very low
Example	Classroom software installed in each lab PC	Google Classroom

2.8 Service Management in Cloud Computing

How providers deliver, measure and promise good service

2.8.1 What Is Service Management?

Core Concept

Service Management is the process of **planning, delivering, monitoring and supporting** cloud services so that they always meet the **agreed service levels**.

Real-life comparison: Think of a hotel. It does not just give you a room. It shows you a list of room types (catalog), gets your room ready (provisioning), checks that the AC and lights work (monitoring), keeps a bill of what you used (metering), helps you at the front desk (support), and promises certain standards (agreement). Cloud service management does the same for computing.

2.8.2 The Service Management Cycle

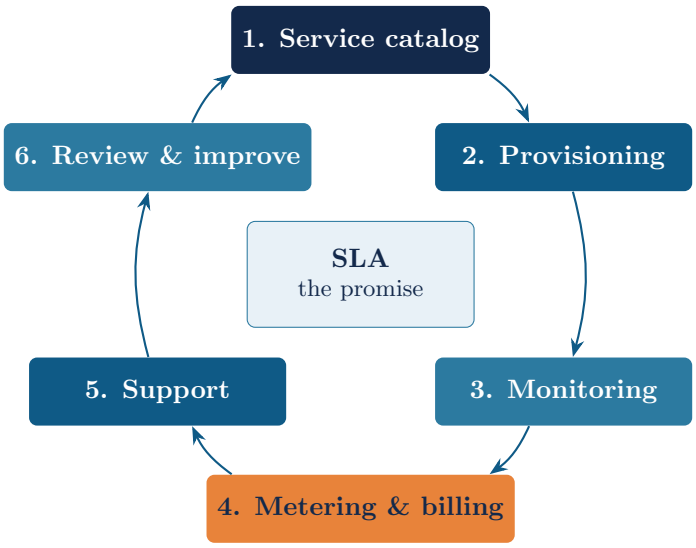


Figure 2.13: The cloud service management cycle, built around the Service Level Agreement (SLA).

2.8.2.1 Service Catalog

A **service catalog** is a list of all the cloud services a provider offers — like a restaurant menu. It shows each service, its features, sizes and prices, so customers can compare and choose the right one quickly.

Example: The AWS website lists hundreds of services (virtual servers, storage, databases), each with sizes and prices.

2.8.2.2 Service Provisioning

Service provisioning is the process of **setting up** a cloud resource and making it ready for the user.

On-demand provisioning means the user requests a resource through the portal or an API and gets it **automatically, within minutes**, without calling or meeting anyone at the provider.

Example: A company's website suddenly gets heavy traffic. Its administrator clicks "launch 5 more servers" (or auto-scaling does it), and the new virtual servers are running in a few minutes — no hardware purchase, no waiting for delivery.

2.8.2.3 Service Monitoring

Service monitoring means **continuously checking** a service's performance, availability and usage — CPU load, memory, response time, errors and uptime.

Example: A company's app becomes slow every evening. Monitoring shows that CPU use reaches 95% at 8 pm. The team adds more servers for the peak hours, and the problem disappears.

2.8.2.4 Service Metering and Billing

Service metering means **measuring how much** of each resource a customer uses — like the electricity meter in your house. The meter readings are used for **pay-per-use billing**: customers pay only for what they actually consume.

Worked example (illustrative rates): A company uses in one month:

Resource	Usage × rate	Cost
Virtual server	240 hours × Rs. 5 / hour	Rs. 1,200
Storage	100 GB × Rs. 2 / GB	Rs. 200
Data sent out	50 GB × Rs. 4 / GB	Rs. 200
Total bill		**Rs. 1,600**

If the server had been switched off for half of those hours, the server cost would drop to Rs. 600. This is why monitoring and metering together help both the provider (accurate bills) and the customer (no paying for idle resources).

2.8.2.5 Support, Review and Improvement

Providers offer help desks, documentation and support tickets. They also review reports regularly, fix repeated problems, and improve the service.

2.8.3 Service Level Agreement (SLA)

Core Concept

A **Service Level Agreement (SLA)** is a **contract** between the cloud provider and the customer. It clearly states the expected service quality — such as uptime, support time and security — and what happens if the provider fails to meet it.

2.8.3.1 What Does "99.9% Uptime" Really Mean?

Service uptime is the percentage of time a service is available and working. **Service downtime** is the time it is not working. Let us calculate for a 30-day month:

$$30 \text{ days} \times 24 \text{ hours} \times 60 \text{ minutes} = 43{,}200 \text{ minutes}$$

Uptime promised	Downtime allowed per month	Downtime allowed per year
99%	432 minutes (≈ 7.2 hours)	≈ 3.65 days
99.9%	43.2 minutes	≈ 8.8 hours
99.99%	4.32 minutes	≈ 53 minutes

Each extra "9" cuts the allowed downtime by ten times. A business that depends on the cloud every minute — a bank, a hospital, an online shop — needs a higher uptime promise.

2.8.3.2 Features of a Good SLA

Feature	Meaning
Uptime guarantee	The availability promised, e.g. 99.9% per month
Performance targets	Response time, speed and capacity promised
Support response time	How fast the provider will answer problems (e.g. within 1 hour for critical issues)

Penalties / service credits	What the provider pays back if a promise is broken
Security and data terms	Who protects what; backup, encryption and data location
Data ownership	The customer owns its data and can take it out
Exit / termination clause	How to leave the provider and get the data back

2.8.3.3 Checklist Before Signing an SLA

1. Is the uptime guarantee high enough for our business?
2. How fast will support respond, and at what hours?
3. What penalties or credits apply if the provider fails?
4. Where will our data be stored (which country), and is it legal for us?
5. Which security certifications does the provider hold?
6. How are backups and disaster recovery handled?
7. How do we get our data back if we leave?
8. What is the total cost, and are there hidden charges?

How an SLA protects a business: If a company's online shop depends on a cloud service, an SLA turns vague promises into *measurable* ones. If uptime falls below the promised level, the company can refer to the SLA, claim service credits or compensation, and — if failures continue — use the exit clause to switch providers.

2.8.4 Quality of Service (QoS)

Core Concept

Quality of Service (QoS) is a measure of **how well** a cloud service performs — how fast, how reliable and how available it is.

QoS measure	Meaning	Video-call example
Availability (uptime)	Is the service working?	Can you join the meeting at all?
Latency	Delay before data arrives	Voice reaches others late; people talk over each other
Response time	Time to answer a request	How long "Join meeting" takes
Throughput / bandwidth	How much data per second	Video quality: HD or blurry
Reliability	Does it keep working without failure?	Do calls drop in the middle?

How QoS is maintained: monitoring all these measures, adding resources when needed, using servers close to users, balancing load, and setting clear targets in the SLA.

2.8.5 Scalability and Elasticity

Feature	Scalability	Elasticity
Meaning	The ability to handle more work by adding resources	The ability to add *and remove* resources **automatically** as demand changes
Direction	Mainly growing	Growing and shrinking
Speed	Planned, can be slower	Automatic, in real time
Example	A shop adds servers as its customer base grows over a year	An online store grows from 3 to 20 servers for a sale, then back to 3

2.8.6 Why Is Service Management So Important?

- It makes sure the provider **keeps its SLA promises**.
- It finds problems **before customers notice** them.
- It gives **accurate bills** based on real usage.
- It helps plan **capacity** for future demand.
- Poor service management leads to **downtime, security gaps, wrong bills and unhappy customers**.

2.9 Data Management in Cloud Computing

Keeping data safe, correct and available

2.9.1 What Is Data Management?

Core Concept

Data Management in cloud computing means managing how data is **stored, organised, copied, secured, backed up and accessed** in the cloud — from the moment it is created until it is deleted.

Data storage in the cloud means keeping files and databases on the provider's servers instead of on local hard disks. The data is usually spread across many disks and locations — this is called **distributed storage**.

2.9.2 Kinds of Data

Feature	Structured data	Unstructured data
Format	Fixed format — rows and columns	No fixed format
Examples	Student marks table, bank transactions, spreadsheets	Photos, videos, emails, PDF files, social-media posts
Stored in	Databases (SQL)	File / object storage, NoSQL databases
Easy to search?	Yes, with simple queries	Harder; needs special tools or AI

A **database** is an organised collection of data. In the cloud, databases are often offered as **Database as a Service (DBaaS)** — the provider handles installation, backups, patching and scaling, so a small company does not need its own database administrator.

Big Data means data sets so large and fast-changing that ordinary tools cannot handle them — for example, millions of online orders or sensor readings every day. The cloud is ideal for Big Data because storage and processing power can be rented on demand.

A **data warehouse** is a large cloud store that collects historical data from many sources (sales, stock, customers) so that a business can analyse long-term trends — for example, Amazon Redshift or Google BigQuery.

2.9.3 The Data Lifecycle

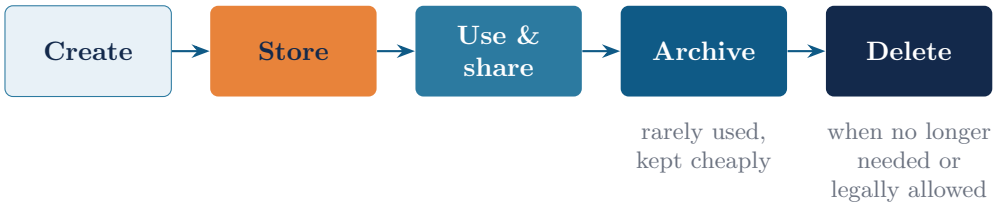


Figure 2.14: The data lifecycle: from creation to archiving or deletion.

Data lifecycle management decides what happens to data at each stage — how long it is kept (retention period), when it is archived, and when it is safely deleted according to the law.

2.9.4 Data Replication, Redundancy and Consistency

Core Concept

Data replication means keeping **copies of the same data on several servers or locations**, so that if one copy is lost, another is still available.

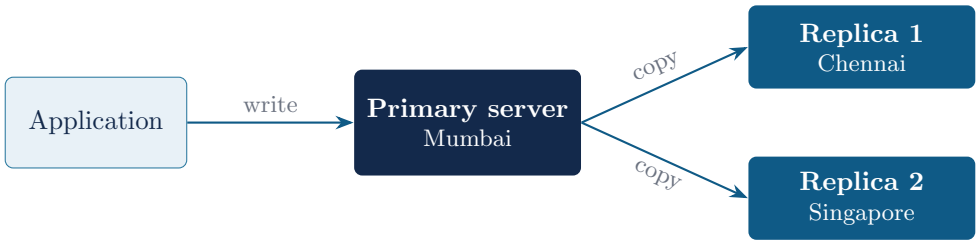


Figure 2.15: Data replication: data written to a primary server is copied automatically to replicas in other places.

- **Data redundancy** means storing more than one copy of data *on purpose*, for safety.
- **Data consistency** means all copies show the **same, up-to-date** value.

Example — the same bank balance everywhere: You withdraw Rs. 2,000 at an ATM in Kochi. Your balance is updated on the primary server, and the change is copied to every replica before (or immediately after) the transaction is confirmed. So whether your phone app connects to the Chennai or Singapore server, you see the same new balance. Banks use **strong consistency**: a change is visible everywhere before it is accepted. Less critical apps (like a "like" count on a post) may use **eventual consistency**: copies catch up within a few seconds.

2.9.5 Backup, Recovery and Archiving

- **Data backup** – making copies of data so it can be restored if the original is lost or damaged.
- **Data recovery** – restoring lost or damaged data from a backup.
- **Data archiving** – moving rarely-used data to cheaper, long-term storage instead of deleting it.

Feature	Backup	Archiving
Purpose	Recover from loss or damage	Keep old data cheaply for the long term
Data	Active, current data (copied)	Old, rarely-used data (moved)
How often	Regularly (daily, hourly)	Occasionally
Storage cost	Normal	Very low
Example	Nightly copy of customer records	Research data from 10 years ago

The 3-2-1 backup rule: keep **3** copies of important data, on **2** different types of storage, with **1** copy in a different location (for example, another cloud region).

Example — an accidentally deleted file: An employee deletes an important customer file. Because the company takes automatic backups every night, the administrator restores

yesterday's copy within minutes. Only the changes made after the last backup are lost — which is why important data is backed up often.

A backup plan for a small business:

1. Turn on automatic daily backups of customer records.
2. Store the backups in a *different* cloud region.
3. Keep backups for at least 30 days (or as the law requires).
4. Encrypt the backups.
5. Test a restore every month to make sure backups actually work.

2.9.6 High Availability and Disaster Recovery

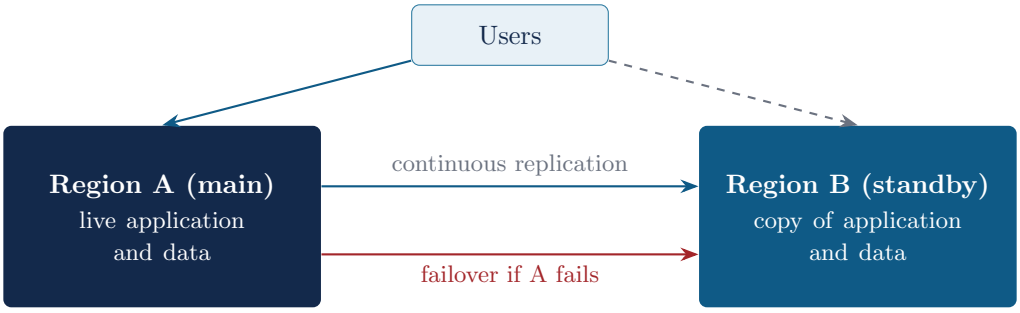


Figure 2.16: Disaster recovery: data is copied to a second region so that service can continue if the first region fails.

Feature	High availability	Disaster recovery
Goal	*Prevent* downtime	*Recover* after a major failure
How	Redundant servers, load balancing, automatic failover	Backups and replicas in another region, recovery plan
Time to recover	Seconds (often unnoticed)	Minutes to hours
Handles	A server or disk failure	Fire, flood, earthquake, region-wide outage

Two important targets in a disaster recovery plan:

- **RPO (Recovery Point Objective)** – how much data, at most, we can afford to lose (for example, the last 1 hour).
- **RTO (Recovery Time Objective)** – how quickly the service must be running again (for example, within 4 hours).

Example: A flood destroys a company's office. Because all its files were in the cloud and copied to another region, employees log in from home the next morning and continue working.

2.9.7 Data Security and Privacy

Feature	Data security	Data privacy
Question	*How* is data protected?	*Who* is allowed to see and use it?
Focus	Stopping hackers, theft and damage	Rights, permissions and consent of the data owner
Tools	Encryption, firewalls, access control	Consent forms, privacy policies, laws
Example	Encrypting stored patient records	Only the patient's doctor may read them

2.9.7.1 Data Encryption

Core Concept

Encryption converts readable data into a **secret code**. Only someone with the correct **key** can turn it back into readable form.

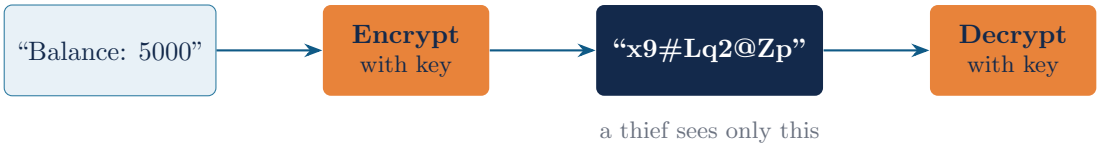


Figure 2.17: Encryption: stolen data is useless without the key.

Data is encrypted in two places: **at rest** (while stored on disks) and **in transit** (while travelling over the network, using HTTPS/TLS).

2.9.7.2 Access Control

Access control decides **who can view or change which data**. Common methods:

- **Strong login** with **multi-factor authentication (MFA)** — a password plus a code on your phone.
- **Role-based access** — a cashier can see order totals, but only the finance manager can see full payment details.
- **Least privilege** — give each person only the access they really need.
- **Access logs** — record who opened what, and when.

Access control and encryption together: Access control is the locked door; encryption is the safe inside the room. Even if someone gets past the door, they still cannot open the safe without the key.

2.9.7.3 Data Integrity

Data integrity means data stays **accurate, complete and unchanged** except by authorised actions. When many employees edit the same cloud document at the same time, tools such as **version history**, **record locking** and **validation checks** make sure one person's changes do not silently overwrite another's.

2.9.7.4 Data Breach

A **data breach** is an incident where data is accessed, stolen or exposed without permission. Ways to prevent breaches:

Method	How it helps
Encryption	Stolen data cannot be read
Multi-factor authentication	A stolen password alone is not enough
Access control	Fewer people can reach sensitive data
Regular updates and patches	Known weaknesses are fixed
Monitoring and alerts	Unusual access is spotted early
Security audits	Independent checks find gaps
Staff training	Fewer mistakes and phishing victims

2.9.8 Data Governance

Core Concept

Data governance is the set of **policies and rules** that decide how data is collected, stored, used, protected, shared and deleted — so that it stays secure, accurate and **legal**.

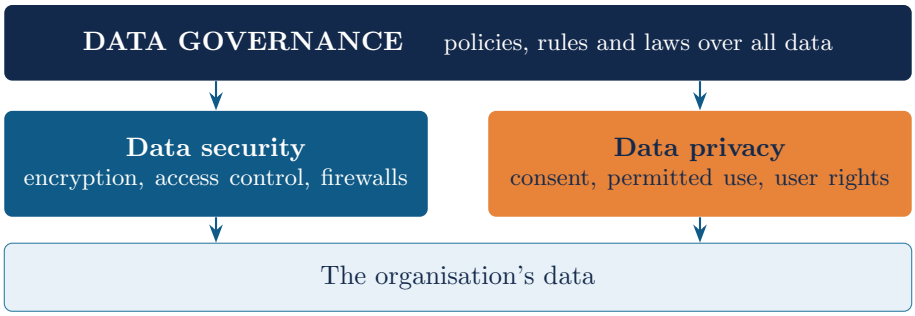


Figure 2.18: Data governance sets the rules; security and privacy carry them out to protect the data.

Why it matters: A company handling financial or customer data must follow laws — for example, India's Digital Personal Data Protection (DPDP) Act 2023. Good governance defines who may access data, how it must be protected, how long it is kept, and how it is deleted, which keeps the company compliant and trusted.

2.9.9 Who Is Responsible for the Data?

Company (cloud user)	Cloud provider
Deciding what data to store and how sensitive it is	Physical security of data centers
Setting user access and permissions	Infrastructure security and availability
Choosing encryption and backup settings	Offering encryption, backup and replication tools
Following data-protection laws	Meeting the SLA and certifications

Advantages of Storing Data in the Cloud

- Scalable — storage grows as needed.
- Cost-effective — pay only for what you use; no disks to buy.
- Accessible from anywhere.
- Automatic replication and backup options.

Disadvantages of Storing Data in the Cloud

- Depends on internet access.
- Security and privacy concerns — data is held by a third party.
- Legal questions about which country the data is stored in.
- Moving large amounts of data out can be slow and costly.

2.10 Resource Management in Cloud Computing

Sharing CPU, memory, storage and network fairly and efficiently

2.10.1 What Is Resource Management?

Core Concept

Resource Management is the process of **allocating, scheduling, monitoring and optimising** cloud resources — CPU, memory, storage and network — so that every application gets what it needs, without waste.

Real-life comparison: A traffic police officer does not build new roads. The officer directs vehicles so that no single road jams while others stay empty. Resource management does the same with computing work.

2.10.2 Physical and Virtual Resources

Feature	Physical resources	Virtual resources
What they are	Real hardware — servers, disks, network cables	Software-made versions created by virtualization
Examples	A rack server with 64 cores	A virtual machine with 4 virtual CPUs
Creating more	Buy and install hardware (days or weeks)	Create in minutes with a click or API call
Sharing	One machine, one owner	Many virtual machines share one physical machine
Flexibility	Fixed	Easy to resize, move and copy

Virtual storage is storage space created through virtualization: many physical disks are combined and then divided into virtual disks of any size for different users.

2.10.3 Resource Pooling

Core Concept

Resource pooling means the provider keeps all its physical resources in one big shared **pool** and hands out portions to many customers, as and when they need them.

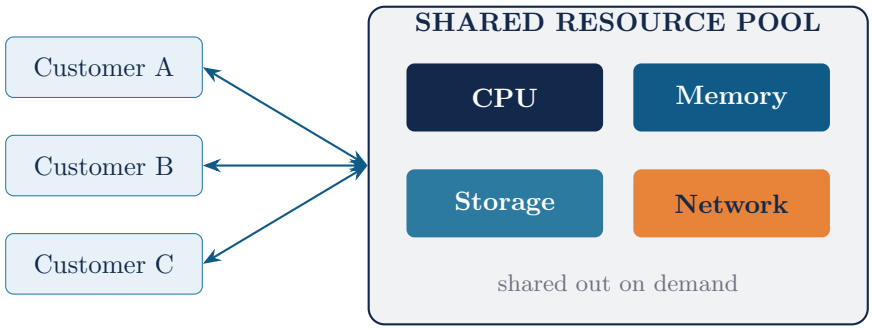


Figure 2.19: Resource pooling: many customers draw from one shared pool of resources.

Capacity is the maximum amount of resources available. If customers together ask for more than the capacity, some requests must wait or be sent elsewhere — that is why providers plan capacity carefully.

2.10.4 The Main Functions of Resource Management

Function	What it does	Everyday comparison
Resource allocation	Assigns CPU, memory, storage and network to users or tasks	Handing out desks in an office
Resource scheduling	Decides the order and timing in which tasks use resources	A doctor's appointment list
Resource monitoring	Tracks how resources are being used	A fuel gauge and speedometer
Load balancing	Spreads requests evenly across servers	Opening more billing counters
Auto-scaling	Adds or removes resources automatically with demand	Extra buses on festival days
Cost management	Tracks and controls spending	A monthly household budget

2.10.4.1 Resource Allocation

Resource allocation gives each user or task a share of the resources based on need and priority, so that no single user takes everything.

Example: A physical server has 32 CPU cores. Customer A's VM gets 8 cores, Customer B's gets 16, and 8 cores are kept free for new requests.

2.10.4.2 Resource Scheduling

Resource scheduling decides **which task runs first** when many requests arrive together. Common methods:

- **First come, first served** – tasks run in the order they arrive.
- **Priority scheduling** – urgent or premium tasks go first (a hospital emergency before a monthly report).
- **Round robin** – each task gets a small time slice in turn, so everyone keeps moving.

Worked example (round robin, time slice = 2 seconds): Three customers' tasks arrive together: Task A needs 4 s, Task B needs 2 s, Task C needs 6 s.



Figure 2.20: Round-robin scheduling: tasks take turns in 2-second slices.

Every customer gets regular turns, so no one waits a very long time — this is how providers serve many customers fairly on shared infrastructure.

2.10.4.3 Load Balancing

Core Concept

Load balancing distributes incoming requests **evenly across several servers**, so that no single server becomes overloaded while others sit idle.

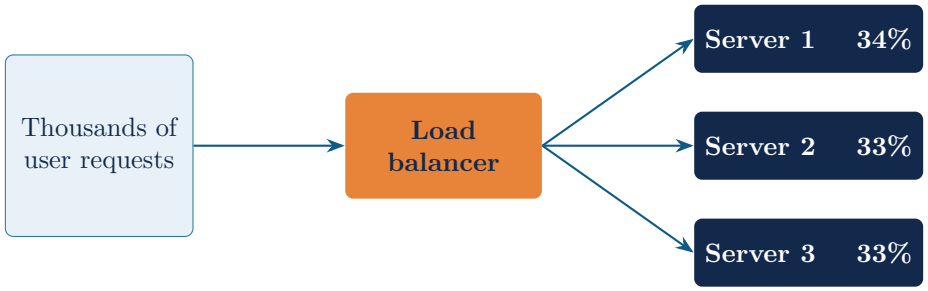


Figure 2.21: A load balancer spreads requests evenly across servers.

Example: An e-commerce site has Server 1 at 95% load while Servers 2 and 3 are at 10%. The load balancer sends new requests to Servers 2 and 3 until all three are around 40%. Pages load faster, and if one server fails, the balancer simply stops sending it traffic.

2.10.4.4 Auto-Scaling

Core Concept

Auto-scaling automatically **adds** resources when demand goes up and **removes** them when demand goes down.

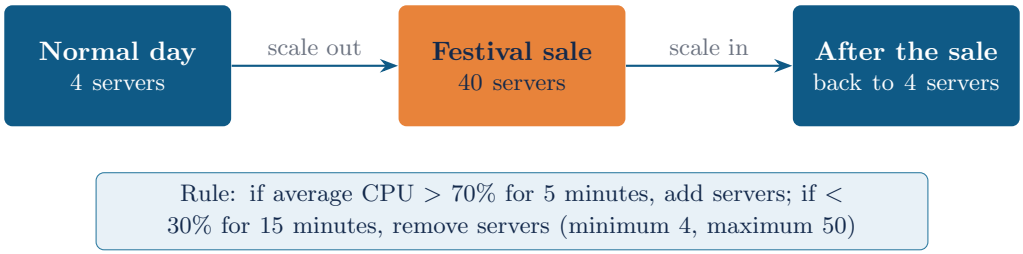


Figure 2.22: Auto-scaling follows simple rules to add and remove servers as demand changes.

Worked example — a festival sale: On a normal day the website has 2,000 visitors at a time, and each server can comfortably handle 500.

$$\frac{2{,}000}{500} = 4 \text{ servers}$$

During the sale, visitors jump ten times to 20,000:

$$\frac{20{,}000}{500} = 40 \text{ servers}$$

Auto-scaling adds 36 servers within minutes and removes them after the sale. The company pays for 40 servers only during the sale hours — not for the whole year.

Feature	Manual resource management	Auto-scaling
Who decides	Staff watch and change resources by hand	Rules change resources automatically
Speed	Slow; may react too late	Fast, in real time
Risk	Over-provisioning (waste) or under-provisioning (crash)	Capacity matches demand
Cost	Often higher (servers left running)	Lower — pay for what is needed
Effort	Needs constant attention	Set once, then monitor

2.10.4.5 Resource and Performance Monitoring

Monitoring tools track CPU, memory, storage, network, response time and errors in real time, and send **alerts** when something crosses a limit. Examples: Amazon CloudWatch, Azure Monitor, Google Cloud Monitoring.

They help to:

- spot slowdowns before customers complain,
- find idle resources that can be switched off,
- show exactly how much is being used and spent each month,
- support high availability through quick alerts and automatic recovery.

2.10.4.6 Cost Management

Cost management means tracking and controlling cloud spending. A simple plan for a start-up:

1. Set a monthly **budget** with **alerts** at 50%, 80% and 100%.
2. Use **monitoring** to find and switch off idle servers.
3. Use **auto-scaling with a maximum limit**, so costs cannot grow without control.
4. Choose the right server size ("right-sizing").
5. Review the bill every month.

2.10.5 Risks of Poor Resource Management

Risks of Poor Resource Management

- **Wasted money** — paying for servers that sit idle.
- **Slow performance** — some servers overloaded while others are empty.
- **Outages** — not enough capacity during peak demand.
- **Broken SLA promises** — leading to penalties and unhappy customers.

2.11 Cloud Implementation

How an organisation actually moves to the cloud

2.11.1 What Is Cloud Implementation?

Core Concept

Cloud Implementation is the process of **planning, setting up, moving to and running** a cloud-based solution so that an organisation can actually use it in daily work.

2.11.2 Steps of Cloud Implementation

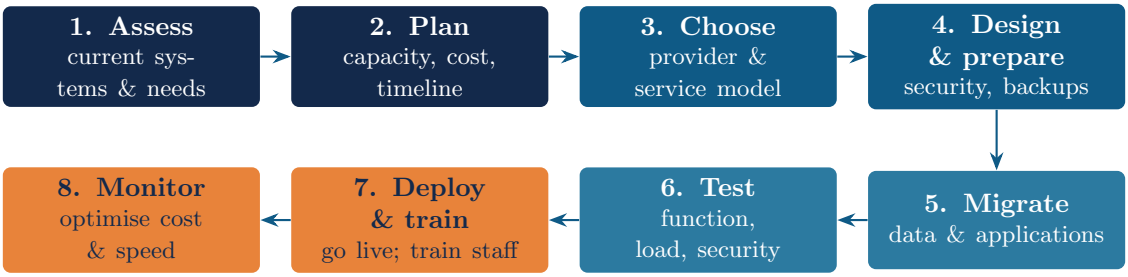


Figure 2.23: The eight steps of cloud implementation.

1. **Assess** – list current systems, data, applications and problems.
2. **Plan** – estimate resources (capacity planning), budget, risks and timeline.
3. **Choose** – select a provider and the right model (IaaS, PaaS or SaaS); sign the SLA.
4. **Design and prepare** – set up accounts, networks, security, access control and backups.
5. **Migrate** – move data and applications, usually in stages.
6. **Test** – check that everything works correctly, quickly and safely.
7. **Deploy and train** – make the system live for users and train staff.
8. **Monitor and optimise** – watch performance and cost; keep improving.

2.11.3 Capacity Planning

Core Concept

Capacity planning means estimating, in advance, how many resources will be needed to handle the expected demand — so the system never runs out at the busiest moment.

Worked example — an online exam portal: 10,000 students will log in at the same time. One server can handle 1,000 students comfortably.

$$\frac{10{,}000}{1{,}000} = 10 \text{ servers}$$

Add a 20% safety margin for surprises:

$$10 \times 1.2 = 12 \text{ servers}$$

The plan: run 12 servers behind a load balancer, keep auto-scaling switched on, and run a load test with 10,000 simulated students before the exam day.

2.11.4 Cloud Migration

Migration means moving data, applications or whole systems from an old setup (paper, local servers) to the cloud. A **cloud migration strategy** is the plan for *how* and *when* each part will be moved.



Figure 2.24: The steps of cloud migration.

Common migration approaches:

Approach	Meaning	Example
Rehost ("lift and shift")	Move the application as it is to cloud servers	Copy an old billing server to an IaaS VM
Replatform	Move with small improvements	Move the database to DBaaS
Refactor	Rebuild the application for the cloud	Rewrite the app to run on PaaS
Repurchase	Replace with ready SaaS	Replace an old email server with Gmail
Retire / Retain	Switch off what is not needed; keep some systems where they are	Keep a sensitive system on-premises

Checklist while migrating data:

1. Take a full backup of all data before starting.
2. Move data in small stages, starting with less critical data.
3. After each stage, check **data integrity** — record counts and checksums must match.
4. Test connections and access at each step.
5. Keep a **rollback** option ready.

2.11.5 Testing Before Going Live

Testing checks that a cloud application works correctly before real users see it.

Type of test	What it checks
Functional testing	Every feature works as expected (login, search, payment)
Load / performance testing	The system stays fast with thousands of simulated users
Security testing	No easy way for attackers to get in; data is encrypted

Integration testing	The app works correctly with other services and APIs
Data verification	Migrated data is complete and correct
Rollback testing	We can return to the old version if something goes wrong
User acceptance testing	Real users confirm it meets their needs

Example: Before launching its new cloud ticket-booking app, a company simulates 50,000 users booking at once. The test shows the payment page slows down at 30,000 users, so the team fixes it *before* launch day — no angry customers.

2.11.6 Deployment and Rollback

Deployment means making the application (or an update) available to real users. A **rollback** means returning to the previous working version when a new version fails.

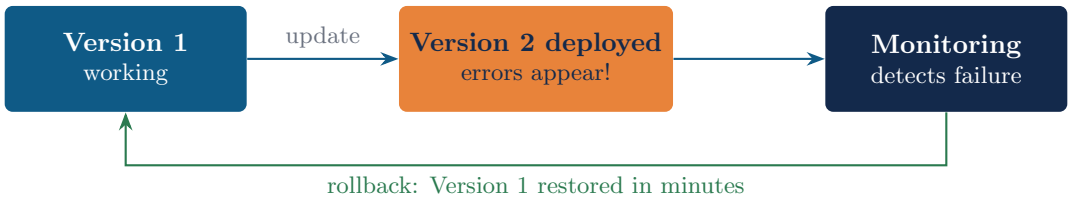


Figure 2.25: A rollback plan restores the last working version quickly after a failed deployment.

Safer ways to deploy:

- **Phased (staged) rollout** – release to a small group of users or one branch first, then to everyone.
- **Blue-green deployment** – keep the old version (blue) running while the new one (green) is tested; switch users across only when green is ready, and switch back instantly if needed.

2.11.7 Example Implementation Plans

2.11.7.1 A School Moving Its Records From Paper to the Cloud

1. List all records: admissions, marks, attendance, fee receipts.
2. Scan and digitise the paper records; keep the originals safe.
3. Choose a secure cloud storage or school-management SaaS.
4. Upload the data in stages, class by class.
5. Check that every record was uploaded correctly.
6. Set permissions: teachers see their own classes; the office sees fees.
7. Train staff, then go live; take automatic backups.

2.11.7.2 A Retail Company Moving Its Inventory System Online

1. Assess current stock systems in every store.
2. Choose a cloud database (DBaaS) or an inventory SaaS.

3. Migrate one store first as a pilot; test stock counts carefully.
4. Roll out store by store.
5. Use real-time replication so stock updates instantly everywhere.
6. Monitor and improve.

How Cloud Implementation Helps Organisations

- Less time spent buying and maintaining servers.
- Faster launch of new services.
- Easy growth to new cities and more users.
- Staff can work from anywhere.
- The organisation can focus on its real business instead of hardware.

2.12 Putting It All Together

An online shop on a festival day. Let us see every topic of this chapter working together.

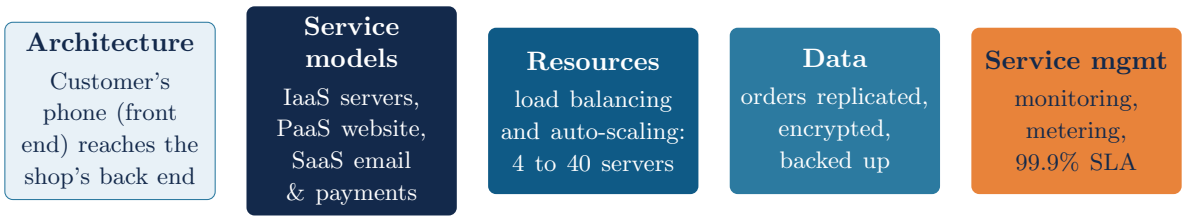


Figure 2.26: How architecture, service models, resource, data and service management work together for an online shop.

The big picture: The *architecture* connects the customer to the cloud. The *service models* decide who manages what. *Resource management* keeps the shop fast during the rush. *Data management* keeps every order safe. *Service management* measures it all and keeps the SLA promise. *Cloud implementation* is how the shop got there in the first place.

2.13 Quick Revision Summary

- **Cloud architecture** = front end (client device + user interface) + back end (application, service, runtime, management, virtualization, servers, storage, security), connected by the Internet.
- **Layers:** client → application (SaaS) → platform (PaaS) → infrastructure (IaaS) → hardware.
- **Middleware** connects applications; **APIs** let software talk to software.
- **Cloud roles:** consumer, provider, broker, carrier, auditor.
- **IaaS** = rent infrastructure (EC2); **PaaS** = rent a platform (App Engine); **SaaS** = use ready software (Gmail). Control falls from IaaS to SaaS.
- **Service management:** catalog → provisioning → monitoring → metering & billing → support → improvement, all around the **SLA**.

- **99.9% uptime** = about 43 minutes of downtime allowed per month.
- **QoS** = availability, latency, response time, throughput, reliability.
- **Data management:** storage, replication, consistency, backup, recovery, archiving, encryption, access control, integrity, governance.
- **High availability** prevents downtime; **disaster recovery** restores after a disaster.
- **Resource management:** pooling, allocation, scheduling, monitoring, load balancing, auto-scaling, cost management.
- **Cloud implementation:** assess → plan → choose → prepare → migrate → test → deploy → monitor; always keep a **rollback** plan.

2.14 Likely Exam Questions and Short Answers

Marks	Question
2 marks	What is cloud computing architecture? What are the front end and back end? What is IaaS / PaaS / SaaS? What is an SLA? What is middleware? What is a cloud broker? What is data replication? What is load balancing? What is auto-scaling? What is rollback?
6 marks	Draw the front end and back end of cloud architecture. Compare IaaS, PaaS and SaaS by who manages what. Write short notes on service management. List the features of a good SLA. Compare backup and archiving. Draw the auto-scaling diagram. Write the steps of cloud migration.
8 marks	Suggest and justify a service model for a given company. Explain how an SLA protects a business. Explain how auto-scaling handles a festival sale. Explain how replication and backup prevent data loss. Explain the steps of implementing a cloud solution for a company.

Model short answers (2 marks):

- **Cloud computing architecture:** the structure of a cloud system, made of a front end (user side) and a back end (provider side) connected through the Internet.
- **IaaS:** virtual servers, storage and networks rented over the Internet; the user manages the OS and above. Example: Amazon EC2.
- **PaaS:** a ready platform (OS, runtime, tools) to build, test and deploy applications. Example: Google App Engine.
- **SaaS:** ready software used through a browser with no installation. Example: Gmail.
- **SLA:** a contract that defines the promised service quality (uptime, support, security) and penalties if it is not met.
- **Middleware:** software that connects applications and services so they can communicate and share data.
- **Cloud broker:** a middleman that helps customers choose, connect to and manage services from cloud providers.
- **Data replication:** keeping copies of the same data on several servers so it is not lost if one fails.

- **Load balancing:** spreading requests evenly across servers so none is overloaded.
- **Auto-scaling:** automatically adding or removing resources as demand changes.
- **Rollback:** returning to the previous working version after a failed update.

Tip for the 8-mark "suggest a service model" answer

Follow this order: **state the model** (IaaS, PaaS or SaaS) → **define it** in one line → **match the company's need** (control, speed, skills, cost) → **draw the who-manages-what stack** → **give a real example** (EC2, App Engine, Gmail) → **advantages** for this company → **one-line conclusion**.

2.15 Practice Section

2.15.1 Group Problem: Choose, Size and Check

The situation: A coaching centre is launching an online test platform. It has two developers and no server administrator. During each weekly test, **6,000 students** log in at the same time; on other days, fewer than 300 students use it. One server comfortably handles **800 students**. The provider's SLA promises **99.9% uptime**.

Step 1 — Which service model?

The centre will build its *own* test platform, but has no one to manage servers or operating systems. So **PaaS** is the best fit: developers write code, and the provider handles servers, OS and runtime. For email and accounts, it can simply use **SaaS** tools.

Step 2 — How many servers during the test?

$$\frac{6{,}000}{800} = 7.5 \;\Rightarrow\; 8 \text{ servers}$$

With a 20% safety margin:

$$8 \times 1.2 = 9.6 \;\Rightarrow\; 10 \text{ servers}$$

Step 3 — How many on normal days? 300 students need just **1 server**. So the centre should use **auto-scaling** (minimum 1, maximum 12) and a **load balancer**, instead of paying for 10 servers all week.

Step 4 — What does 99.9% uptime allow?

$$43{,}200 \text{ minutes} \times 0.001 = 43.2 \text{ minutes of downtime per month}$$

If the platform goes down for 2 hours during a test, the SLA has been broken, and the centre can claim service credits.

Step 5 — Protecting the results. Store results in a managed database (DBaaS) with replication to another zone, daily backups, encryption, and role-based access (students see only their own marks).

Decision	Answer
Service model	PaaS (plus SaaS for email and office work)
Servers on test day	10 (with auto-scaling up to 12)

Servers on normal days	1
Allowed downtime	43.2 minutes per month
Data protection	Replication, daily backup, encryption, access control

Check: Always match the *skills* of the team to the service model, calculate *peak* demand with a safety margin, and use *auto-scaling* so you do not pay for peak capacity all the time.

2.15.2 Check Your Understanding

Q1. Which part of the cloud architecture does the user see?
The front end — the client device and the user interface.

Q2. In which service model does the user manage the operating system?
IaaS.

Q3. Why should a company read the SLA carefully?
Because it states the promised uptime, support and penalties — the company's protection if the service fails.

Q4. What is the difference between backup and replication?
Replication keeps live copies on other servers for instant availability; backup keeps saved copies from a point in time so data can be restored after deletion or damage.

Q5. Why is auto-scaling cheaper than keeping extra servers running?
Because extra servers are added only when demand rises and removed when it falls, so you pay only for what you use.

Q6. What should you do before moving data to the cloud?
Take a full backup, plan the stages, and keep a rollback option ready.

2.15.3 Exam Preparation

Marks	Likely questions
2 marks	Define front end, back end, cloud portal, runtime environment, service catalog, service metering, QoS, uptime, elasticity, DBaaS, NaaS, data integrity, data governance, resource pooling, capacity planning, migration, deployment.
6 marks	Draw the layers of cloud architecture. Write short notes on middleware and APIs. Make a table of IaaS, PaaS and SaaS providers. Prepare an SLA checklist. Compare high availability and disaster recovery. Compare manual resource management and auto-scaling. Prepare a testing checklist for a cloud application.
8 marks	A company wants to rent servers but install its own OS — which model? A cloud deployment fails halfway — how does rollback help? A hospital must not lose patient data if a server fails — explain replication. A business wants to control cloud costs — explain monitoring and resource management.

Tip: Learn three diagrams well — the **front-end / back-end architecture**, the **who-manages-what stack** for IaaS, PaaS and SaaS, and the **auto-scaling** diagram. Most "draw" questions in this chapter use one of them.

Chapter 3

Fog and Edge Computing

3.1 Let's Start With a Simple Question

Think about this for a second:

1. A self-driving car is moving at 60 km/h. A child suddenly runs onto the road.
2. The car's cameras see the child. Should the car send the video to a cloud server 1,500 km away, wait for the answer, and only then press the brake?

The obvious answer: No! By the time the answer comes back, the car would already have moved forward. The car must **think by itself, right there, in a few milliseconds**. Processing data right where it is created — instead of sending it far away — is called **Edge Computing**.

In Chapter 1 we learnt that cloud computing stores and processes our data in huge data centers far away. The cloud is powerful, but it is *far*. In this chapter we learn how **Fog Computing** and **Edge Computing** bring computing *closer* to us, and how they **complete** the cloud.

3.2 What Is Edge Computing?

Core Concept

Edge Computing means processing data **at or near the place where it is created** — on the device itself or on a nearby small computer — instead of sending all of it to a distant cloud data center.

Even simpler way to think about it: Instead of sending every question to a head office in another city, you ask the manager sitting in your own building. You get the answer much faster.

The word **"edge"** means the **boundary of the network** — the place where the digital world touches the real world. Your phone, a CCTV camera, a smartwatch, a sensor in a factory, a Wi-Fi router — all of these sit at the edge.

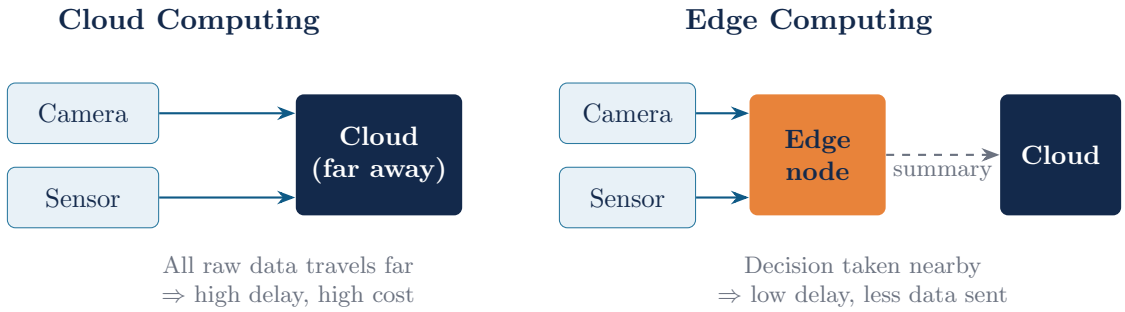


Figure 3.1: Cloud Computing sends all raw data far away; Edge Computing decides nearby and sends only a summary.

The main building blocks you will hear about again and again are:

- **Edge device** – a device at the edge that creates or uses data, such as a sensor, camera, smartwatch or phone.
- **Edge server** – a small server kept close to the devices (in a shop, factory, hospital or mobile tower) that runs applications for them.
- **Edge node** – any computing point at the edge: a device, a gateway or an edge server.
- **Gateway** – a "middleman" box that connects local devices to the Internet. It translates between different languages (protocols) and can filter data before sending it.

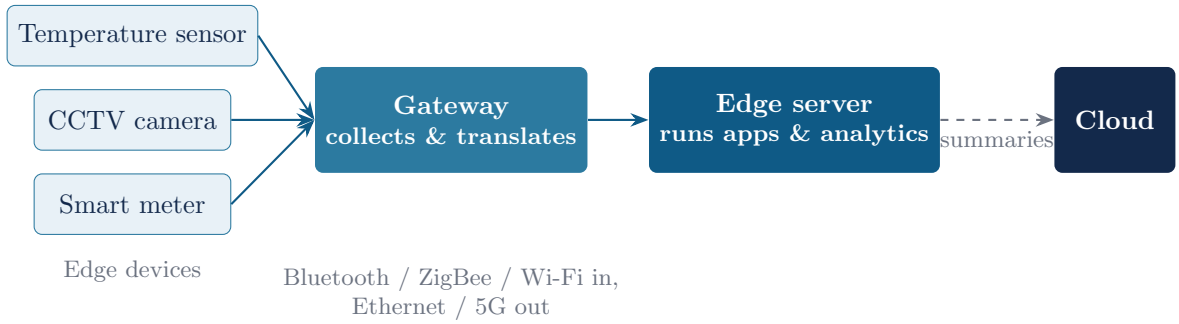


Figure 3.2: Edge devices, a gateway and an edge server working together.

3.3 Real-Life Example 1 — A Smart Doorbell Camera

The scenario: A smart doorbell camera watches your front door 24 hours a day.

Without Edge Computing:

1. The camera sends the full video, every second, to the cloud.
2. The cloud checks whether there is a person in the video.
3. The cloud sends an alert back to your phone.

This uses a lot of internet data, and the alert comes late.

With Edge Computing:

1. A small chip *inside the camera* checks the video itself.
2. Only when it sees a person does it send a short clip and an alert.
3. The cloud simply stores the clip so you can watch it later.

Core Concept

The camera has become *smart*. It sends only what matters, and it reacts immediately.

3.4 Real-Life Example 2 — A Self-Driving Car

Question: Where does a self-driving car do its thinking?

Answer: Inside the car. The car carries a powerful on-board computer — its own edge node.

Why this matters: Let us do a small calculation.

- Speed of the car = 60 km/h $\approx$ 16.7 metres per second.
- If the answer comes from a far cloud server after 100 milliseconds (0.1 s), the car travels $16.7 \times 0.1 \approx 1.7$ metres before reacting.
- If the on-board computer answers in 5 milliseconds, the car travels only about 8 centimetres.

A difference of 1.7 metres can be the difference between a safe stop and an accident. That is why life-critical decisions must be taken **at the edge**.

3.5 Why Do We Need Edge Computing?

Today, billions of devices — phones, cameras, sensors, cars, watches — are connected to the Internet. This is called the **Internet of Things (IoT)**. If every device sends everything to the cloud, four problems appear:

Problem	What it means
Latency (delay)	Data travels a long way to the data center and back. Too slow for brakes, alarms or robots.
Bandwidth	Thousands of cameras and sensors create huge amounts of data. Sending all of it is costly and jams the network.
Connectivity	Farms, ships, oil rigs, mines and moving vehicles often have weak or no internet.
Privacy	Sensitive data (health, video, bank details) must leave the building to be processed.

Edge Computing solves all four: it **decides nearby**, **sends less**, **keeps working offline**, and **keeps private data local**.

3.6 Important Words You Must Know

Most short (2-mark) questions come from these words. Learn them well.

Term	Simple meaning
Latency	The delay between sending data and getting a reply, measured in milliseconds (ms).
Response time	Total time from an event (or request) until the system reacts.
Bandwidth	The maximum amount of data a network link can carry per second (for example, 100 Mbps).
Network congestion	A "traffic jam" of data: more data is sent than the network can carry, so everything slows down.
Real-time processing	Processing data the moment it arrives, so the result is ready immediately.
Local processing	Processing data on the device or a nearby node, not in the cloud.
Data filtering	Throwing away unneeded or repeated data at the edge and sending only useful data.
Edge analytics	Analysing data at the edge to get insights or alerts immediately.
Offline capability	The ability to keep working even when the internet is down, and sync later.
Cloud offloading	Sending a heavy task from a weak device to the cloud (or a nearby edge server) to be done there.
Distributed computing	Work is shared among many computers in different places that cooperate over a network.
Scalability	The ability to grow (more devices, users, data) without slowing down.
Micro data center	A small, self-contained data center (a few racks with power, cooling and security) kept close to users.

Embedded system	A small computer built inside a device to do one job, like the chip in a washing machine.
Sensor	A device that measures something physical (temperature, motion, moisture) and turns it into data.
IoT device	An everyday object with a sensor, a small processor and a network connection.
Data privacy	Keeping personal data protected and under the owner's control.

3.6.1 Worked Example: How Much Data Can the Edge Save?

A shopping mall has **100 CCTV cameras**. Each camera sends video at **4 Mbps** (megabits per second).

$$100 \times 4 \text{ Mbps} = 400 \text{ Mbps (sent all the time)}$$

In one day there are 86,400 seconds, so:

$$400 \text{ Mb/s} \times 86{,}400 \text{ s} = 34{,}560{,}000 \text{ Mb} = 4{,}320{,}000 \text{ MB} \approx 4.32 \text{ TB per day}$$

Now suppose each camera has an edge chip that sends video **only when it sees movement**, and movement happens about **2%** of the time:

$$4.32 \text{ TB} \times 0.02 \approx 86 \text{ GB per day}$$

Result: The data sent to the cloud falls from about 4.32 TB to about 86 GB per day — roughly **50 times less**. This is what we mean when we say Edge Computing **saves bandwidth**.

3.7 What Is Fog Computing?

Core Concept

Fog Computing is a **middle layer** of computing between the edge devices and the cloud. It is made of gateways, routers and local servers that collect data from many devices, process it nearby, and send only the important part to the cloud.

Why the name "fog"? The word was introduced by **Cisco** around 2012. Fog is simply *a cloud that is close to the ground.* In the same way, fog computing is cloud-like computing that sits close to the users.

Simple example: In a smart building, 500 sensors (temperature, light, people count) are spread over 10 floors. Each floor has a small fog server. The fog server:

- collects readings from all sensors on that floor,
- switches off lights and AC in empty rooms,
- sends only an hourly summary to the cloud.

3.7.1 Edge vs Fog — What Is the Difference?

Students often ask: "Are edge and fog the same thing?" They overlap, but here is an easy way to separate them:

Edge = computing *on* the device or right next to it (one device or a small group).
Fog = computing in the *in-between* layer that serves *many* devices in an area (a building, a factory, a district).

Feature	Edge Computing	Fog Computing
Where it runs	On or next to the device	Between the devices and the cloud
Serves	One device or a small group	Many devices in an area
Typical hardware	Sensor chip, smart camera, phone	Gateway, router, local server, cloudlet
Main job	Instant reaction, filtering at source	Collecting, coordinating, local analytics
Link to the cloud	Usually through fog	Acts as the bridge to the cloud

Because the two words overlap so much, books often use one combined name: **FEC — Fog and Edge Computing**.

3.8 Cloud vs Fog vs Edge — Head-to-Head Comparison

Feature	Cloud	Fog	Edge
Location	Distant data centers	Local network (gateway, router, local server)	On or next to the device
Delay (latency)	High	Medium	Lowest
Computing power	Very high	Moderate	Limited
Storage	Huge, long-term	Medium, short-term	Small
Needs internet?	Always	Partly	Can work offline
View of data	Whole system (global)	One area or site	One device
Best for	Big-data analytics, AI training, backups	Combining data, local analytics	Instant decisions, filtering
Example	AWS data center	Factory gateway	Smart camera chip

3.9 The Three-Tier Architecture: Cloud, Fog and Edge

When cloud, fog and edge are used together, we get a **three-tier (three-layer) architecture**. Data flows *up*, getting smaller at each layer. Decisions and updates flow *down*.

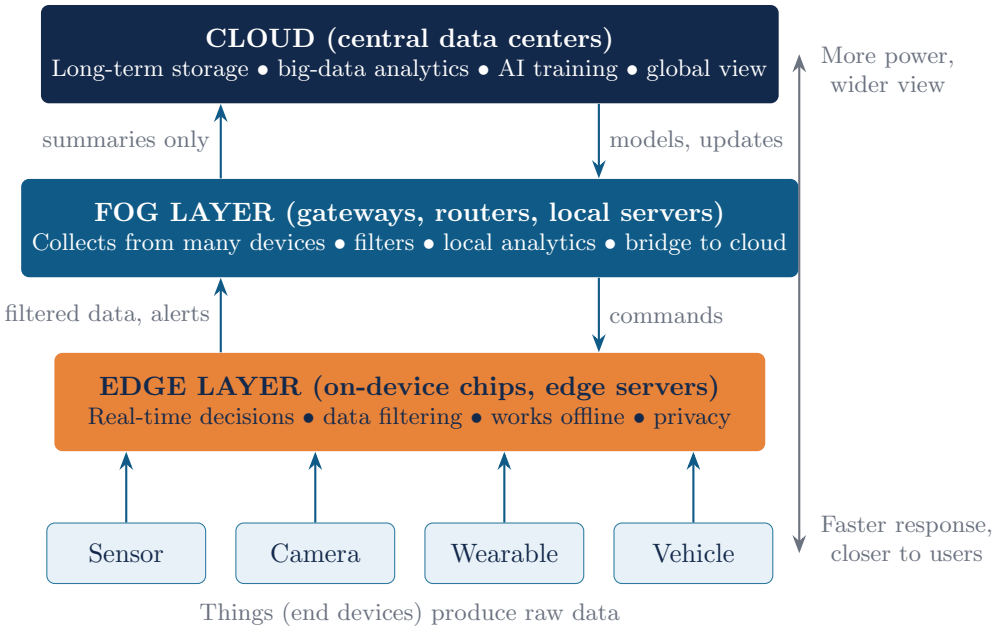


Figure 3.3: The Cloud–Fog–Edge three-tier architecture. Raw data goes up and is filtered at each layer; decisions and updates come down.

Layer	Position	Role
Edge	Closest to users and devices	Senses, acts, takes real-time decisions, filters data
Fog	In the middle	Collects from many edges, coordinates, does local analytics, stores data for a short time
Cloud	Farthest from users	Stores everything long-term, finds big patterns, trains AI, manages the whole system

3.9.1 Following the Data: A Smartwatch Example

Let us follow one heartbeat reading from a smartwatch all the way to the cloud.

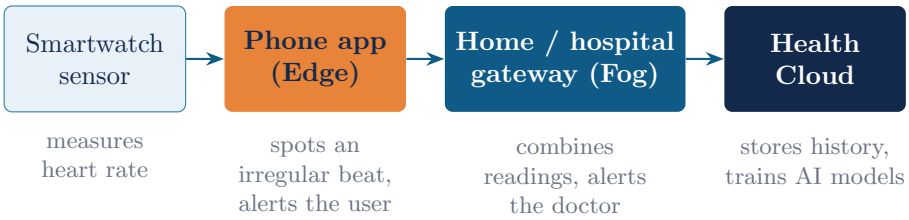


Figure 3.4: Data flow from a wearable sensor through the edge and fog layers to the cloud.

3.9.2 Many Edge Nodes, One Fog Node

In real systems, one fog node usually looks after many edge nodes — like a class teacher looking after many students.



Figure 3.5: Several edge nodes connected to one fog node, which talks to the cloud.

Real-life comparison: Students (edge) solve simple problems themselves. The class teacher (fog) handles the whole class and reports to the principal. The principal (cloud) sees the whole school and plans for the future.

3.10 Relevant Technologies

The building blocks that make Fog and Edge Computing possible

3.10.1 The Big Picture

Fog and Edge Computing did not appear suddenly. It grew out of many older and newer technologies. Each one solved a small part of the puzzle. Think of FEC as a meeting point where all of them come together.

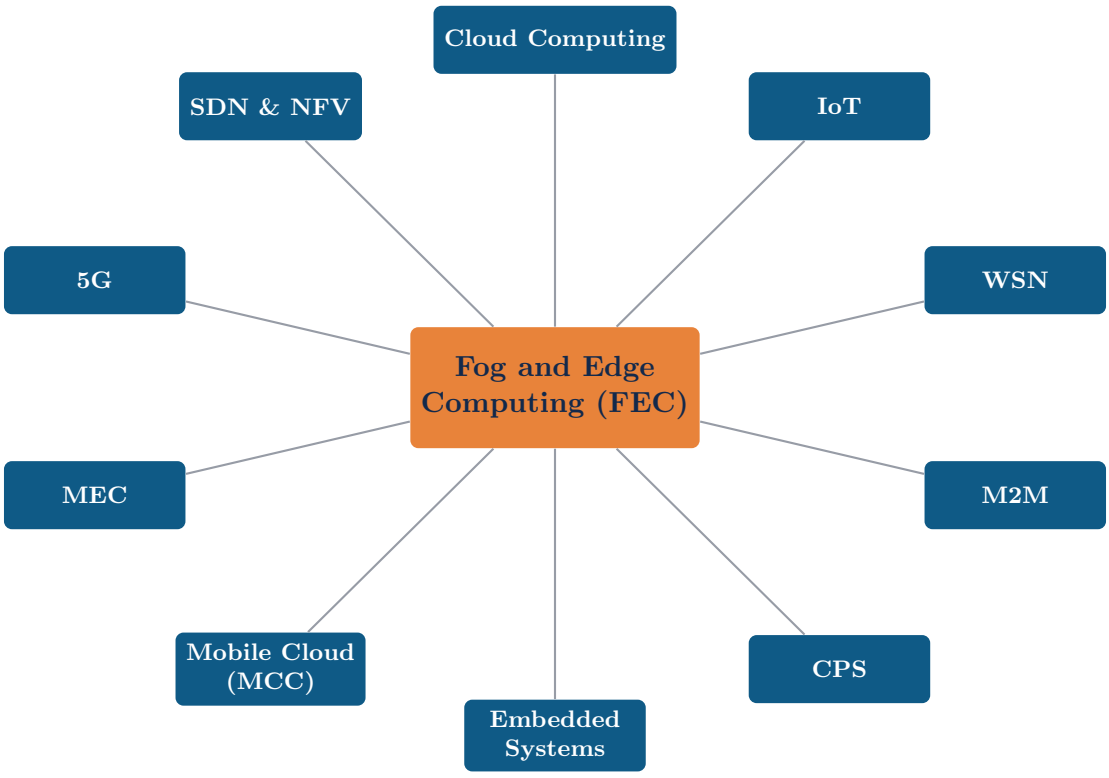


Figure 3.6: The technologies that come together in Fog and Edge Computing.

Let us look at each one, one at a time.

3.10.2 Internet of Things (IoT)

Core Concept

The **Internet of Things (IoT)** is a network of everyday physical objects — lights, meters, cars, watches, machines — that carry **sensors**, a small **processor** and a **network connection**, so they can collect and share data over the Internet.

Real-world examples:

- **Smart electricity meter** — sends your power usage to the electricity board automatically.
- **Fitness band** — tracks steps and heart rate.
- **Smart street light** — switches on only when it gets dark or someone passes.

Link to FEC: IoT devices create the huge flood of data that Fog and Edge Computing must handle. Without IoT, there would be no need for FEC.

3.10.3 Wireless Sensor Networks (WSN)

A **Wireless Sensor Network** is a group of many small, battery-powered sensor nodes spread over an area. Each node measures something and passes its reading, hop by hop, to a collection point called the **sink**.

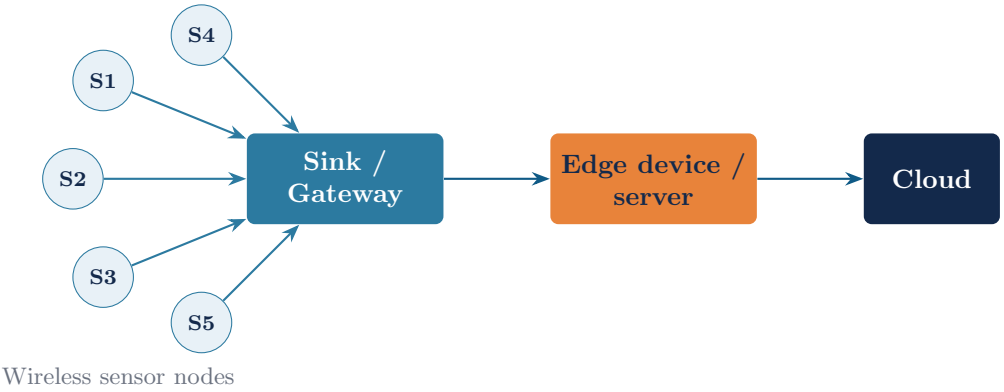


Figure 3.7: A wireless sensor network: sensor nodes report to a sink, which connects to an edge device and the cloud.

Real-world example: Sensors spread across a forest measure temperature and smoke, to warn early about forest fires.

Link to FEC: Sensor nodes are very weak (tiny battery, tiny memory). A nearby edge node does the heavy thinking for them.

3.10.4 Machine-to-Machine Communication (M2M)

M2M means machines talking *directly* to other machines, with little or no human help.

Real-world example: A vending machine tells the supplier's computer, "I am running out of water bottles," and a refill is scheduled automatically.

Link to FEC: Edge nodes let nearby machines coordinate with each other locally, without asking a far-away cloud every time.

3.10.5 Cyber-Physical Systems (CPS)

A **Cyber-Physical System** is a system where software (the "cyber" part) **senses and controls** physical things (the "physical" part) in a tight loop: *sense* → *compute* → *act*.

Real-world examples: a smart power grid, an industrial robot arm, the automatic braking system in a car.

Link to FEC: These control loops need answers in milliseconds, so the computing must happen at the edge.

3.10.6 Embedded Systems

Core Concept

An **embedded system** is a small computer built *inside* a larger device to do one dedicated job.

Real-world examples: the chip in a washing machine, the controller in a car's engine, the processor in a smartwatch, boards like Arduino and Raspberry Pi.

Link to FEC: Embedded systems give devices the power to *think for themselves*. Modern ones can even run small AI models, so a device can make decisions without the cloud.

3.10.7 From Mobile Computing to Mobile Edge Computing

This is a short history that leads straight to Edge Computing.

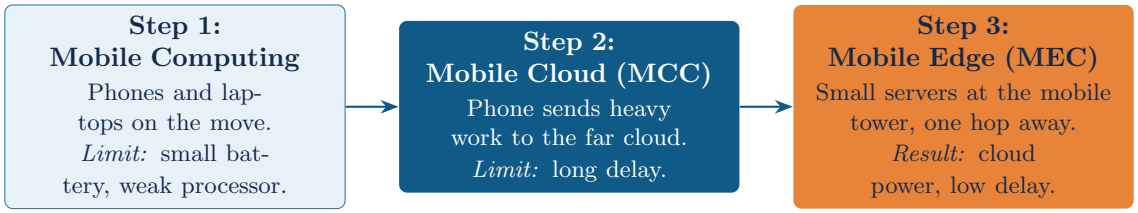


Figure 3.8: The path from mobile computing to mobile edge computing.

- **Mobile Cloud Computing (MCC):** the phone *offloads* heavy tasks (like recognising a photo) to the cloud and gets the result back. This saves battery, but the cloud is far, so it is slow.
- **Multi-access (Mobile) Edge Computing (MEC):** mobile operators place small servers at their **base stations** (mobile towers). Now heavy work is sent only one network hop away. MEC is a standard created by **ETSI** (the European Telecommunications Standards Institute).
- **Cloudlet:** a similar idea — a small data center in a building (a college, a coffee shop) that nearby phones can use.

3.10.8 5G Networks

Core Concept

5G is the fifth generation of mobile networks. In simple words, it is a mobile network that is **much faster**, has **very low delay**, and can connect a **huge number of devices** at the same time.

How 5G helps Edge Computing:

- **Very low delay** — devices reach a nearby edge server in a few milliseconds.
- **High speed** — large data like video and AR graphics move quickly.
- **Massive connections** — a very large number of IoT devices can connect in a small area.
- **Network slicing** — a separate "lane" of the network can be reserved for critical uses like ambulances or factory robots.
- **MEC inside 5G** — operators put edge servers inside the 5G network itself.

Real-life feeling: 5G is a wide, fast highway; the edge server is a service station built right next to the highway exit.

3.10.9 SDN and NFV

- **SDN (Software-Defined Networking):** The "brain" of the network is moved into software. A central controller decides how traffic flows, so data can be sent to the nearest or least-busy edge node.
- **NFV (Network Function Virtualization):** Network jobs such as firewalls and routers become *software* running on ordinary servers, instead of special boxes. Once the network is built from ordinary servers, the same servers can also run edge applications.

Easy way to remember: 5G gives the *speed*, SDN gives *smart routing*, and NFV turns the network itself into a place where we can *compute*.

3.10.10 Content Delivery Networks (CDN)

A **Content Delivery Network** is a group of servers spread across many cities that keep *copies* of popular content (videos, images, web pages) close to users. CDNs are one of the oldest and most successful forms of edge computing.



Figure 3.9: A CDN copies popular content to edge servers in each city, so viewers are served nearby.

Real-life example: When a new film releases on a streaming app, millions of people watch it.

Instead of all of them pulling the film from one far server, each city's CDN server keeps a copy. The film starts quickly and buffers less.

3.10.11 Quick Summary of Relevant Technologies

Technology	What it is	How it helps FEC
IoT	Connected everyday objects	Creates the data the edge processes
WSN	Tiny battery sensor nodes	Weak nodes; edge does the heavy work
M2M	Machines talking to machines	Edge coordinates machines locally
CPS	Software controlling physical things	Control loops need edge-level speed
Embedded systems	Small computers inside devices	Let devices decide for themselves
MCC	Phones offload work to the cloud	Led to moving the cloud closer
MEC	Servers at mobile towers	Cloud power one hop away
5G	Fast, low-delay mobile network	Fast link to edge servers
SDN / NFV	Software-controlled networks	Smart routing; network can host apps
CDN	Content copies near users	Earliest form of the edge idea

3.11 Fog and Edge Computing Completing the Cloud

3.11.1 What Does "Completing the Cloud" Mean?

Core Concept

Fog and Edge Computing do **not replace** the cloud. They **fill the gap** between the far-away cloud and the devices, so that computing is available everywhere — from the device, through nearby nodes, up to the central data center.

This smooth chain from the cloud down to the devices is called the **cloud-to-things continuum**.

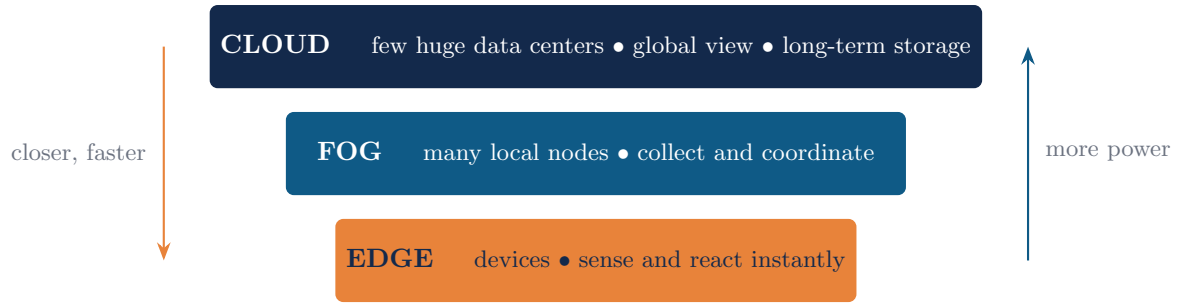


Figure 3.10: The cloud-to-things continuum. Moving up gives more power; moving down gives more speed and closeness.

3.11.2 An Easy Analogy: Touching a Hot Stove

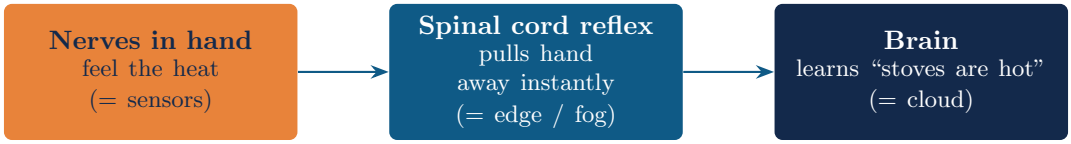


Figure 3.11: The hot-stove analogy: urgent decisions are made locally; learning happens centrally.

When you touch a hot stove, your hand moves away *before* your brain even thinks about it — the spinal cord handles it. Later, the brain learns and remembers. If every decision had to go to the brain first, you would get burnt. In the same way, if every IoT decision had to go to the cloud, many applications would be too slow. **Both levels are needed** — and that is exactly what "completing the cloud" means.

3.11.3 Two Big Questions: SCALE and SCANC

The whole idea of FEC can be understood through **two questions**:



Figure 3.12: SCALE tells us *why* FEC is useful; SCANC tells us *how* it works.

Understand the difference with a hospital example. A hospital monitors patients with IoT devices.

- What the hospital **wants** — quick responses, safe patient data, less network traffic, automatic coordination of devices — these are *goals*, so they belong to **SCALE**.
- What the hospital **uses** — local storage, computing, a GPU for AI, network protocols, control software — these are *means*, so they belong to **SCANC**.

A simple analogy: For a car, SCALE is what the car should give you (safety, speed, mileage); SCANC is the parts inside that make it happen (engine, brakes, sensors, gears).

Exam Tip

If a question asks for the **advantages** of FEC, write **SCALE**. If it asks **how an FEC node works** or what **mechanisms** it supports, write **SCANC**.

3.12 SCALE — The Five Advantages of FEC

3.12.1 S — Security

Core Concept

Security means protecting devices, applications and data from unauthorised access, harmful software and other attacks. FEC brings security services and updates **close to the devices**.

Two easy words first:

- A **vulnerability** is a weakness in software that an attacker can use — like a door with a broken lock.
- A **patch** is a small update from the maker that fixes that weakness — like repairing the lock.

Example: 500 soil sensors on a farm in Wayanad need a security patch.

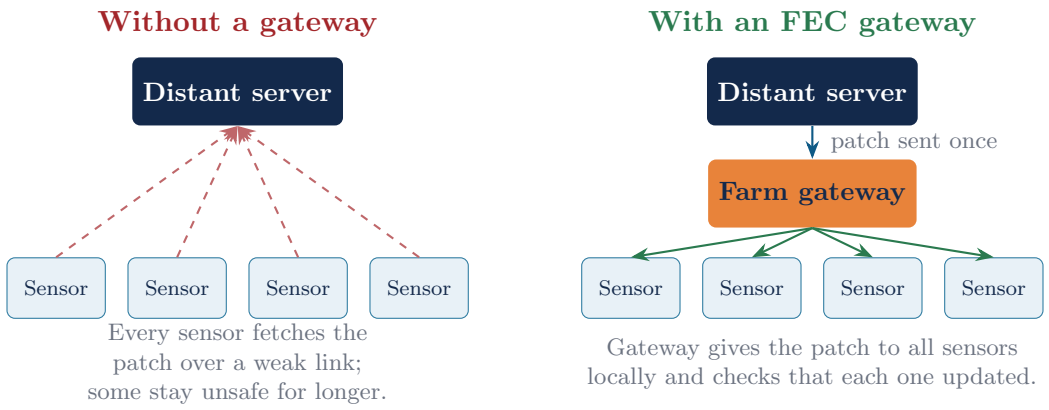


Figure 3.13: Distributing a security patch with and without a local FEC gateway.

Be careful: A gateway is *not* automatically safe. It must check the identity of each device, verify that updates are genuine, and protect passwords.

Other examples: a hospital gateway that decides which devices may read patient data; a factory gateway that checks whether a sensor is allowed to send control commands.

3.12.2 C — Cognition

Core Concept

Cognition means the system is **aware of its surroundings** (its context) and can **adapt its behaviour**. It does not mean human intelligence — it means smart rules, sensors and algorithms.

A cognitive system can do four things:

Ability	Simple meaning
Self-adaptation	Changes its behaviour when conditions change
Self-organisation	Arranges and coordinates its parts with little human help
Self-healing	Notices some failures and tries to recover
Self-expression	Tells other components what it can do and how it is doing

Example: a smart air conditioner with an occupancy sensor.

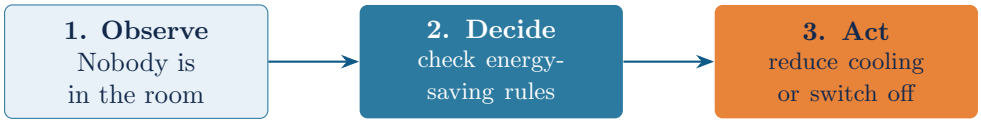


Figure 3.14: Cognition in action: observe, decide and act locally, without waiting for the cloud.

The AC does not send every reading to the cloud and wait. It moves from being a *passive* device (only reporting) to an *active* one (responding intelligently).

3.12.3 A — Agility

Core Concept

Agility is the ability to **develop, deploy and change** computing services quickly and flexibly.

The cloud is run by a few very large companies. FEC opens the door for **small organisations and local providers** to offer computing too, using small edge hardware and open software. The book mentions **ETSI MEC** (edge computing inside mobile networks) and **Indie Fog** (where ordinary people and small businesses rent out their spare equipment).

Example: A shop has a powerful local server that sits idle after business hours. With the shop's permission, proper security and limits, a local provider uses it to process nearby data for:

- local video analysis,
- traffic-data processing,
- environmental monitoring,
- smart-building management.

Important: An ordinary Wi-Fi router cannot safely rent out its resources. Suitable hardware, isolation and management are needed.

3.12.4 L — Latency

Core Concept

Latency is the time taken for data to travel through a system and for the **response** to come back. FEC reduces latency because data does not need to travel far.

Some tasks can wait; some cannot:

- A **monthly report** can wait a few seconds for the cloud.
- A **robot arm** must stop *immediately* when a worker's hand enters its danger zone.

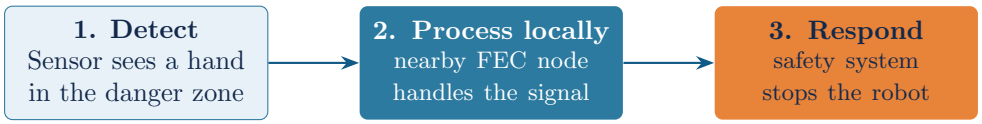


Figure 3.15: Low latency at the edge: an industrial robot is stopped without a trip to the cloud.

A new word — Softwarization: changing what a device does by changing its *software*, instead of replacing the hardware. For example, a new routing rule or a new processing task can be loaded in seconds. This makes quick local changes possible.

Note: Safety-critical machines still need properly certified safety systems. FEC helps, but it alone does not guarantee a safe stop.

3.12.5 E — Efficiency

Core Concept

Efficiency means using computing power, network bandwidth, storage and energy **wisely** — processing and filtering data nearby, and sending only what matters to the cloud.

Example: an elderly-care home with 100 wearable devices.

Without local processing	With local processing
Every reading goes to the cloud	A gateway receives all readings
Network usage is high	It spots patterns that need attention
The cloud handles thousands of routine readings	It sends only alerts and summaries
Alerts arrive late	Staff get alerts immediately

The exact saving depends on how much data is filtered and how often devices send — but the idea is always the same: **process near, send less**.

3.12.6 Compare All Five SCALE Advantages

Advantage	Main question	Example
Security	How do we protect devices and data?	Gateway distributes verified updates
Cognition	How can a system adapt to its surroundings?	AC responds to room occupancy
Agility	How can services be deployed flexibly?	Local provider offers edge processing
Latency	How can we respond quickly?	Robot stopped locally
Efficiency	How can we reduce waste?	Filtering sensor data locally

Quick classroom check — which advantage matters most?

- A banking app with confidential data → **Security**
- A smart home that adjusts to who is present → **Cognition**
- A robot that must react quickly → **Latency**
- A remote village with little bandwidth → **Efficiency**
- A small business launching a new local service → **Agility**

Remember: one application usually enjoys several advantages at once.

3.13 SCANC — How Fog and Edge Computing Works

3.13.1 The FEC Node

An **FEC node** is any computer near the devices that can do useful work for them — a smart gateway, an edge server, a roadside computer or a suitably equipped network box. Think of it as a **small computing center** serving one college, factory, hospital or neighbourhood. Every FEC node offers five mechanisms, remembered as **SCANC**.

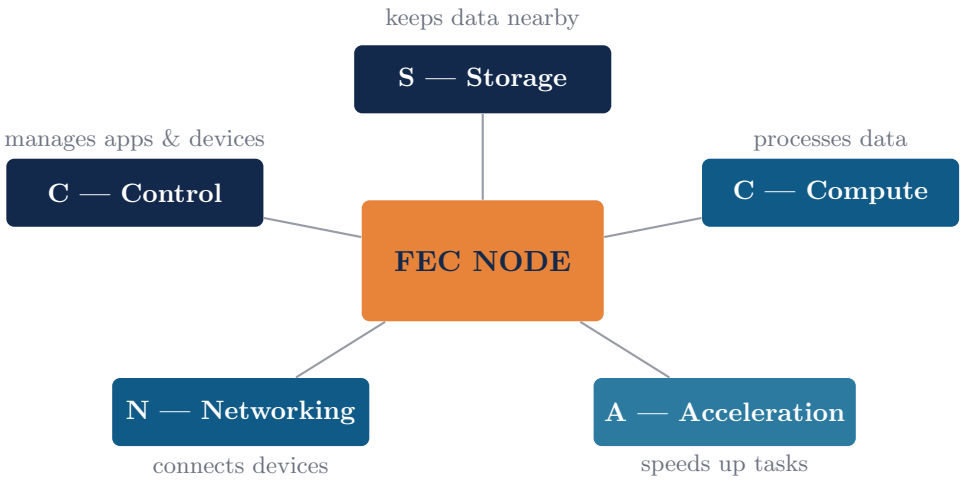


Figure 3.16: The five mechanisms (SCANC) of an FEC node.

Mechanism	Simple meaning	College example
Storage	Keeps useful data nearby	Stores recent CCTV footage
Compute	Processes information	Analyses video to detect people
Acceleration	Speeds up selected tasks	A GPU for AI video analysis
Networking	Connects devices and services	Collects readings from wireless sensors
Control	Manages apps and actions	Deploys an app or controls a device

How they work together — traffic at the college gate:

1. **Networking:** cameras send video to a nearby gateway.
2. **Storage:** the gateway keeps the recent video.
3. **Compute:** software counts the vehicles.
4. **Acceleration:** a GPU speeds up the video analysis.
5. **Control:** the system shows a warning or changes the gate signal.

Core Concept

SCALE describes the **benefits** we want; SCANC describes the **capabilities** that deliver them. They are related, but not a strict one-to-one match.

3.13.2 S — Storage

Core Concept

Storage means keeping data **on or near the FEC node**, so it can be used again quickly without fetching it from a far server. A local copy of data that is likely to be needed again is called a **cache**.

Example 1: A popular film in Kozhikode. 1,000 people in Kozhikode watch the same new film.



Figure 3.17: Caching: the film is copied once to a nearby cache and served to many viewers from there.

Benefits: less repeated transfer from the far server, faster start, less load on long-distance links, smoother streaming. **But:** if the original changes, the cache must be refreshed.

Example 2: A pothole warning for connected cars. (A *pothole* is a hole in the road surface, often formed after heavy rain.)

1. A car detects a pothole and tells a roadside computer.
2. The roadside computer stores the warning with its location and time.

3. Other cars approaching that road receive the warning.

4. When the pothole is repaired, the warning is removed.

Storage is not the same as computation. Saving a video is *storage*. Counting the people in that video is *computation*.

3.13.3 C — Compute

FEC nodes offer processing in two main ways.

3.13.3.1 I/PaaS — Bring Your Own Application

In **Infrastructure / Platform as a Service (I/PaaS)**, the FEC provider gives you a ready computing environment — a **virtual machine (VM)** or a **container** (for example, Docker) — and you run *your own* program in it.

Example: A college computer science department writes its own vehicle-detection program and runs it on a nearby FEC server, so CCTV footage is analysed locally.

Kitchen analogy: You rent a kitchen with all the equipment, but you bring your own recipe and ingredients.

3.13.3.2 SaaS — Use a Ready-Made Service

In **Software as a Service (SaaS)**, you simply use a service that is already running on the FEC node. The book gives two kinds:

Feature	ODP — On-demand Data Processing	CaaS — Context as a Service
Input	Data you send with a request	Information the node gathers itself from its surroundings
When it works	When you ask	May collect and analyse all the time
Output	The result of processing your data	Useful information about a situation
Example	Upload a photo → "Three people detected"	Ask about a road → "Heavy traffic on this road"

Exam Tip

Own app on a VM or container → **I/PaaS**. Ready service that processes *your* data → **ODP**. Ready service that tells you about the *surroundings* → **CaaS**.

3.13.4 A — Acceleration

Acceleration means making selected tasks faster or more efficient. There are two kinds.

3.13.4.1 Networking Acceleration

The route a network operator chooses is not always the fastest route for *your* application. Two technologies help:

- **SDN (Software-Defined Networking)** lets routes be chosen by software.
- **Network virtualization** lets several logical networks (and routing tables) run on one physical network.

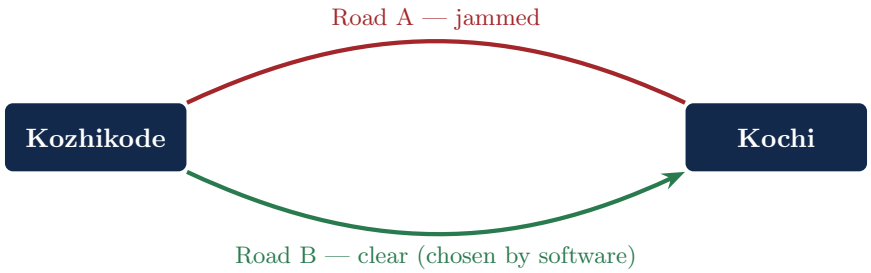


Figure 3.18: The road analogy for SDN: a smart traffic system sends vehicles along the clear road.

Note: SDN cannot pick *any* path on the Internet; it works within the paths and permissions the network allows.

3.13.4.2 Computing Acceleration

Special chips do certain jobs much faster than a normal processor (CPU).

Feature	GPU (Graphics Processing Unit)	FPGA (Field-Programmable Gate Array)
Main strength	Thousands of small calculations in parallel	Hardware logic that can be re-wired after manufacturing
Typical use	AI, image and video analysis	Streaming data, signal processing, special AI tasks
Example	Detect people in factory video	Re-configure a road camera from counting vehicles to checking helmets
Energy use	Depends on the work	Can be very efficient for the right task

Neither chip is always better — it depends on the task, design and power budget.

3.13.5 N — Networking

Networking connects devices, FEC nodes and the cloud. It works in two directions.

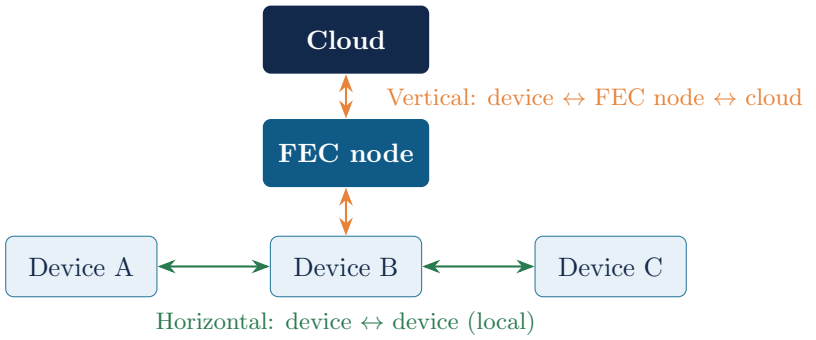


Figure 3.19: Vertical networking connects the layers; horizontal networking connects nearby devices.

3.13.5.1 Vertical Networking

Vertical networking connects the levels: device → FEC node → cloud. It uses two styles of communication.

A. Request / Response. A client asks, a service answers. Example: a college app asks for the lab temperature and gets the latest reading. Technologies: **HTTP** (web and APIs), **CoAP** (a light protocol for small devices) and **TCP/UDP sockets**.

B. Publish / Subscribe. A device *publishes* a message to a **topic** on a **broker**; everyone who has *subscribed* to that topic receives it. Technologies: **MQTT**, **XMPP**, **AMQP**.

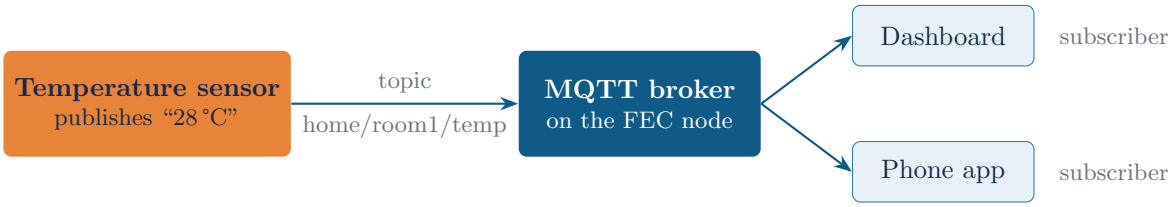


Figure 3.20: Publish/subscribe with MQTT: one message, delivered to every subscriber of the topic.

Real-life comparison: It is like a WhatsApp group — one person posts, and everyone in the group receives the message.

3.13.5.2 Horizontal Networking

Horizontal networking connects nearby devices *directly*, so not everything has to go through the cloud. Technologies: **Bluetooth**, **ZigBee** (based on IEEE 802.15.4) and **Z-Wave**.

Example: A motion sensor in a corridor detects a person and tells the light controller; the lights switch on immediately — no cloud needed.

Feature	Vertical	Horizontal
Direction	Device → gateway → cloud	Device ↔ device
Purpose	Connect computing levels	Local interaction

Example	Sensor sends readings to the cloud	Motion sensor switches on corridor lights
Technologies	HTTP, CoAP, MQTT	Bluetooth, ZigBee, Z-Wave

Why gateways are so useful: a gateway speaks both "languages". It can read ZigBee sensors on one side and send the data over the Internet (IP) on the other.

3.13.6 C — Control

Control decides *what runs where*, *what devices do*, *how different systems talk*, and *who is allowed in*.

Type	Main question	Example
Deployment	Which program runs where, and when?	Move CCTV analysis from an overloaded gateway to a less busy one
Actuation	What action should a device perform?	Smoke detected → local controller triggers the alarm
Mediation	How can different systems talk to each other?	Translate a road-hazard message between two car brands
Security	Who can access which resources?	Only approved apps may read patient data on a hospital gateway

- An **actuator** is a part that performs a physical action — opening a valve, switching a light, running a motor.
- **Authentication** checks *who* you are; **authorization** checks *what* you are allowed to do.

Exam Tip

Do not mix up the first two: **deployment** controls where *software* runs; **actuation** controls the *physical actions* of devices.

3.13.7 Quick Revision: SCALE and SCANC

SCALE — why FEC is useful	SCANC — how FEC works
Security — verified updates nearby	**Storage** — caching video, recent readings
Cognition — AC reacts to occupancy	**Compute** — VMs, containers, ODP, CaaS
Agility — deploy a local edge app quickly	**Acceleration** — SDN, GPUs, FPGAs
Latency — local industrial control	**Networking** — MQTT, CoAP, Bluetooth, ZigBee
Efficiency — filter sensor data locally	**Control** — deploy, actuate, mediate, secure

3.14 Putting It All Together

3.14.1 Example 1: A Smart Traffic Junction

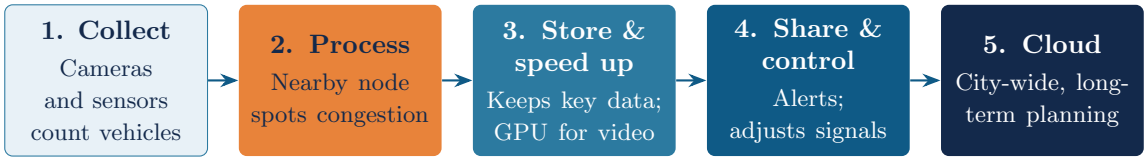


Figure 3.21: A smart traffic junction uses every SCANC mechanism and enjoys every SCALE advantage.

In this one application, **SCALE** describes the benefits (security, cognition, agility, low latency, efficiency) and **SCANC** describes the capabilities (storage, compute, acceleration, networking, control). Nearby processing and the central cloud work *together*.

3.14.2 Example 2: A Smart Hospital

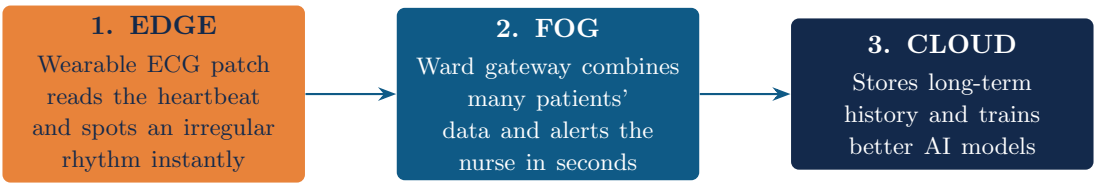


Figure 3.22: A smart hospital, layer by layer: fast decisions near the patient, heavy work in the cloud.

Core Concept

The main lesson is **not** that the cloud is unnecessary. Cloud, fog and edge work **together**, each doing the job best suited to its place.

3.15 Advantages and Disadvantages of Edge Computing

Advantages

1. **Very low latency** — decisions in milliseconds because data does not travel far.
2. **Saves bandwidth and cost** — only filtered data and summaries go to the cloud.
3. **Works offline** — critical functions continue even when the internet is down.
4. **Better privacy** — raw personal data can stay inside the building.
5. **Less network congestion** — the core network carries much less data.
6. **Easy to scale locally** — add more edge nodes where demand grows.
7. **More reliable** — no single central point of failure.

Disadvantages

1. **Limited resources** — edge devices have small processors, memory and batteries.
2. **Hard to manage** — thousands of devices must be updated and monitored.
3. **Physical risk** — devices in public places can be stolen or tampered with.

4. **More points to secure** — every device is a possible entry point for attackers.
5. **Upfront cost** — many devices, gateways and servers must be bought and installed.
6. **No global view** — the cloud is still needed for big analysis and long-term storage.

3.16 Where Is Edge Computing Used?

Area	How the edge is used	Main benefit
Self-driving cars	On-board computer processes cameras and radar	Millisecond braking
Smart homes	Voice assistant hears its wake word on the device	Fast response, privacy
Healthcare	Wearables and bedside gateways check vital signs	Instant alerts
Factories	Edge server analyses machine vibration	Faults caught early
Retail	Shelf sensors detect low stock	No empty shelves
Agriculture	Field gateway controls water pumps	Works without internet
Smart cities	Junction controllers manage traffic lights	Less congestion
Video streaming	CDN servers keep films near viewers	Less buffering
Gaming and AR	Edge servers near players	Less lag
Energy	Substation edge detects grid faults	Prevents outages

3.16.1 When Should You Choose the Edge, and When the Cloud?

Choose the Cloud when...	Choose the Edge when...
The work is not urgent (reports, backups)	A reaction is needed in milliseconds
Huge computing power is needed (AI training)	There is too much data to send
A global view of all data is needed	The internet is weak or unreliable
Data must be shared across many locations	Privacy requires data to stay local
Long-term storage is needed	The task is local (one machine, one room)

3.17 Hierarchy of Fog and Edge Computing

Where exactly do fog and edge computers sit?

3.17.1 What Does "Hierarchy" Mean?

Core Concept

The **hierarchy** of Fog and Edge Computing is the arrangement of computing resources in **layers**, from the devices at the bottom up to the cloud at the top. Each layer has its own job.

Real-life comparison: A country is run at three levels — the village panchayat handles local matters quickly, the district office handles a wider area, and the central government plans for the whole country. Computing is organised the same way.

3.17.2 The Simple Three-Layer View

Layer	Position	Role	Examples
Edge	Closest to users	Sense, act, take real-time decisions, filter data	Sensors, cameras, phones, edge servers
Fog	Middle	Collect from many devices, coordinate, local analytics, short-term storage	Gateways, routers, local servers, cloudlets
Cloud	Farthest from users	Long-term storage, big-data analytics, AI training, global management	AWS, Azure, Google Cloud

Load distribution means spreading the work across these layers so that no single layer is overloaded: urgent, small jobs at the edge; area-level jobs in the fog; heavy, global jobs in the cloud.

3.17.3 The Book's View: Inner, Middle and Outer Edge

Our reference book (Buyya and Srirama, *Fog and Edge Computing: Principles and Paradigms*) looks more closely at the space between the cloud and the devices and divides it into **three tiers**.

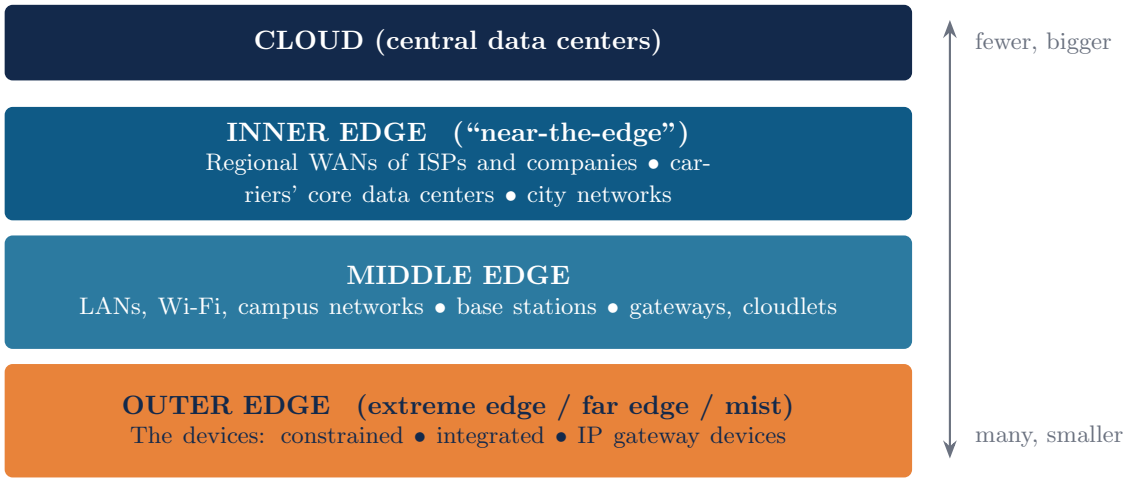


Figure 3.23: The hierarchy of Fog and Edge Computing: inner edge, middle edge and outer edge.

3.17.3.1 Inner Edge

- **What it covers:** the wide-area networks (WANs) of big companies and Internet Service Providers (ISPs), the core data centers of mobile operators (called the *Evolved Packet Core*, EPC), and metropolitan networks that cover a city.
- **Its job:** earlier it only provided connectivity; today it also caches and processes data in many locations to give users a better experience.
- **Examples:** Google places its servers inside ISPs' data centers (the Google Edge Network) so that its services load faster; telecom companies such as AT&T, Vodafone, Telstra and Deutsche Telekom offer "local cloud" services inside a country.

3.17.3.2 Middle Edge

- **Local Area Networks (LAN):** Ethernet, Wi-Fi and campus networks. Routers and gateways with virtualization (for example, Cisco's IR829 industrial router) can run applications. Servers one hop away from the devices form a small local cloud called a **cloudlet**.
- **Cellular networks:** base stations of all sizes (macro, micro, pico and femto cells) can host edge servers for connected cars, mobile healthcare, and AR/VR. Companies like Nokia, ADLink and Huawei make MEC-ready hardware.

3.17.3.3 Outer Edge

The outer edge is the devices themselves. The book divides them into three kinds.

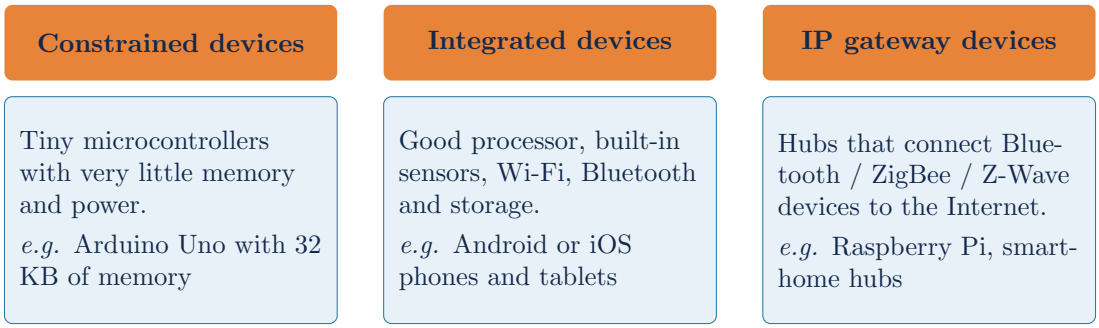


Figure 3.24: The three kinds of outer-edge devices.

- **Constrained devices** can have their software (firmware) updated remotely, so they can become self-aware and cooperate with each other (machine-to-machine). Computing at this level is sometimes called **mist computing**.
- **Integrated devices** have operating systems that limit virtualization, but tools like Termux are early steps towards running edge applications on them.
- **IP gateway devices** usually run Linux and can run Docker containers, so they are popular as FEC nodes in projects and research.

3.18 Business Models

How companies — and even individuals — earn money from Fog and Edge Computing

3.18.1 First, Some Basic Words

Term	Simple meaning
Business model	How a company creates value for customers and earns money from it
Revenue model	The exact way money comes in: subscription, pay-per-use, licence, and so on
Vendor	A company that sells edge hardware or software (Cisco, Dell, NVIDIA)
Service provider	A company that runs edge services for customers (telecom operators, cloud providers)
Ecosystem	All the players who together make edge computing work
Market opportunity	A growing need that a business can earn from

3.18.2 The Three Business Models of FEC

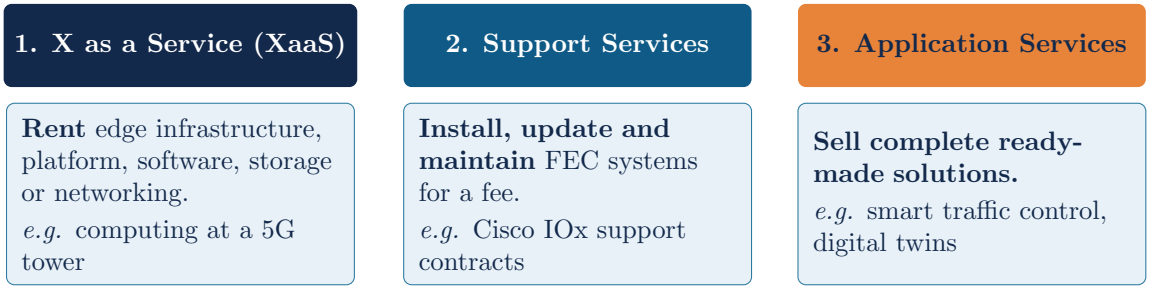


Figure 3.25: The three business models of Fog and Edge Computing.

3.18.2.1 X as a Service (XaaS)

Just like cloud services, edge resources — **infrastructure, platform, software, networking, cache or storage** — are rented out. A special idea from the book is **IndieFog**: ordinary people and small businesses become providers by renting out the spare capacity of equipment they already own ("consumer as provider"). The book mentions similar real ideas such as Fon, which builds a worldwide Wi-Fi network from customers' home routers.

3.18.2.2 Support Services

Providers **install, configure, update and maintain** FEC software on the customer's own hardware, usually through monthly or yearly contracts. **Example:** Cisco sells IOx-enabled routers, and customers pay extra for software updates and technical support.

3.18.2.3 Application Services

Providers deliver **complete solutions**, so the customer does not have to build anything. Examples from the book:

- **Digital twins** — a live virtual copy of a factory or machine, analysed at the edge within tens of milliseconds.
- **Traffic control** — real-time systems for city governments that support connected and self-driving vehicles.
- **Ambient Assisted Living** — systems that help elderly people live safely at home.

3.18.3 Edge as a Service

Core Concept

In **Edge as a Service (EaaS)**, a provider owns and runs edge servers (at mobile towers, micro data centers or customer sites) and **rents** computing, storage and networking near the users, on demand.

The customer does not buy any hardware. The provider takes care of power, cooling, security and updates. Examples include telecom operators' MEC services and cloud services such as AWS Wavelength and AWS Outposts.

3.18.4 Pay-Per-Use vs Subscription

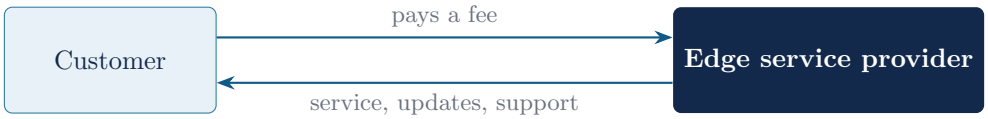


Figure 3.26: How an edge service business works: the customer pays, the provider delivers and maintains the service.

Feature	Pay-per-use	Subscription
How you pay	For what you actually use (hours, GB, requests)	A fixed fee every month or year
Monthly bill	Changes with usage	Same every month
Best for	Small users, uneven or seasonal demand	Steady, continuous usage
Risk to customer	The bill can suddenly rise	Paying for capacity you do not use
Good for provider because	It attracts small customers	It gives steady income
Example	Paying per GPU-hour at a 5G edge	Monthly platform licence for a factory

Real-life comparison: Pay-per-use is like an auto-rickshaw meter — you pay for the distance you travel. A subscription is like a monthly bus pass — one fixed price, however much you travel.

3.18.5 The Edge Computing Ecosystem

No single company can build edge computing alone. Many players work together:

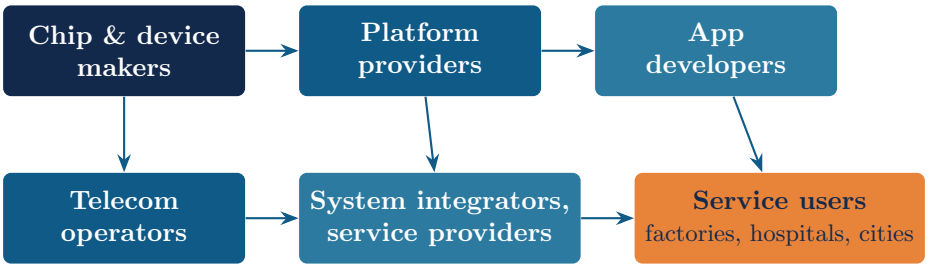


Figure 3.27: The edge computing ecosystem: many companies combine their parts to serve the final users.

3.18.6 Monetizing Edge Data

Monetization means earning money from something. Data collected at the edge can be turned into money in three main ways:

1. **Selling insights** — for example, anonymous footfall counts from a mall to shop owners.
2. **Premium services** — for example, an instant-alert service for a monthly fee.
3. **Saving costs** — for example, predicting machine failures so the factory loses less money.

Important: Data must be used only with **consent** and according to data-protection laws.

3.19 Edge Computing Platforms

3.19.1 What Is an Edge Computing Platform?

Core Concept

An **Edge Computing platform** is software that lets an organisation **deploy, run, manage and secure** applications on many edge devices **from one central place**.

Why we need it: Imagine a company with 10,000 edge devices in 500 cities. Updating each one by hand would take months. A platform lets one engineer update all of them from a single screen.

The word **platform** simply means the *base layer* of software and tools on which applications are built and run.

3.19.2 What Does a Platform Do?

Job	Simple meaning
Runtime	Runs applications inside containers or virtual machines on each device
Deployment	Installs and starts an application or update on edge devices, usually remotely
Orchestration	Automatically decides where each app runs; starts, stops, scales, updates or moves it
Device management	Registers, configures, monitors, updates and retires devices
Connectivity	Talks to devices using many protocols (MQTT, Modbus, OPC UA, Bluetooth)
Local data and AI	Processes data and runs AI models on the device
Offline and sync	Keeps working without internet; syncs with the cloud later
Security	Device identity, encryption, access control
Multi-vendor support	Works with devices and software from different manufacturers

3.19.3 Orchestration: One Controller, Many Devices

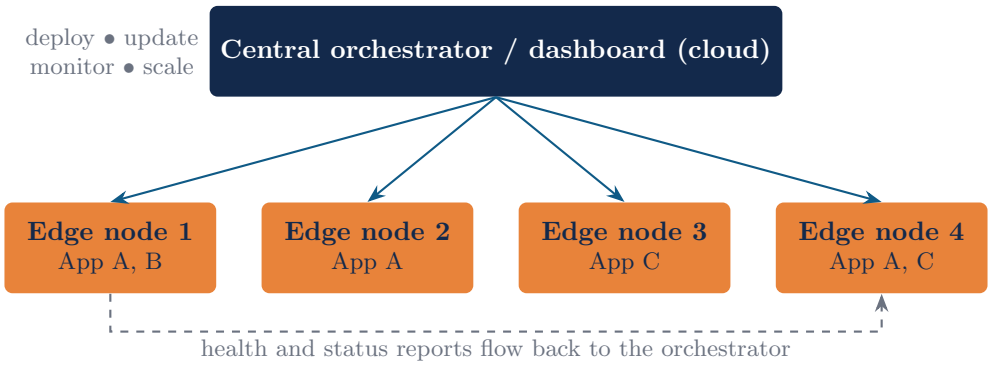


Figure 3.28: Orchestration: one central controller deploys, updates and monitors applications on many edge nodes.

3.19.4 Platforms Named in the Book

Name	Type	What it does
ETSI MEC	Standard	Rules and APIs for computing inside mobile networks
OpenFog Reference Architecture	Standard	A blueprint for fog systems; adopted as IEEE 1934-2018
Cisco IOx	Commercial	Runs applications on Cisco routers and gateways
EdgeX Foundry	Open source	A vendor-neutral toolkit from the Linux Foundation for connecting devices
OpenStack++	Open source	VM-based cloudlets for mobile offloading (Carnegie Mellon University)
WSO2 IoT Server	Open source	Connects devices via MQTT/XMPP; stream processing on devices
Apache Edgent	Open source	Stream processing split between cloud and edge (from IBM)

3.19.5 Major Platforms Used Today

Platform	Company	One key feature
AWS IoT Greengrass	Amazon	Runs functions, containers and machine-learning models on devices; works offline
Azure IoT Edge	Microsoft	Apps packed as container "modules", managed from Azure IoT Hub
KubeEdge	CNCF (open source, started by Huawei)	Extends Kubernetes to edge nodes; became a CNCF graduated project in 2024
EdgeX Foundry	LF Edge, Linux Foundation	Open, vendor-neutral microservices to connect any device
Google Distributed Cloud Edge	Google	Google-managed Kubernetes on hardware at the customer's site
NVIDIA Jetson	NVIDIA	Small GPU computers for AI at the edge

Note: Platforms change quickly. For example, Google retired its Cloud IoT Core service in August 2023. Always check the official documentation for the latest status.

3.19.6 Checklist: Choosing a Platform

1. Write down what you need: how fast, how much data, must it work offline?
2. List your existing devices, protocols and vendors.
3. Check that the platform runs on your hardware.
4. Check its security and legal-compliance features.
5. Check how it connects to your cloud, and whether you get locked in to one vendor.
6. Compare the total cost: licences, hardware and support.
7. Try it in a small pilot first, measure the results, then expand.

3.20 Opportunities and Challenges

3.20.1 Opportunities

Opportunities

1. **New low-delay services** — self-driving cars, AR/VR, cloud gaming, remote surgery, smart factories.
2. **5G and edge together** — telecom companies can sell edge computing and reserved network "slices", a brand-new source of income.
3. **A growing hardware market** — the book quotes a forecast that FEC hardware would reach about US$7.66 billion by 2022, with more and more routers and gateways shipping "edge-ready".
4. **Room for small players** — start-ups and even individuals (IndieFog) can become

providers.

5. **Edge intelligence** — running AI on the device itself opens up new products.
6. **Industry gains** — healthcare (remote monitoring), retail (smart stores), manufacturing (predictive maintenance), agriculture, energy and smart cities.

3.20.2 Challenges

Challenges

1. **Security** — thousands of devices means thousands of possible entry points.
2. **Physical security** — devices at bus stops and in parks can be stolen or opened.
3. **Device management** — registering, updating and monitoring thousands of devices.
4. **Interoperability** — devices from different makers may not work together.
5. **Standardization** — there are not yet enough common rules that everyone follows.
6. **Scalability** — growing from 50 devices to 5,000 without breaking.
7. **Limited resources** — small processors, little memory, small batteries.
8. **Cost** — buying, installing and maintaining equipment at many sites.
9. **Maintenance** — repairing devices spread across remote places.
10. **Energy** — many devices running all day use a lot of power.
11. **Network reliability** — links to edge sites can be slow or drop.
12. **Data ownership** — it is not always clear who owns edge data: the user, the maker or the provider.
13. **Compliance** — following data-protection laws such as India's DPDP Act 2023 or Europe's GDPR.

3.20.3 How Can We Handle These Challenges?

Challenge	Simple solution
Security	Encryption, device identity checks, secure start-up (secure boot), regular patches
Physical security	Locked tamper-proof boxes, tamper alarms, encrypted storage, CCTV
Device management	A central platform with over-the-air updates and automatic monitoring
Interoperability	Open protocols (MQTT, OPC UA), open platforms (EdgeX), translating gateways
Standardization	Follow standards such as ETSI MEC and IEEE 1934
Scalability	Automation, extra fog nodes, cloud-native tools
Limited resources	Light AI models, offload heavy work to the fog, add GPU/AI chips
Cost	Edge as a Service, pay-per-use, start small and grow
Maintenance	Remote monitoring, predictive maintenance, remote restart
Energy	Low-power chips, sleep modes, solar power, send less data
Network reliability	Offline mode, store-and-forward, backup links (wired + 4G/5G)
Data ownership	Clear contracts, user consent, data policies
Compliance	Keep personal data local, anonymize it, keep access logs, audit regularly

3.20.4 The Book's Three Big Challenges

The reference book highlights three challenge areas.

1. **Out-of-Box Experience (OOBE).** Customers want edge equipment to work like a new phone: switch on and go. This can be done by selling routers with the edge software already inside (*OOBE equipment*), or by letting users install edge software like installing Windows (*OOBE software*). *Difficulty:* which software to include, different hardware types (x86 and ARM) need different versions, and costs are high without common standards.

2. **Open Platforms.** Commercial platforms exist, but open ones were few and early. No platform could yet manage *all three* tiers (inner, middle and outer edge) together. Customers also want to avoid being locked to one vendor.

3. **System Management.** Managing an FEC system happens in three phases:

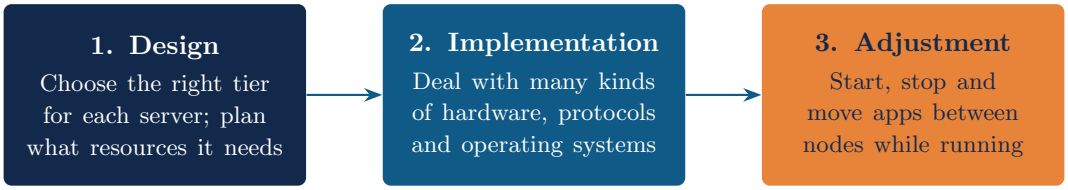


Figure 3.29: The three phases of FEC system management.

In the adjustment phase, moving software between nodes must be reliable and fast — which is hard at outdoor sites where wireless signals often drop.

3.20.5 Opportunities and Challenges at a Glance

Opportunities	Challenges
New low-delay services	Security of many scattered devices
New income for telecom companies with 5G	High setup and maintenance cost
Lower bandwidth and cloud bills	Limited power of each device
Better privacy — data stays local	Devices from different makers not working together
Works offline, more reliable	Managing thousands of devices
Jobs and start-ups in edge AI	Lack of standards; legal compliance

Why study both? A company that sees only the opportunities may waste money; a company that sees only the challenges may miss a big chance. Looking at both helps it decide wisely — often by starting with a **small pilot** and then expanding.

3.21 Quick Revision Summary

- **Edge Computing** = processing data at or near where it is created. **Fog Computing** = the middle layer of gateways and local servers. **Cloud** = central data centers.
- **Why edge?** Latency, bandwidth, connectivity, privacy.
- **Relevant technologies:** IoT, WSN, M2M, CPS, embedded systems, MCC, MEC, 5G, SDN, NFV, CDN.
- **FEC completes the cloud** — it does not replace it; together they form the cloud-to-things continuum.
- **SCALE (advantages):** Security, Cognition, Agility, Latency, Efficiency.
- **SCANC (mechanisms):** Storage, Compute, Acceleration, Networking, Control.
- **Compute:** I/PaaS (your own app), SaaS — ODP (process my data) and CaaS (tell me about the surroundings).
- **Networking:** vertical (HTTP, CoAP, MQTT) and horizontal (Bluetooth, ZigBee, Z-Wave).
- **Control:** deployment, actuation, mediation, security.
- **Hierarchy:** Edge → Fog → Cloud; in the book, inner, middle and outer edge (constrained, integrated, IP gateway devices).
- **Business models:** X as a Service, support services, application services; also Edge as a Service, pay-per-use, subscription.
- **Platforms:** AWS IoT Greengrass, Azure IoT Edge, KubeEdge, EdgeX Foundry, Cisco IOx.
- **Challenges:** security, device management, interoperability, standards, limited resources, cost, energy, compliance; in the book, OOBE, open platforms, system management.

3.21.1 The Big Question: Where Should This Task Run?

> "When a sensor produces data, how do we decide whether to handle it at the edge, in the fog, or in the cloud?"

The answer in plain words: Ask three questions. *Is it urgent?* Then do it at the edge. *Does it need data from many devices in one area?* Then do it in the fog. *Does it need huge power, a global view or long-term storage?* Then send it to the cloud.

3.22 Likely Exam Questions and Short Answers

Marks	Question
2 marks	What is Edge Computing? What is Fog Computing? Define latency and bandwidth. What is a gateway? What is edge analytics? What is the hierarchy of FEC? What is Edge as a Service? What is orchestration? What is interoperability?
6 marks	Compare Cloud, Fog and Edge Computing. Explain three technologies that support Edge Computing. Draw the Cloud–Fog–Edge architecture. Explain SCALE. List and explain three business models. Explain three challenges of Edge Computing.
8 marks	Apply Edge Computing to a given scenario (self-driving car, smart hospital, smart city, remote farm). Explain how the Cloud–Fog–Edge hierarchy handles data in a smart city. Discuss a business model or platform choice for a company.

Model short answers (2 marks):

- **Edge Computing:** processing data at or near its source instead of in a distant cloud data center, e.g. a smart camera detecting motion by itself.
- **Fog Computing:** a middle layer of gateways, routers and local servers between edge devices and the cloud that collects, filters and processes data for many devices.
- **Latency:** the delay between sending data and receiving a response, measured in milliseconds.
- **Bandwidth:** the maximum amount of data a network link can carry per second.
- **Gateway:** a device that connects local devices to the Internet, translating protocols and filtering data.
- **Edge analytics:** analysing data at the edge to get insights and alerts immediately.
- **Hierarchy of FEC:** the arrangement of computing in layers — edge, fog and cloud — each with its own role.
- **Edge as a Service:** renting computing, storage and networking near users from a provider who owns the edge servers.
- **Orchestration:** automatically deciding where applications run on many edge nodes and starting, updating or moving them.

- **Interoperability:** the ability of devices and software from different makers to work together.

Tip for the 8-mark answer

Follow this order: **introduction** (define Edge Computing, restate the problem) → **why the cloud alone fails here** (latency, bandwidth, connectivity or privacy) → **step-by-step edge solution** (sense → process locally → act → send summary) → **diagram** (Device → Edge → Fog → Cloud, labelled with the scenario's devices) → **benefits** (from SCALE) → **one-line conclusion**.

3.23 Practice Section

3.23.1 Group Problem: Where Should Each Task Run?

A factory has **200 sensors**. Each sensor sends a **1 KB** reading every **10 milliseconds** (that is, 100 readings per second). The factory's internet upload speed is **50 Mbps**. A machine must be stopped within **20 ms** if a fault appears. The round trip to the cloud takes about **80 ms**; a local edge server answers in about **2 ms**.

Step 1 — Can we send all raw data to the cloud?

$$200 \text{ sensors} \times 100 \text{ readings/s} \times 1 \text{ KB} = 20{,}000 \text{ KB/s} = 20 \text{ MB/s} = 160 \text{ Mbps}$$

The upload link is only 50 Mbps, so **no** — the raw data is more than three times what the link can carry.

Step 2 — What if the edge sends one average per second per sensor?

$$200 \times 1 \text{ KB/s} = 200 \text{ KB/s} = 1.6 \text{ Mbps}$$

That is **100 times less** and fits easily.

Step 3 — Can the cloud stop the machine in time? The cloud needs about 80 ms, but the limit is 20 ms, so **no**. The edge server needs about 2 ms, so **yes**.

Step 4 — Place each task.

Task	Where?	Why
Stop a faulty machine	Edge	Must happen within 20 ms
Average readings every second	Edge	Cuts data 100 times
Live dashboard for the whole plant	Fog	Needs data from all machines in one place
Alarm to the shift supervisor	Fog	Area-level decision, needed in seconds
Yearly trend report	Cloud	Needs long-term storage
Training a better fault-detection model	Cloud	Needs huge computing power

Check: Urgent → edge. Area-level → fog. Heavy or long-term → cloud. Always calculate the *data rate* and the *time limit* before deciding.

3.23.2 Check Your Understanding

Q1. Why can't we simply send all IoT data to the cloud?
Because it causes high delay, uses too much bandwidth, fails when the internet is weak, and exposes private data.

Q2. Does Edge Computing replace the cloud?
No. It completes the cloud: the edge handles urgent local work, while the cloud handles heavy and long-term work.

Q3. What is the difference between SCALE and SCANC?
SCALE lists the advantages (why); SCANC lists the mechanisms (how).

Q4. Which layer of the hierarchy is closest to the user?
The edge layer (in the book, the outer edge).

Q5. Give one example each of pay-per-use and subscription.
Pay-per-use: paying per GPU-hour on a 5G edge server. Subscription: a fixed monthly fee for an edge platform.

Q6. Why is interoperability a challenge?
Devices from different makers use different protocols and data formats, so they cannot easily work together.

3.23.3 Exam Preparation

Marks	Likely questions
2 marks	Define edge device, edge server, edge node, fog node. What is cloud offloading? What is offline capability? What is a micro data center? What is a CDN? What is pay-per-use? What is device management? What is data ownership?
6 marks	Write short notes on 5G and Edge Computing. Explain how CDNs use the edge idea. Draw the data flow from a sensor to the cloud. Compare pay-per-use and subscription. Explain the features of a good edge platform. Write short notes on security challenges at the edge.
8 marks	A smart city wants to process data from thousands of sensors — explain the Cloud–Fog–Edge solution. A hospital deploys hundreds of edge devices — explain the security challenges and solutions. A company manages 10,000 edge devices — explain how orchestration helps.

Tip: Learn the one master diagram (the three-tier architecture) well. Most "draw" questions can be answered by relabelling it for the scenario — a wearable, a traffic camera or a farm sensor.

Chapter 4

Federating Edge Resources and Optimization in Fog Computing

4.1 Let's Start With Two Simple Questions

Think about this for a second:

1. Your mobile SIM is from one company, but when you travel to another state or country, your phone still works on another company's network. How is that possible?

2. A fog system has thousands of devices, hundreds of edge nodes and a few clouds. For one app, which part should run on which machine?

> **The answers in short:** (1) The mobile companies have **joined their networks together** by agreement — this is called **federation** (or roaming). Edge resources owned by many different companies need the same kind of joining. (2) There are millions of possible ways to place an app's parts. Choosing the *best* one — fastest, cheapest, using least energy — is an **optimization problem**.

In Chapter 3 we learnt what fog and edge computing are and why they complete the cloud. In this chapter we look at two harder, real-world questions:

- **Part A — Federation:** What problems appear when we try to join edge resources from many owners and regions into one big system?

- **Part B — Optimization:** How do we describe a fog system with mathematics, measure it, and find the best way to run applications on it?

This chapter follows two chapters of our reference book, Buyya and Srirama, *Fog and Edge Computing: Principles and Paradigms* (Wiley, 2019): "Addressing the Challenges in Federating Edge Resources" by Baktir, Sonmez, Ersoy, Ozgovde and Varghese, and "Optimization Problems in Fog and Edge Computing" by Zoltán Ádám Mann.

4.2 What Does "Federating Edge Resources" Mean?

Core Concept

To **federate** means to **join separate groups into one larger system** while each group keeps its own owner and control. **Federating edge resources** means connecting edge nodes owned by different companies, in different cities and countries, so that they work together as **one global edge layer** that any user or application can use.

Today's situation: Most edge resources are built *ad hoc* (for one purpose), used *privately* by one company, *not open to the public*, and *unevenly spread* — many in rich cities, few in villages and developing regions.

The goal: A **federated edge** where edge nodes from many owners form one shared fabric. Work that today goes to distant cloud data centers could then be handled by nearby edge nodes anywhere in the world — giving *equal access* to people in developed and developing regions alike.

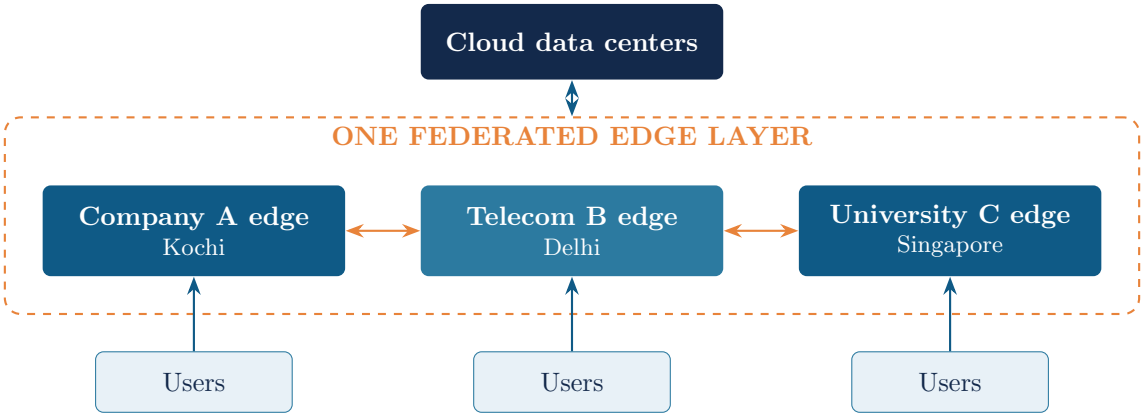


Figure 4.1: A federated edge: edge nodes owned by different organisations in different regions work together as one layer.

Real-life comparison: Think of ATMs. Your card is from one bank, but you can take money from almost any bank's ATM, because banks have agreed on common rules and a shared network. A federated edge needs the same kind of agreement — for computing instead of cash.

A note on the word "edge": In this chapter, "edge" is used as an **umbrella term** that covers mobile cloud computing, cloudlets, fog computing and multi-access edge computing (MEC).

4.3 Challenges in Federating Edge Resources

Federation sounds simple, but it is **not yet practical**. There are technical problems, and also **social, legal and geopolitical** issues (for example, different countries have different data laws). The book groups the technical problems into four families.

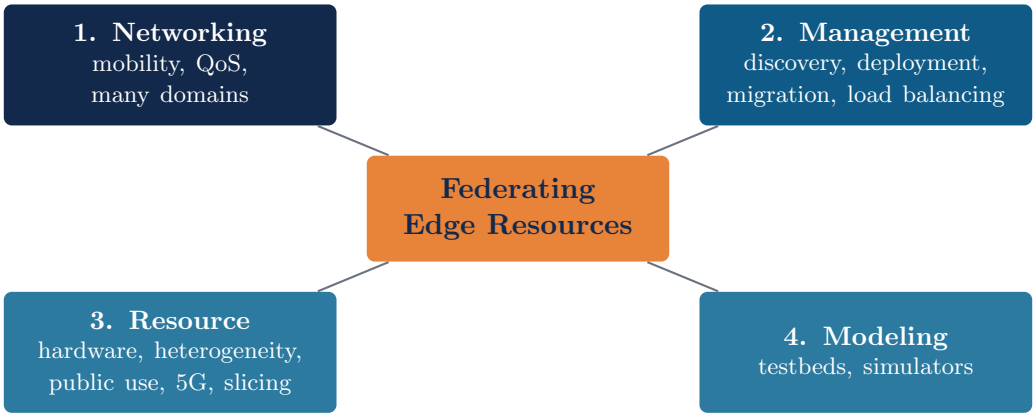


Figure 4.2: The four families of challenges in federating edge resources.

4.3.1 The Networking Challenge

The network must stay **dynamic** (devices and users keep changing) and keep good **Quality of Service (QoS)**, while **hiding all the complexity** from the user. The book lists five networking problems.

Challenge	Why it happens	What is needed
User mobility	Users keep moving between edge nodes (in cars, trains, while walking)	Smooth handover of the service to the next node
QoS in a dynamic environment	Load and network conditions change every moment	Constant monitoring and quick re-routing
Service-centric model	Users care about *what* service they need, not *where* the server is	Services identified by name/ID, with replicas in many places
Reliability and service mobility	Edge nodes join, leave or fail without warning	Fast replication or migration of services, with low overhead
Multiple administrative domains	Requests cross networks owned by different companies and countries	Global view plus local control in each domain

4.3.1.1 From "Where" to "What": The Service-Centric Model

Today's Internet is *host-centric*: to use a service you must reach a particular server address. In a federated edge, the same service may run on many edge nodes. So the network should work in a **service-centric** way — the user asks for "video analysis service", and the network finds the best copy. This needs unique service IDs, replicas and coordination, which the present Internet design does not support well.

Real-life comparison: When you book an auto-rickshaw through an app, you don't choose a particular driver — you ask for a ride, and the app finds the nearest free one. Service-centric networking does the same for computing.

4.3.1.2 Multiple Administrative Domains

A user in Kochi may be best served by a node owned by another company, or even in another country. The request may pass through several networks using different technologies. The book's key architectural idea is:

Core Concept

A federated edge needs a **logically centralised global view** (someone who sees the big picture) combined with **distributed local control** inside each administrative domain.

4.3.2 Addressing the Networking Challenge: Software-Defined Networking (SDN)

Core Concept

Software-Defined Networking (SDN) separates the **control plane** (the "brain" that decides where traffic should go) from the **data plane** (the switches and routers that simply forward packets). The brain becomes software running in an **SDN controller**, which has a **global view** of the network.

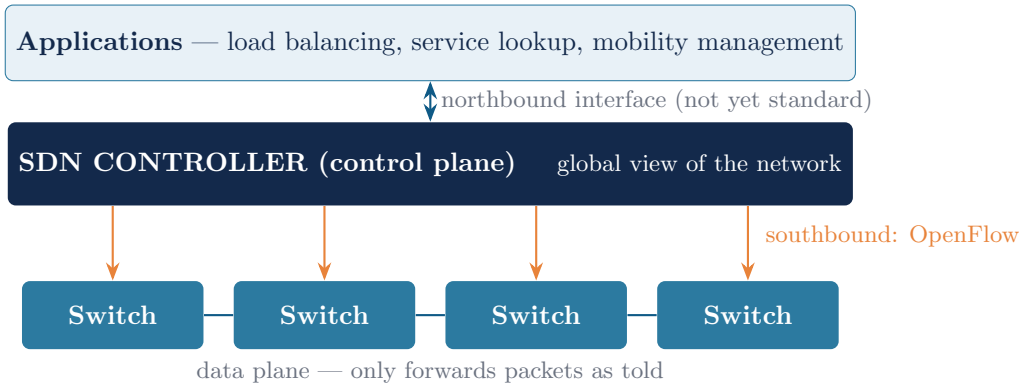


Figure 4.3: SDN: the controller (brain) is separated from the switches (muscles).

Key words:

- **Southbound interface** – how the controller talks *down* to switches. **OpenFlow** is the standard.
- **Northbound interface** – how applications talk *up* to the controller. This is not yet standardised.
- **East-west interface** – how *different controllers* talk to each other.
- **Flow rules** – instructions installed in switches: "send packets for service X to node Y".

4.3.2.1 How SDN Helps a Federated Edge

Problem	How SDN helps
Nodes joining and leaving	Topology discovery (OpenFlow Discovery Protocol, OFDP) detects new, removed or changed nodes and updates flow rules immediately

User mobility	Supports handover at the application layer when a user moves during offloading
Overloaded servers or congestion	Controller changes flows to send traffic to a less busy node — balancing both computing and network load
Service-centric access	A northbound app maps service IDs to locations; the controller picks the best copy and rewrites packet headers
Service mobility	Flow rules choose target nodes and migration paths that avoid congestion
Different vendors	Provides a common, interoperable control layer; less vendor dependency

4.3.2.2 Many Controllers for Many Domains

The original SDN idea used one controller for one campus. That cannot scale to a global edge. So each domain has its own controller, and controllers cooperate through the **east-west interface**, in one of two ways:

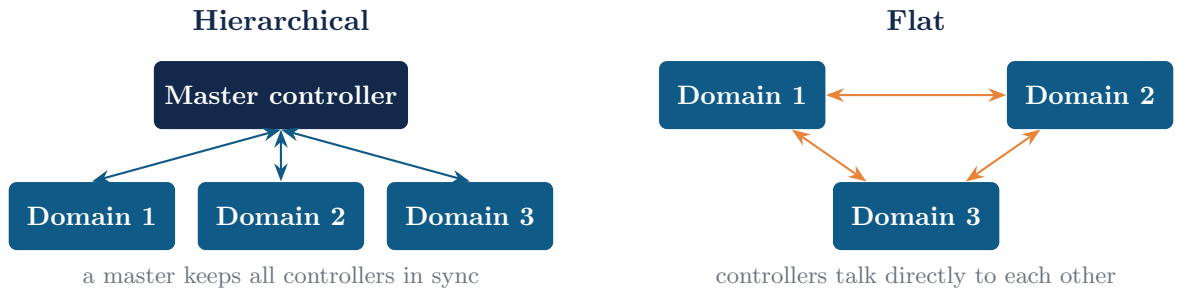


Figure 4.4: Hierarchical and flat arrangements of SDN controllers across administrative domains.

4.3.2.3 Future Research Directions (Networking)

1. **Wireless SDN** – most future edge users will be mobile, so SDN and OpenFlow must work well on wireless networks.
2. **Standard interfaces** – standardise the northbound interface; make east-west communication reliable and efficient.
3. **More programmability** – OpenFlow offers only partial programmability today.
4. **Wide-area scalability** – distributed SDN control across many administrative domains.

4.3.3 The Management Challenge

Adding an edge layer between users and the cloud adds **more things to manage** — and joining clusters across regions makes it harder still. The book identifies four management challenges.

Challenge	Meaning	Why it is hard at the edge
Discovery	Finding which edge nodes exist and are available	Nodes must be visible to devices, to the cloud and to each other — one by one and as groups
Deployment	Placing a workload on a suitable edge node	Workloads must match each node's abilities; many nodes must be checked (benchmarked) in real time

Migration	Moving a running service from one node to another	VMs and even containers are heavy for small edge nodes; network paths also matter
Load balancing	Scaling busy services up and idle ones down	Monitoring itself uses scarce resources on small devices

Discovery can be *self-initiated* (each node announces itself — loosely coupled), *externally monitored* (a manager watches all nodes — tightly coupled), or a mix of both.

4.3.3.1 What Current Research Offers

- **Discovery:** FocusStack works for a single owner; ParaDrop and Edge-as-a-Service (EaaS) support many tenants. *Federated* discovery across owners is still unsolved.
- **Deployment:** Most work is about provisioning *before* deployment; support *after* deployment for small, fine-grained workloads is still needed.
- **Migration:** Moving VMs between clusters is slow — live migration can take minutes. **Containers** are a lighter, promising alternative.
- **Monitoring and auto-scaling:** Existing monitoring tools are too heavy or do not scale; lightweight auto-scaling has been shown on single nodes.

4.3.3.2 Example 1: The Edge-as-a-Service (EaaS) Platform

EaaS tackles **discovery** using a lightweight protocol, tested on Raspberry Pi edge nodes.

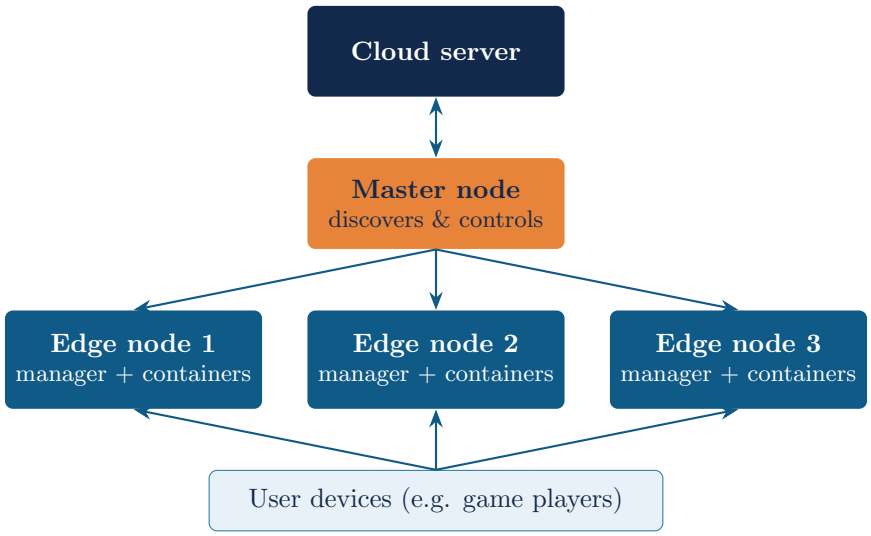


Figure 4.5: The EaaS platform: a master node discovers edge nodes, installs managers on them and runs containers.

- Three tiers: cloud, edge nodes, user devices. A **master node** installs a small manager on each edge node.
- Services run in **Docker or LXD containers**; launching, starting, stopping or terminating a container takes only a few seconds.
- Tested with an online game similar to Pokémon Go, running up to 50 containers on one node.

- **Limits:** depends on a central master, assumes a common marketplace, and does not yet secure remote commands.

4.3.3.3 Example 2: The ENORM Framework

ENORM (Edge NOde Resource Management) tackles **deployment** and **load balancing** on individual edge nodes — without needing a master.

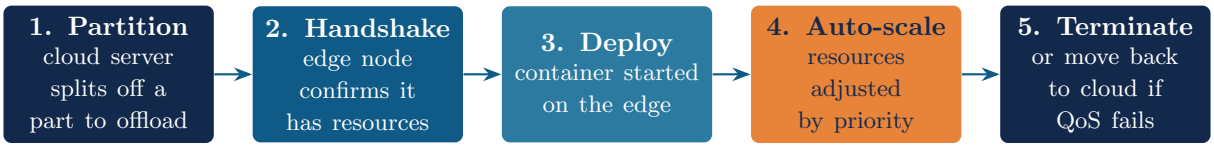


Figure 4.6: The ENORM life cycle for a service offloaded from the cloud to an edge node.

- Because the edge node may also be a router or base station, its **basic job (routing) must keep its QoS**; other services are given priorities.
- A service that cannot meet its QoS at the edge is sent **back to the cloud**.
- **Results (online game):** application latency reduced by **20–80%**, and data sent to the cloud reduced by up to **95%**.

4.3.3.4 Future Research Directions (Management)

1. **Coordinating different kinds of nodes** – routers, base stations, switches and low-power devices with different processors, using standard protocols.
2. **Real-time benchmarking** – helping the cloud quickly find edge nodes that can meet a service's Service Level Objectives (SLOs).
3. **Rapid migration** – moving functions or programs instead of whole VM or container snapshots (suits serverless computing).
4. **Fine-grained auto-scaling** – giving exactly the resources an app needs, instead of fixed-size units, without destabilising the node.

4.3.4 Miscellaneous Challenges: Resources and Modeling

4.3.4.1 Resource Challenges

1. **Deploying edge nodes** – Should edge nodes be routers with extra computing (edge-enabled gateways), dedicated micro clouds, or both? Upgrading existing routers everywhere may take a long time.
2. **One architecture for very different hardware** – software and middleware must run on everything from home routers to micro clouds; today's virtualization does not suit all of them.
3. **Public usability** – how will the public access edge nodes, and how will auditing, billing, security and privacy work?
4. **Working with future networks** – Wi-Fi may give way to 5G; ETSI MEC is part of 5G, while "over-the-top" (OTT) providers would work differently. Each choice shapes federation differently.
5. **Network slicing** – slices can give each service guaranteed QoS, but very fine slices are

hard to scale; federated slices may need global adjustment.

Pricing is another open question: how should owners charge each other and users for shared edge resources?

4.3.4.2 Modeling Challenges

Before building a global edge, researchers must **test ideas**. There are two ways:

Feature	Testbed (real hardware)	Simulator (software model)
Example	Living Edge Lab	EdgeCloudSim, iFogSim
Realism	Very high	Depends on the model
Cost	Expensive	Cheap
Scale	Hard to make very large	Thousands of devices easily
Repeatability	Hard	Easy — run the same test again

A good edge simulator must model:

1. **Computational resources** – VMs and containers, including resizing and migration.
2. **Demand** – many kinds of devices, their traffic, offloading and arrival patterns.
3. **Mobility** – users moving between edge servers, causing service migration or replication.
4. **Networks** – delays over Wi-Fi, Bluetooth and cellular links, and network slicing.
5. **Efficiency** – handle huge numbers of IoT and M2M devices while staying easy to use.

Core Concept

Federating edge resources is difficult. Besides networking, management, resource and modeling problems, there are social and legal issues — and the technologies are still young and changing fast. Solving these is what will turn many private edges into one public, global edge.

4.4 The Case for Optimization in Fog Computing

Why fog computing needs careful, mathematical decision-making

4.4.1 First, What Is Optimization?

Core Concept

Optimization means choosing the **best solution** from all possible solutions, according to a chosen goal, while obeying all the rules (constraints).

Every optimization problem has four parts:

Part	Meaning	Everyday example: going to college
Input	The facts we are given	Distance, bus times, auto fares, your budget

Decision variables	What we are free to choose	Bus, auto or bike; which route
Constraints	Rules every solution must obey	Must reach before 9 am; spend at most Rs. 50
Objective function	The value we want to minimise or maximise	Minimise travel time (or cost)

A solution that obeys all constraints is called **feasible**. The feasible solution with the best objective value is called **optimal**. Sometimes there are several goals at once (fast *and* cheap) — this is **multi-objective optimization**, and usually we must accept a **trade-off**.

4.4.2 Why Does Fog Computing Need Optimization?

In the cloud alone, the main question was "which server in the data center should run this VM?". Fog computing makes decisions much harder because its resources are:

Property of fog	Why it makes decisions harder
Heterogeneous	Clouds, edge servers, routers, phones and sensors all have very different power
Resource-constrained	End devices and edge nodes have little CPU, memory and battery
Geographically distributed	Many nodes in many places; distance changes delay
Dynamic	Users move, nodes join and leave, load and network quality change every second
Multi-owner	Different parts belong to different providers with different prices
Conflicting goals	Faster response may cost more money or energy

The central question of fog optimization: *What should be done where, and when?* — which part of which application should run on the device, on which edge node, or in which cloud, so that performance, energy, cost and other goals are balanced.

4.4.3 A Worked Example: To Offload or Not?

A phone must run a photo-recognition task of **1,000 million CPU cycles**. The photo is **2 MB** (= 16 megabits).

- **Phone:** CPU speed 1 GHz (1,000 million cycles per second); uses 0.9 W while computing.
- **Edge server:** 10 GHz; the phone uses 1.2 W while sending data.

Option 1 — run on the phone:

$$\text{time} = \frac{1000}{1000} = 1.0 \text{ s}, \qquad \text{energy} = 0.9 \text{ W} \times 1.0 \text{ s} = 0.90 \text{ J}$$

Option 2 — offload to the edge over a 20 Mbps link:

$$\text{send} = \frac{16}{20} = 0.8 \text{ s}, \quad \text{compute} = \frac{1000}{10{,}000} = 0.1 \text{ s}, \quad \text{time} = 0.9 \text{ s}$$

$$\text{phone energy} = 1.2 \text{ W} \times 0.8 \text{ s} = 0.96 \text{ J}$$

Option 3 — offload to the edge over a 50 Mbps link:

$$\text{send} = \frac{16}{50} = 0.32 \text{ s}, \quad \text{time} = 0.32 + 0.1 = 0.42 \text{ s}, \quad \text{energy} = 1.2 \times 0.32 \approx 0.38 \text{ J}$$

Option	Time	Phone energy	Best for
Run on phone	1.00 s	0.90 J	Works offline
Edge, 20 Mbps	0.90 s	0.96 J	Slightly faster, but uses more battery
Edge, 50 Mbps	0.42 s	0.38 J	Faster *and* saves battery

Core Concept

The "best" choice **depends on the situation** — network speed, device power, server load and the goal. Since these change all the time, fog systems need **systematic optimization**, not guesswork.

4.5 Formal Modeling Framework for Fog Computing

Describing a fog system with simple mathematics

4.5.1 Why Do We Need a Formal Model?

Many researchers study fog optimization, but each paper used to describe the problem in its own way. A **formal modeling framework** gives everyone a **common language**: the same symbols for resources, applications and decisions, so that problems can be stated precisely, compared, and solved with known algorithms.

Real-life comparison: Before a building is constructed, the architect draws a plan with standard symbols. Any engineer can read it. A formal model is the "plan" of a fog optimization problem.

4.5.2 The Fog Infrastructure as Layers

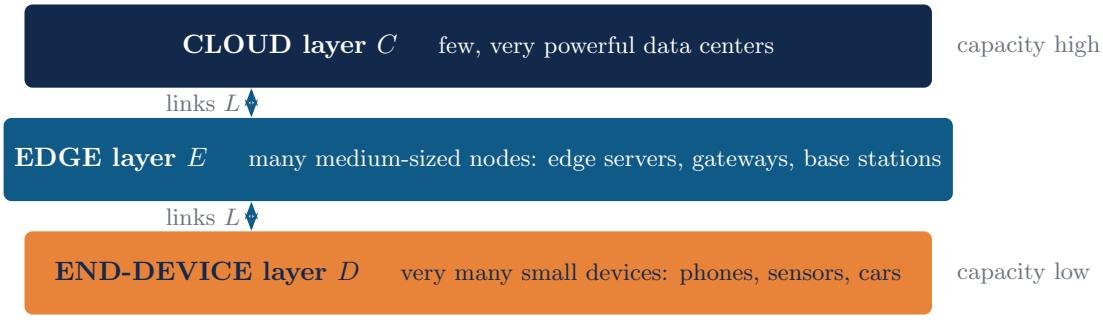


Figure 4.7: The fog infrastructure as three layers of resources connected by links.

4.5.3 The Building Blocks of the Model

Symbol	Name	Meaning
$R = C \cup E \cup D$	Resources	All machines: cloud nodes, edge nodes and end devices
$cap(r)$	Capacity	How much a resource r can offer (CPU, memory, storage)
L	Links	Network connections between resources
$lat(\ell),\ bw(\ell)$	Latency, bandwidth	Delay and data rate of a link ℓ
A	Applications	The apps that must run on the fog
S_a	Components	The parts (services) of application a
$dem(s)$	Demand	Resources needed by component s
x	Placement (decision)	$x(s) = r$ means "run component s on resource r"

An application is often drawn as a small **graph**: components are boxes, and arrows show which component sends data to which.

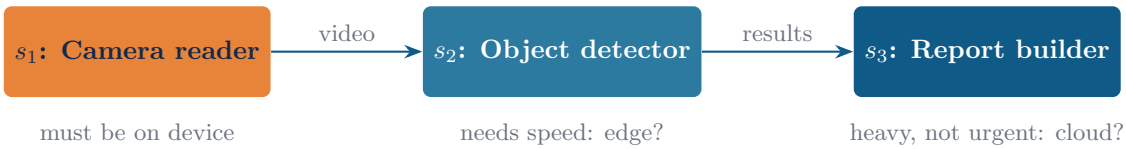


Figure 4.8: An application as a graph of components. The optimization decides where each component runs.

4.5.4 The Optimization Problem in Words and Symbols

Decide: a placement x that maps every component s to a resource r.

Subject to (constraints):

1. **Capacity:** the total demand of components placed on a resource cannot exceed its capacity:
$$\sum_{s\,:\,x(s)=r} dem(s) \;\le\; cap(r) \qquad \text{for every resource } r$$
2. **Latency:** data between two connected components must arrive within the allowed time (for example, ≤ 50 ms).
3. **Bandwidth:** data flowing over a link cannot exceed its bandwidth.
4. **Placement rules:** some components must run in certain places (a camera reader on the device; sensitive data inside the country).

Optimise (objective): minimise (or maximise) a chosen **metric** — total delay, energy, cost — or a weighted combination of several metrics.

4.5.5 Worked Example: Placing Three Components

Three edge nodes have free CPU capacity: **E1 = 4 units, E2 = 2 units, E3 = 3 units**. Three components need: **A = 3, B = 2, C = 2** units. Latency from the users to each node: E1 = 5 ms, E2 = 8 ms, E3 = 12 ms. Goal: **minimise the total latency**.

Step 1 — Try putting everything on the closest node, E1: demand $3 + 2 + 2 = 7 > 4$. **Not feasible.**

Step 2 — Try a sensible placement:

Placement	E1 (cap 4)	E2 (cap 2)	E3 (cap 3)	Latency
A→E1, B→E2, C→E3	3 ✓	2 ✓	2 ✓	25 ms
A→E3, B→E1, C→E1	4 ✓	0	3 ✓	**22 ms**
A→E3, B→E1, C→E2	2 ✓	2 ✓	3 ✓	25 ms

Answer: put **B and C on E1** and **A on E3**: total latency **22 ms**, all capacities respected. Notice that the "obvious" idea — put the biggest component on the closest node — was *not* the best. With thousands of components and nodes, we cannot check by hand; we need algorithms.

4.6 Metrics

How do we measure whether a solution is good?

Core Concept

A **metric** is a measurable quantity used to judge a solution — it becomes the **objective** we optimise or a **constraint** we must satisfy. The four main families of metrics in fog computing are **performance**, **resource usage**, **energy consumption** and **financial cost**.

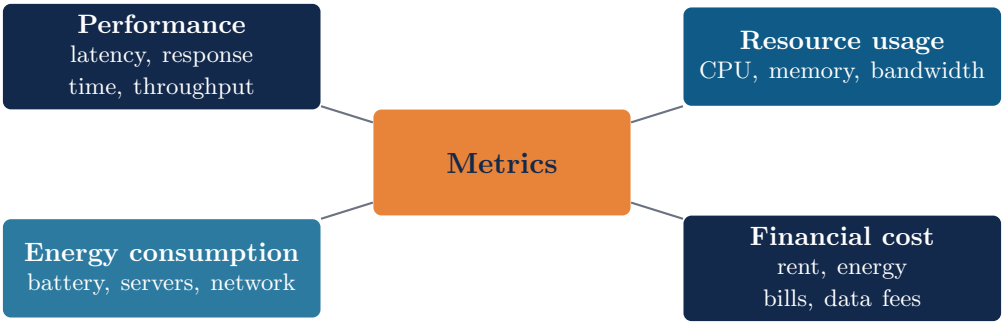


Figure 4.9: The four main families of metrics in fog computing.

4.6.1 Performance

Performance tells us **how fast** the system works. Common measures:

- **Latency** – delay for data to travel from one point to another.
- **Response time** – total time from a request until the answer arrives.
- **Throughput** – how many requests (or how much data) are handled per second.
- **Deadline misses** – how many tasks finish late.

Response time is built from several parts:

$$\text{response time} = \text{transmission} + \text{propagation} + \text{queuing} + \text{processing}$$

Part	Meaning	Reduced by
Transmission	Time to push the data onto the link = data size / bandwidth	Faster links, less data
Propagation	Time for the signal to travel the distance	Nearer servers (edge)
Queuing	Waiting because the server is busy	Load balancing, more capacity
Processing	Time to actually compute = work / CPU speed	Faster CPUs, accelerators

4.6.2 Resource Usage

Resource usage tells us **how much** of the infrastructure a solution uses — CPU, memory, storage and network bandwidth. Good solutions avoid both **overloading** (slow, risky) and **under-use** (waste).

$$\text{utilisation of a resource} = \frac{\text{used capacity}}{\text{total capacity}} \times 100\%$$

Example: An edge node with 8 CPU cores runs services using 6 cores: utilisation $= 6/8 = 75\%$. Network usage is also important in fog — sending less data over the core network is one of the main reasons for using the edge.

4.6.3 Energy Consumption

Energy matters in three places: **battery-powered end devices**, **edge and cloud servers** (electricity and cooling), and the **network** that carries the data.

$$\text{energy (joules)} = \text{power (watts)} \times \text{time (seconds)}$$

Example: A sensor transmitting at 0.5 W for 2 seconds uses $0.5 \times 2 = 1$ J. If it sends only summaries and transmits for 0.2 s instead, it uses 0.1 J — ten times less, and the battery lasts much longer. Energy is often in **conflict** with performance: running everything on powerful servers is fast, but uses more energy.

4.6.4 Financial Cost

Financial cost is the **money** spent: renting cloud and edge resources, buying and running equipment, electricity, network and data-transfer fees, and software licences.

$$\text{total cost} = \sum_{\text{resources}} \text{price per unit} \times \text{units used}$$

Example: Running a component in the cloud costs Rs. 4 per hour; on a nearby edge node Rs. 7 per hour. The edge is costlier but faster — so the optimizer may use the edge only for latency-critical components.

4.7 Further Quality Attributes

Beyond speed, resources, energy and money

Besides the four main metrics, a fog solution must also be judged on other qualities. They are often harder to measure as a single number, so they are usually written as **constraints** or as extra objectives.

Attribute	Meaning	Example in fog
Reliability	The system keeps working correctly without failures	Run a backup copy of a health-alert service on a second edge node
Availability	The share of time the service is up	99.9% uptime for a traffic-control service
Security	Protection from attacks and unauthorised access	Place sensitive components only on trusted, verified nodes
Privacy	Personal data is used only as allowed	Keep raw patient data on hospital edge nodes; send only summaries to the cloud
Scalability	Keeps working well as users and devices grow	Placement still works when devices grow from 100 to 10,000
Fault tolerance	Survives the failure of a node or link	Automatically restart services elsewhere when an edge node fails
SLA compliance	Meets the promises in the service agreement	Response time under 100 ms for 99% of requests
Data quality & freshness	Data is accurate and recent	Sensor readings processed within 1 second

Example — reliability as a constraint: Suppose each edge node is available 99% of the time and fails independently. If a critical service runs on one node, it is up 99% of the time. If it runs on two nodes (one backup):

$$\text{availability} = 1 - (1 - 0.99)^2 = 1 - 0.0001 = 0.9999 = 99.99\%$$

The optimizer can therefore add a rule: "critical services must have at least two copies on different nodes" — which uses more resources, showing again the **trade-off** between quality and cost.

4.8 Optimization Opportunities along the Fog Architecture

Where in the fog hierarchy can we optimise?

Optimization can happen **inside each layer** of the fog architecture, and **between layers**. Each place has its own typical problems.

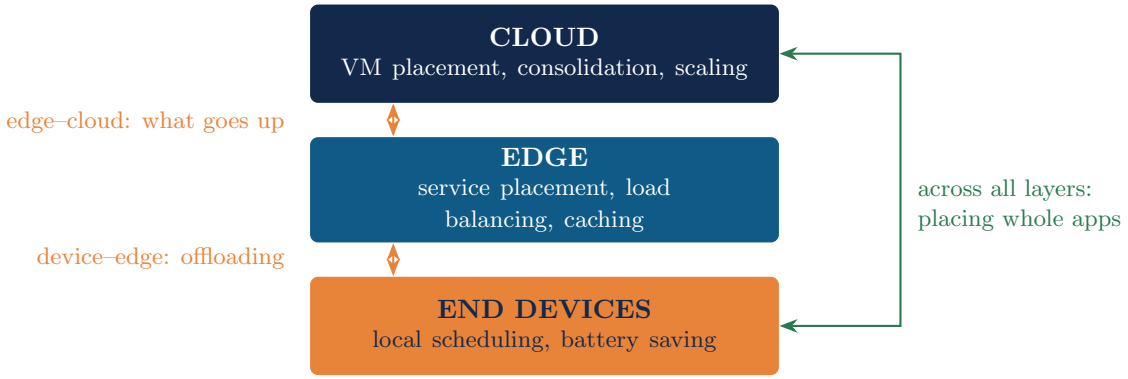


Figure 4.10: Optimization opportunities inside each layer of the fog architecture and between layers.

4.8.1 Inside Each Layer

Layer	Typical optimization problems	Main goals
Cloud	Which physical server runs which VM; packing VMs onto fewer servers (consolidation); auto-scaling	Low energy and cost, meet SLAs
Edge	Which edge node runs which service; balancing load among nearby nodes; what data to cache; where to place new edge nodes	Low latency, avoid overloading small nodes
End devices	Scheduling tasks on the device; turning radios and sensors on and off; choosing how often to send data	Save battery, respond quickly

4.8.2 Between Layers

Interaction	Optimization question	Example
End device ↔ edge	Should a task run on the device or be **offloaded** to an edge node? To which node?	A phone offloads face recognition to the nearest edge server
Edge ↔ cloud	Which work stays at the edge and which goes to the cloud? How much data is sent up?	Edge filters video; cloud trains AI models
Device ↔ cloud	Is a direct trip to the cloud acceptable for this task?	Weekly backup goes straight to the cloud
Across all layers	Where should each **component of an application** run in the whole hierarchy?	Camera reader on device, detector on edge, reports in cloud

Key idea: Optimization problems inside the cloud were studied for many years. Fog computing adds *new* problems — especially **offloading** between devices and edge, and **placing applications across all layers** — where distance, small capacities, mobility and battery limits all matter at once.

4.9 Optimization Opportunities along the Service Life Cycle

When can we optimise?

A service passes through three main phases in its life. Different decisions — and different optimization problems — belong to each phase.

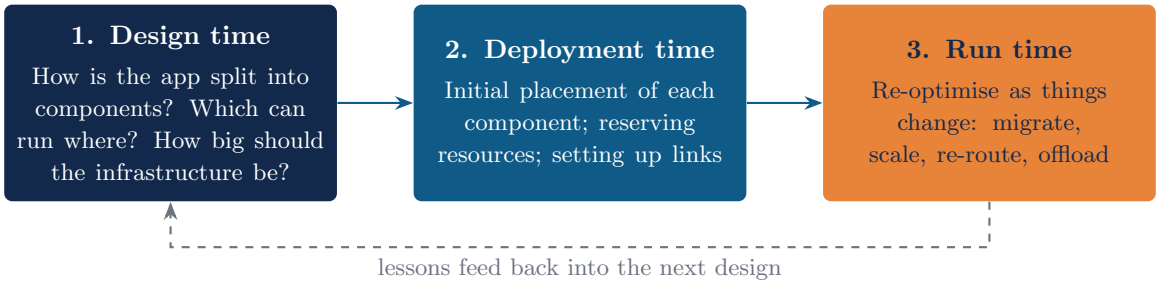


Figure 4.11: Optimization opportunities along the service life cycle: design time, deployment time and run time.

4.9.1 Design Time

Decisions made **before** the service exists.

- **Application partitioning** – splitting the app into components that can run on different machines.
- **Placement rules** – deciding which components *may* run on devices, edge or cloud (for example, privacy rules).
- **Infrastructure planning** – how many edge nodes, of what size, and where to place them (for example, which base stations should get edge servers).

Example: A city plans its smart-traffic system. It decides to split the software into "camera reader", "vehicle counter" and "city report", and to install edge servers at its 20 busiest junctions.

4.9.2 Deployment Time

Decisions made when the service is **first installed**.

- **Initial placement** – mapping each component to a resource (the formal problem we saw earlier).
- **Resource reservation** – how much CPU, memory and bandwidth each component gets.
- **Network set-up** – routes and flow rules between components.

Example: When the traffic system goes live, the optimizer places each junction's vehicle counter on its nearest edge server and the city report in the cloud.

4.9.3 Run Time

Decisions made **while the service is running**, as conditions change.

- **Migration** – moving a component to another node (for example, following a moving user).
- **Scaling** – adding or removing copies as load changes.
- **Dynamic offloading** – deciding task by task whether to compute locally or remotely.
- **Load balancing and re-routing** – moving traffic away from busy nodes or links.
- **Recovery** – restarting services elsewhere after a failure.

Example: During a festival, one junction's edge server becomes overloaded. The run-time optimizer moves part of its work to a neighbouring junction's server.

Feature	Design time	Deployment time	Run time
When	Before building	At installation	While running
How often	Once (or rarely)	Once per deployment	Continuously
Time to decide	Days or weeks	Minutes	Seconds or less
Typical problem	Partitioning, infrastructure sizing	Initial placement	Migration, scaling, offloading
Solution method	Exact, thorough methods	Exact or heuristic	Fast heuristics

Core Concept

The **earlier** the phase, the more time we have to find a very good solution. The **later** the phase, the faster the decision must be — so run-time optimization uses quick, "good enough" methods.

4.10 How Are Fog Optimization Problems Solved?

Most fog placement problems are **NP-hard**: as the number of components and nodes grows, the number of possible placements explodes. With just 10 components and 10 nodes there are 10^{10} (ten billion) possible placements. So different methods are used:

Method	Idea	When to use
Exact methods (e.g. integer linear programming)	Guaranteed best answer	Small problems; design time
Greedy heuristics (e.g. first-fit, nearest node first)	Make the best local choice step by step	Run time; need fast answers
Metaheuristics (genetic algorithms, simulated annealing, particle swarm)	Smart search that improves solutions gradually	Large problems; good, not perfect answers
Game theory and auctions	Each owner or user acts for itself; find a fair balance	Multi-owner, federated edges
Machine learning	Learn good decisions from past data	Changing, repeating patterns

4.11 Putting It All Together

A federated smart-city video service. Cameras across a city send video to be analysed for accidents.

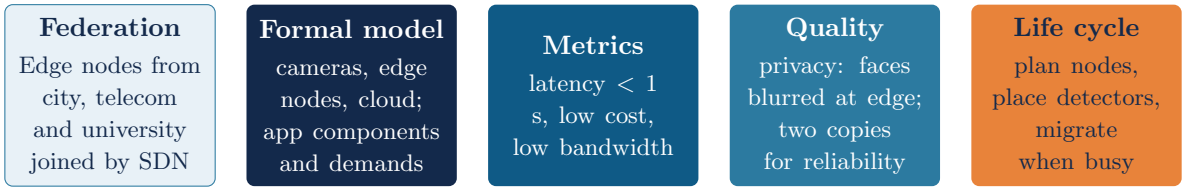


Figure 4.12: All the ideas of this chapter working together in one fog application.

The big picture: *Federation* gives us a large, shared pool of edge resources. The *formal model* describes them and the application precisely. *Metrics* and *quality attributes* say what "good" means. The *fog architecture* and the *service life cycle* tell us where and when to optimise. Optimization algorithms then find the best placement — and keep improving it as conditions change.

4.12 Quick Revision Summary

- **Federating edge resources** = joining edge nodes of many owners and regions into one global edge layer.
- **Four challenge families:** networking, management, resource, modeling (plus pricing, social, legal, geopolitical).
- **Networking challenges:** user mobility, QoS in a dynamic environment, service-centric model, reliability and service mobility, multiple administrative domains. **Solution:** SDN (controller + OpenFlow; northbound, southbound, east-west; hierarchical or flat controllers).
- **Management challenges:** discovery, deployment, migration, load balancing. **Examples:** EaaS (discovery, containers), ENORM (deployment and auto-scaling; latency −20–80%, data to cloud −95%).
- **Resource challenges:** deploying nodes, heterogeneity, public use, 5G/MEC, network slicing. **Modeling:** testbeds vs simulators (EdgeCloudSim, iFogSim).
- **Optimization** = best feasible solution: input, decision variables, constraints, objective.
- **Why fog needs it:** heterogeneous, constrained, distributed, dynamic, multi-owner, conflicting goals — "what should run where?".
- **Formal model:** resources $R = C \cup E \cup D$ with capacities; links with latency and bandwidth; applications as component graphs; placement $x(s) = r$; capacity, latency, bandwidth and placement constraints.
- **Metrics:** performance, resource usage, energy, financial cost. **Further quality attributes:** reliability, availability, security, privacy, scalability, fault tolerance, SLA compliance.
- **Along the architecture:** inside cloud, edge, devices; between layers (offloading); across all layers (app placement).
- **Along the life cycle:** design time, deployment time, run time.

4.13 Likely Exam Questions and Short Answers

Marks	Question
2 marks	What is federation of edge resources? What is a service-centric model? What is SDN? What is OpenFlow? What is the east-west interface? What is ENORM? What is optimization? What is a feasible solution? What is a metric? Name two further quality attributes.
6 marks	Explain the networking challenges in federating edge resources. Explain how SDN addresses them. Explain the management challenges. Explain the formal modeling framework. Explain the metrics used in fog optimization.
8 marks	Discuss the challenges in federating edge resources and the solutions proposed. Make the case for optimization in fog computing with an example. Explain the optimization opportunities along the fog architecture and along the service life cycle.

Model short answers (2 marks):

- **Federating edge resources:** joining edge nodes owned by different organisations and regions into one shared, globally usable edge layer.
- **Service-centric model:** the network delivers a requested *service* from the best available copy, instead of a fixed server address.
- **SDN:** a networking approach that separates the control plane (software controller) from the data plane (forwarding switches).
- **OpenFlow:** the standard southbound protocol used by an SDN controller to install flow rules in switches.
- **East-west interface:** the interface through which SDN controllers of different domains communicate.
- **ENORM:** an edge node resource management framework for deploying and auto-scaling services offloaded from the cloud to edge nodes.
- **Optimization:** choosing the best solution, according to an objective, among all solutions that satisfy the constraints.
- **Feasible solution:** a solution that satisfies all the constraints.
- **Metric:** a measurable quantity — such as latency, energy or cost — used to judge a solution.
- **Further quality attributes:** reliability, availability, security, privacy (any two).

Tip for the 8-mark answer

Follow this order: **define** the main term → **draw the diagram** (four challenge families, SDN layers, three fog layers, or the three life-cycle phases) → **explain each part** with one example → **give the solution or opportunity** (SDN, EaaS, ENORM, offloading, migration) → **mention the trade-off** (speed vs energy vs cost) → **one-line conclusion**.

4.14 Practice Section

4.14.1 Group Problem: Where Should the Video Analysis Run?

A traffic camera produces a video clip of **40 megabits** every 10 seconds. Detecting accidents needs **4,000 million CPU cycles** per clip. The result must be ready within **2 seconds**.

Option	Link speed	CPU speed	Cost per hour
Camera's own chip	—	1 GHz	Rs. 0
Junction edge server	100 Mbps	8 GHz	Rs. 7
Cloud	20 Mbps (+ 0.1 s round trip)	40 GHz	Rs. 4

Step 1 — Time on the camera:

$$\frac{4000}{1000} = 4 \text{ s} \quad > 2 \text{ s} \Rightarrow \textbf{not feasible}$$

Step 2 — Time on the edge:

$$\text{send} = \frac{40}{100} = 0.4 \text{ s}, \quad \text{compute} = \frac{4000}{8000} = 0.5 \text{ s}, \quad \text{total} = 0.9 \text{ s} \Rightarrow \textbf{feasible}$$

Step 3 — Time in the cloud:

$$\text{send} = \frac{40}{20} = 2.0 \text{ s}, \quad \text{compute} = \frac{4000}{40{,}000} = 0.1 \text{ s}, \quad \text{total} = 2.0{+}0.1{+}0.1 = 2.2 \text{ s} \Rightarrow \textbf{not feasible}$$

Step 4 — Decide: Only the edge meets the 2-second deadline. Even though the cloud is cheaper per hour, it breaks the constraint. **Answer: run the analysis on the junction edge server.**

Step 5 — Think further:

- *Metric:* latency was a constraint; cost was the objective among feasible options.
- *Quality attribute:* for reliability, keep a second copy on a neighbouring junction's server.
- *Architecture:* this is a device–edge offloading decision.
- *Life cycle:* decided at deployment time; at run time, if the edge server gets overloaded, migrate to the neighbour.

Check: Always test *constraints first* (is it feasible?), then compare the *objective* (which feasible option is best?).

4.14.2 Check Your Understanding

Q1. Why is federating edge resources not yet practical?
Because of networking, management, resource and modeling challenges, and also social, legal and geopolitical issues.

Q2. What does an SDN controller do?
It acts as the network's brain, keeping a global view and installing flow rules in switches to decide where traffic goes.

Q3. Name the four management challenges.
Discovery, deployment, migration and load balancing.

Q4. What is the difference between a constraint and an objective?
A constraint is a rule every solution must obey; the objective is the value we try to make as small or as large as possible.

Q5. Why do run-time decisions use fast heuristics?
Because conditions change every second, and there is no time to search for the perfect answer.

Q6. Give one optimization opportunity between end devices and the edge.
Deciding whether to run a task on the device or offload it to an edge node, and to which node.

4.14.3 Exam Preparation

Marks	Likely questions
2 marks	Define discovery, deployment, migration and load balancing in a federated edge. What is EaaS? What is a testbed? Name two edge simulators. What is NP-hard? Define energy consumption as a metric. What is application partitioning?
6 marks	Compare hierarchical and flat SDN controllers. Explain the EaaS platform. Explain ENORM. Explain resource and modeling challenges. Explain further quality attributes with examples. Compare design-time, deployment-time and run-time optimization.
8 marks	"Federating edge resources is difficult" — discuss with networking and management challenges and solutions. Explain the formal modeling framework for fog computing with a placement example. Explain optimization opportunities along the fog architecture and the service life cycle.

Tip: Learn four diagrams — the **four challenge families**, the **SDN layers**, the **three fog layers with optimization opportunities**, and the **three life-cycle phases**. Together they answer most questions in this module.

References and Further Reading

Textbooks

1. R. Buyya and S. N. Srirama (eds.), *Fog and Edge Computing: Principles and Paradigms*, Wiley, 2019. Chapters used in this book:

   - C. Chang, S. N. Srirama and R. Buyya, "Internet of Things (IoT) and New Computing Paradigms" — Chapter 3 (preprint: arXiv:1812.00591).

   - C. Baktir, C. Sonmez, C. Ersoy, A. Ozgovde and B. Varghese, "Addressing the Challenges in Federating Edge Resources" — Chapter 4 (preprint: arXiv:1803.05255).

   - Z. Á. Mann, "Optimization Problems in Fog and Edge Computing" — Chapter 4.

2. K. Hwang, G. C. Fox and J. J. Dongarra, *Distributed and Cloud Computing: From Parallel Processing to the Internet of Things*, Morgan Kaufmann, 2012 — standard reference for virtualization, virtual clusters and data-center automation.

3. R. Buyya, C. Vecchiola and S. T. Selvi, *Mastering Cloud Computing*, McGraw Hill, 2013 — standard reference for cloud architecture, service models and management.

Standards and Reports

1. P. Mell and T. Grance, *The NIST Definition of Cloud Computing*, NIST Special Publication 800-145, 2011.

2. F. Liu et al., *NIST Cloud Computing Reference Architecture*, NIST Special Publication 500-292, 2011 — cloud roles (consumer, provider, broker, carrier, auditor).

3. ETSI, *Multi-access Edge Computing (MEC)* specifications — etsi.org.

4. IEEE 1934-2018, *Standard for Adoption of OpenFog Reference Architecture for Fog Computing.*

Research Papers

1. A. Yousefpour et al., "All one needs to know about fog computing and related edge computing paradigms: A complete survey," *Journal of Systems Architecture*, 2019.

2. N. Wang, B. Varghese, M. Matthaiou and D. S. Nikolopoulos, "ENORM: A Framework for Edge NOde Resource Management," *IEEE Transactions on Services Computing*, 2017.

3. C. Sonmez, A. Ozgovde and C. Ersoy, "EdgeCloudSim: An environment for performance evaluation of edge computing systems," *Transactions on Emerging Telecommunications*

Technologies, 2018.

4. H. Gupta, A. V. Dastjerdi, S. K. Ghosh and R. Buyya, "iFogSim: A toolkit for modeling and simulation of resource management techniques in IoT, edge and fog computing environments," *Software: Practice and Experience*, 2017.

Websites and Documentation

1. Amazon Web Services documentation (EC2, S3, Elastic Beanstalk, IoT Greengrass, CloudWatch) — docs.aws.amazon.com

2. Microsoft Azure documentation (Virtual Machines, App Service, IoT Edge, Azure Monitor) — learn.microsoft.com

3. Google Cloud documentation (Compute Engine, App Engine, Distributed Cloud Edge) — cloud.google.com/docs

4. EdgeX Foundry, LF Edge (Linux Foundation) — edgexfoundry.org

5. KubeEdge, Cloud Native Computing Foundation — kubeedge.io; CNCF graduation announcement, October 2024 — cncf.io

6. Open Networking Foundation, OpenFlow and SDN — opennetworking.org

7. Digital Personal Data Protection Act, 2023, Government of India — meity.gov.in

Product names, versions and service details change quickly; always check the official documentation for the latest information.